**TRƯỜNG ĐẠI HỌC SÀI GÒN**
**KHOA CÔNG NGHỆ THÔNG TIN**




# BÁO CÁO ĐỒ ÁN HỌC PHẦN
# CÔNG NGHỆ PHẦN MỀM

**ĐỀ TÀI:**

# HỆ THỐNG ĐẶT CÀ PHÊ TRỰC TUYẾN BREWLITE

**Giảng viên hướng dẫn:** Từ Lãng Phiêu

**Sinh viên thực hiện:**

| | |
|---|---|
| Phạm Minh Chiến – 3124410027 | (Trưởng nhóm – Auth) |
| Đinh Nhật Lan – 3124410182 | (Thành viên – Catalog) |
| Vũ Hoàng Huy – 3124410125 | (Thành viên – Detail) |
| Quách Tuấn Hào – 3124560029 | (Thành viên – Cart) |
| Tôn Thất Hạnh – 3124410079 | (Thành viên – Order) |
| Nguyễn Trần Gia Kiệt – 3124410176 | (Thành viên – Payment) |
| Nguyễn Quang Huy – 3124560034 | (Thành viên – History) |

**TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026**

# MỤC LỤC


**1 GIỚI THIỆU DỰ ÁN** **1**
- 1.1 Tổng quan đề tài và Product Vision . . . 1
- 1.2 Phạm vi hệ thống (MVP Scope) . . . 1
- 1.3 Ngăn xếp công nghệ (Technology Stack) . . . 2
- 1.4 Quy trình phát triển phần mềm . . . 3

**2 YÊU CẦU HỆ THỐNG** **6**
- 2.1 Các tác nhân hệ thống (Stakeholders & Actors) . . . 6
- 2.2 Danh mục yêu cầu chức năng (Functional Requirements) . . . 6
- 2.3 Danh mục yêu cầu phi chức năng (Non-Functional Requirements) . . . 7
- 2.4 User Story tiêu biểu (US-05: Đặt hàng và áp dụng ưu đãi) . . . 8

**3 KIẾN TRÚC HỆ THỐNG** **9**
- 3.1 Kiến trúc thành phần (Component Architecture) . . . 9
- 3.2 Kiến trúc triển khai hệ thống (Deployment Architecture) . . . 11
  - 3.2.1 Triển khai cục bộ đa container với Docker Compose . . . 11
  - 3.2.2 Triển khai trực tuyến trên nền tảng Đám mây (Cloud Production Deployment) . . . 13
- 3.3 Sơ đồ lớp Backend (Backend Class Diagram) . . . 15
- 3.4 Luồng tương tác nghiệp vụ tổng thể (Overall Business Flow) . . . 16

**4 THIẾT KẾ CƠ SỞ DỮ LIỆU** **17**
- 4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống . . . 17
- 4.2 Chi tiết 12 thực thể trong Cơ sở dữ liệu . . . 19
- 4.3 Ràng buộc toàn vẹn và Chiến lược đánh chỉ mục . . . 20
- 4.4 Thiết kế CSDL hỗ trợ nghiệp vụ nâng cao . . . 20

**5 THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG** **22**
- 5.1 Phân hệ Xác thực Người dùng (Auth) . . . 22
  - 5.1.1 Đặc tả Use Case . . . 22
  - 5.1.2 Sơ đồ Hoạt động (Activity Diagram) . . . 24
  - 5.1.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 25
  - 5.1.4 Hợp đồng API (API Contract) . . . 26
  - 5.1.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 26
  - 5.1.6 Kịch bản Kiểm thử (Test Cases) . . . 26
  - 5.1.7 Giao diện Người dùng . . . 27
- 5.2 Phân hệ Danh mục Sản phẩm (Product Catalog) . . . 28
  - 5.2.1 Đặc tả Use Case . . . 28
  - 5.2.2 Sơ đồ Hoạt động (Activity Diagram) . . . 29
  - 5.2.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 30
  - 5.2.4 Hợp đồng API (API Contract) . . . 31
  - 5.2.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 31
  - 5.2.6 Kịch bản Kiểm thử (Test Cases) . . . 31
  - 5.2.7 Giao diện Người dùng . . . 32
- 5.3 Phân hệ Chi tiết Sản phẩm (Product Detail) . . . 33
  - 5.3.1 Đặc tả Use Case . . . 33

5.3.2 Sơ đồ Hoạt động (Activity Diagram) . . . 35
5.3.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 36
5.3.4 Hợp đồng API (API Contract) . . . 37
5.3.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 37
5.3.6 Kịch bản Kiểm thử (Test Cases) . . . 37
5.3.7 Giao diện Người dùng . . . 38
5.4 Phân hệ Giỏ hàng (Cart) . . . 39
5.4.1 Đặc tả Use Case . . . 39
5.4.2 Sơ đồ Hoạt động (Activity Diagram) . . . 40
5.4.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 41
5.4.4 Hợp đồng API và Ánh xạ Cơ sở dữ liệu . . . 42
5.4.5 Kịch bản Kiểm thử (Test Cases) . . . 43
5.4.6 Giao diện Người dùng . . . 44
5.5 Phân hệ Đặt hàng (Order) . . . 45
5.5.1 Đặc tả Use Case . . . 45
5.5.2 Sơ đồ Hoạt động (Activity Diagram) . . . 47
5.5.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 48
5.5.4 Hợp đồng API (API Contract) . . . 49
5.5.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 49
5.5.6 Kịch bản Kiểm thử (Test Cases) . . . 50
5.5.7 Giao diện Người dùng . . . 51
5.6 Phân hệ Thanh toán (Payment) . . . 52
5.6.1 Đặc tả Use Case . . . 52
5.6.2 Sơ đồ Hoạt động (Activity Diagram) . . . 54
5.6.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 55
5.6.4 Hợp đồng API (API Contract) . . . 56
5.6.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 56
5.6.6 Kịch bản Kiểm thử (Test Cases) . . . 57
5.6.7 Giao diện Người dùng . . . 58
5.7 Phân hệ Lịch sử Đơn hàng (Order History) . . . 59
5.7.1 Đặc tả Use Case . . . 59
5.7.2 Sơ đồ Hoạt động (Activity Diagram) . . . 60
5.7.3 Sơ đồ Tuần tự (Sequence Diagram) . . . 61
5.7.4 Hợp đồng API (API Contract) . . . 62
5.7.5 Ánh xạ Cơ sở dữ liệu (Database Mapping) . . . 62
5.7.6 Kịch bản Kiểm thử (Test Cases) . . . 63
5.7.7 Giao diện Người dùng . . . 64

**6 HIỆN THỰC CÁC NGHIỆP VỤ NÂNG CAO 65**
6.1 Máy trạng thái Đơn hàng (Order State Machine) . . . 65
6.2 Xử lý Thanh toán Bất biến (Idempotent Payment) . . . 66
6.3 Kiểm soát Đồng thời Tồn kho (Concurrency Control) . . . 67
6.4 Nghiệp vụ Khuyến mãi và Điểm tích lũy Thành viên . . . 68
6.4.1 Cơ chế Mã giảm giá (Promotions Engine) . . . 68
6.4.2 Cơ chế Điểm thưởng Thành viên (Loyalty Points) . . . 68

**7 HIỆN THỰC HÓA VÀ ĐỐI CHIẾU THIẾT KẾ 70**
7.1 Ma trận Ánh xạ Nhiệm vụ GD1 & GD2 sang Mã nguồn . . . 70
7.2 Minh chứng Kỹ thuật Cốt lõi . . . 71
7.2.1 Cơ chế Mã hóa Bcrypt và Cấp phát Token trong AuthService . . . 71
7.2.2 Interactive Transaction và Khóa Lạc quan trong OrdersService . . . 71
7.2.3 Kiểm tra Bất biến Idempotency và Xử lý Tranh chấp trong PaymentsService . . . 72

**8 KIỂM THỬ HỆ THỐNG 73**
8.1 Chiến lược và Môi trường Kiểm thử . . . 73
8.1.1 Chiến lược kiểm thử đa tầng (Multi-tier Testing Strategy) . . . 73
8.1.2 Cấu hình Môi trường Kiểm thử Thực tế . . . 74
8.1.3 Dữ liệu Khởi tạo Kiểm thử (Seed Data) . . . 74
8.1.4 Các lệnh thực thi kiểm thử và Tóm tắt kết quả . . . 75
8.2 Bảng Tổng hợp Kết quả Kiểm thử theo Phân hệ . . . 75
8.3 Ba Kịch bản Kiểm thử Bắt buộc của Task 10 . . . 76
8.3.1 Kịch bản TS-FR-07-02: Chặn chuyển trạng thái sai của đơn hàng . . . 76
8.3.2 Kịch bản TS-FR-08-01: Cùng Idempotency-Key chỉ tạo đúng một payment trong CSDL . . . 77
8.3.3 Kịch bản TS-FR-09-01: Đặt đồng thời không làm tồn kho âm (Optimistic Locking) . . . 77
8.4 Kiểm thử Giao diện Người dùng (User Interface Testing) . . . 78
8.5 Lỗi Phát hiện và Giới hạn Kỹ thuật Đã biết . . . 79
8.6 Ma trận Truy vết Test Case về Yêu cầu và Use Case . . . 80

**9 ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN 81**
9.1 Đánh giá Kết quả Đạt được . . . 81
9.2 Các Hạn chế Hiện tại . . . 81
9.3 Hướng Phát triển trong Tương lai . . . 82

**A MA TRẬN TRUY VẾT YÊU CẦU (TRACEABILITY MATRIX) 83**

**B DANH MỤC API ENDPOINTS 85**

**C TỪ ĐIỂN DỮ LIỆU (DATA DICTIONARY) 87**
C.1 Bảng `users` . . . 87
C.2 Bảng `categories` . . . 87
C.3 Bảng `products` . . . 88
C.4 Bảng `product_size_prices` . . . 88
C.5 Bảng `toppings` . . . 88
C.6 Bảng `promotions` . . . 89
C.7 Bảng `orders` . . . 89
C.8 Bảng `order_items` . . . 89
C.9 Bảng `order_item_toppings` . . . 90
C.10 Bảng `payments` . . . 90
C.11 Bảng `loyalty_transactions` . . . 90
C.12 Bảng `order_status_history` . . . 91

**D DANH MỤC CHI TIẾT CÁC CA KIỂM THỬ (TEST CASES) 92**

# DANH MỤC HÌNH VẼ


1.1 Tổng kết tiến độ, vai trò phân công và bài học cải tiến quy trình của nhóm . . . 5

3.1 Sơ đồ Kiến trúc thành phần tổng thể hệ thống BrewLite . . . . . 10
3.2 Sơ đồ Triển khai hệ thống đa container với Docker Compose . . . . . 12
3.3 Sơ đồ Lớp (Class Diagram) tầng Backend NestJS . . . . . 15

4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống BrewLite . . . . . 18

5.1 Sơ đồ Use Case - Phân hệ Xác thực . . . . . 23
5.2 Sơ đồ Hoạt động - Phân hệ Xác thực . . . . . 24
5.3 Sơ đồ Tuần tự - Phân hệ Xác thực (Happy path & Exception paths) . . . . . 25
5.4 Giao diện Đăng nhập và Đăng ký người dùng . . . . . 27
5.5 Sơ đồ Use Case - Phân hệ Danh mục Sản phẩm . . . . . 28
5.6 Sơ đồ Hoạt động - Phân hệ Danh mục Sản phẩm . . . . . 29
5.7 Sơ đồ Tuần tự - Phân hệ Danh mục Sản phẩm . . . . . 30
5.8 Giao diện Thực đơn sản phẩm và Bộ lọc danh mục BrewLite . . . . . 32
5.9 Sơ đồ Use Case - Phân hệ Chi tiết Sản phẩm . . . . . 33
5.10 Sơ đồ Hoạt động - Phân hệ Chi tiết Sản phẩm . . . . . 35
5.11 Sơ đồ Tuần tự - Phân hệ Chi tiết Sản phẩm . . . . . 36
5.12 Giao diện Chi tiết món và Tùy biến Size / Topping . . . . . 38
5.13 Sơ đồ Use Case - Phân hệ Giỏ hàng . . . . . 39
5.14 Sơ đồ Hoạt động - Phân hệ Giỏ hàng . . . . . 40
5.15 Sơ đồ Tuần tự - Phân hệ Giỏ hàng (Client-side Architecture) . . . . . 41
5.16 Giao diện Quản lý Giỏ hàng và Điều chỉnh số lượng BrewLite . . . . . 44
5.17 Sơ đồ Use Case - Phân hệ Đặt hàng . . . . . 45
5.18 Sơ đồ Hoạt động - Phân hệ Đặt hàng . . . . . 47
5.19 Sơ đồ Tuần tự - Phân hệ Đặt hàng (Interactive Transaction & Pricing Engine) . 48
5.20 Giao diện Xác nhận Đặt hàng và Áp dụng Khuyến mại . . . . . 51
5.21 Sơ đồ Use Case - Phân hệ Thanh toán . . . . . 52
5.22 Sơ đồ Hoạt động - Phân hệ Thanh toán . . . . . 54
5.23 Sơ đồ Tuần tự - Phân hệ Thanh toán (Idempotent Flow & Gateway Integration) 55
5.24 Giao diện Cổng thanh toán không tiền mặt đa phương thức . . . . . 58
5.25 Sơ đồ Use Case - Phân hệ Lịch sử Đơn hàng . . . . . 59
5.26 Sơ đồ Hoạt động - Phân hệ Lịch sử Đơn hàng . . . . . 60
5.27 Sơ đồ Tuần tự - Phân hệ Lịch sử Đơn hàng . . . . . 61
5.28 Giao diện Lịch sử và Trạng thái Đơn hàng của Khách hàng . . . . . 64

6.1 Sơ đồ Máy trạng thái Đơn hàng BrewLite (9 chuyển tiếp hợp lệ) . . . . . 65

# DANH MỤC BẢNG BIỂU


1.1 Ngăn xếp công nghệ sử dụng trong dự án BrewLite . . . . . 3

2.1 Bảng tổng hợp Yêu cầu Chức năng (Functional Requirements) . . . . . 7

3.1 Bảng tổng hợp hạ tầng và biến môi trường triển khai Cloud Production . . . . . 14

5.1 Đặc tả Use Case: Đăng ký và Đăng nhập người dùng . . . . . 23
5.2 Danh mục API Endpoints - Phân hệ Xác thực . . . . . 26
5.3 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Xác thực . . . . . 26
5.4 Đặc tả Use Case: Xem và Tìm kiếm Danh mục Sản phẩm (UC-CAT-01) . . . . 28
5.5 Danh mục API Endpoints - Phân hệ Danh mục Sản phẩm . . . . . 31
5.6 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Danh mục . . . . . 31
5.7 Đặc tả Use Case: Xem chi tiết và Tùy biến món (UC-DET-01) . . . . . 34
5.8 Danh mục API Endpoints - Phân hệ Chi tiết Sản phẩm . . . . . 37
5.9 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Chi tiết Sản phẩm . . . . . 37
5.10 Đặc tả Use Case: Quản lý Giỏ hàng (UC-CART-01) . . . . . 39
5.11 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Giỏ hàng . . . . . 43
5.12 Đặc tả Use Case: Tạo đơn hàng trực tuyến (UC-ORD-01) . . . . . 46
5.13 Danh mục API Endpoints - Phân hệ Đặt hàng . . . . . 49
5.14 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Đặt hàng . . . . . 50
5.15 Đặc tả Use Case: Thực hiện Thanh toán Trực tuyến (UC-PAY-01) . . . . . 53
5.16 Danh mục API Endpoints - Phân hệ Thanh toán . . . . . 56
5.17 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Thanh toán . . . . . 57
5.18 Đặc tả Use Case: Tra cứu Lịch sử và Chi tiết Đơn hàng (UC-HIS-01) . . . . . 59
5.19 Danh mục API Endpoints - Phân hệ Lịch sử Đơn hàng . . . . . 62
5.20 Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Lịch sử . . . . . 63

7.1 Ma trận Đối chiếu Thiết kế GD1 & GD2 sang Mã nguồn thực tế . . . . . 70

8.1 Tổng hợp số lượng và trạng thái Test Cases theo từng phân hệ . . . . . 76
8.2 Ma trận truy vết ca kiểm thử về Yêu cầu Chức năng và Use Case . . . . . 80

A.1 Ma trận Truy vết Toàn diện Yêu cầu - Thiết kế - Cài đặt - Kiểm thử . . . . . 83

B.1 Danh mục chi tiết toàn bộ các API Endpoints hệ thống BrewLite . . . . . 85

D.1 Danh mục đầy đủ các ca kiểm thử hệ thống BrewLite . . . . . 92

# Chương 1

# GIỚI THIỆU DỰ ÁN

## 1.1 Tổng quan đề tài và Product Vision

Trong xu hướng chuyển đổi số mạnh mẽ của ngành dịch vụ ăn uống (F&B), mô hình phục vụ truyền thống tại các quầy cà phê thường xuyên đối mặt với tình trạng quá tải trong các khung giờ cao điểm. Khách hàng phải xếp hàng dài chờ đợi, nhân viên thu ngân dễ gặp sai sót khi ghi nhận các tùy chọn đồ uống phức tạp (độ ngọt, lượng đá, kích cỡ, topping), đồng thời việc thanh toán tiền mặt gây chậm trễ trong luồng vận hành.

Hệ thống đặt cà phê trực tuyến **BrewLite** được nghiên cứu và xây dựng nhằm cung cấp giải pháp đặt món tinh gọn (Lite), hiện đại và tiện lợi cho khách hàng. Với tầm nhìn sản phẩm (*Product Vision*):

> *"BrewLite là nền tảng đặt món trực tuyến giúp khách hàng yêu thích cà phê có thể dễ dàng duyệt thực đơn, cá nhân hóa đồ uống theo khẩu vị, đặt hàng nhanh chóng và thanh toán an toàn mà không cần chờ đợi tại quầy; đồng thời giúp cửa hàng tối ưu hóa quy trình xử lý đơn hàng và kiểm soát chặt chẽ tồn kho."*

Mục tiêu cốt lõi của đề tài bao gồm:

- **Trải nghiệm người dùng mượt mà:** Khách hàng có thể tương tác trực quan với thực đơn, phân loại theo danh mục, tùy biến linh hoạt các thông số đồ uống trước khi thêm vào giỏ hàng.
- **Tính toán chi phí minh bạch:** Tích hợp bộ máy tính giá tự động bao gồm phụ thu theo size/topping, áp dụng mã giảm giá (voucher) và tích lũy điểm thưởng thành viên thân thiết (Loyalty Points) sau khi hoàn tất thanh toán.
- **Độ tin cậy và an toàn giao dịch:** Đảm bảo tính toàn vẹn dữ liệu trong môi trường đa người dùng thông qua cơ chế kiểm soát đồng thời (Optimistic Locking) cho tồn kho sản phẩm và cơ chế xử lý thanh toán bất biến/trùng lặp (Idempotent Payment).

## 1.2 Phạm vi hệ thống (MVP Scope)

Trong khuôn khổ đồ án môn học Công nghệ Phần mềm, hệ thống tập trung hoàn thiện các tính năng cốt lõi cho sản phẩm khả dụng tối thiểu (MVP) trong hai giai đoạn phát triển chính (Giai đoạn 1 và Giai đoạn 2):

1. **Phân hệ Xác thực (Auth):** Đăng ký tài khoản, đăng nhập hệ thống, cấp phát mã thông báo JWT an toàn và phân quyền người dùng.
2. **Phân hệ Danh mục sản phẩm (Product Catalog):** Hiển thị danh sách sản phẩm phân nhóm theo danh mục (Cà phê, Trà...), lọc món còn hàng và tìm kiếm.
3. **Phân hệ Chi tiết sản phẩm (Product Detail):** Xem thông tin mô tả chi tiết, hình ảnh, đơn giá và cho phép lựa chọn size, mức đường, mức đá cùng danh sách topping đi kèm.

4. **Phân hệ Giỏ hàng (Cart):** Quản lý các món đã chọn ở phía trình duyệt (Client-side), cập nhật số lượng, chỉnh sửa tùy chọn và tạm tính giá trị đơn hàng.

5. **Phân hệ Đặt hàng (Order):** Tạo đơn hàng chính thức từ giỏ hàng, kiểm tra tính khả dụng của tồn kho, áp dụng mã ưu đãi `WELCOME10`, tự động tính điểm tích lũy dự kiến và chuyển đổi trạng thái đơn hàng.

6. **Phân hệ Thanh toán (Payment):** Khởi tạo giao dịch thanh toán trực tuyến qua cổng giả lập (Mock Payment Gateway), xử lý bảo mật chống trùng lặp với `Idempotency-Key`.

7. **Phân hệ Lịch sử đơn hàng (Order History):** Tra cứu danh sách các đơn hàng đã thực hiện, xem chi tiết từng đơn hàng và trạng thái thanh toán/xử lý.

**Giới hạn phạm vi (Out-of-Scope trong MVP):** Để đảm bảo tiến độ và chất lượng cho các nghiệp vụ trọng tâm của khách hàng:

- **Đổi điểm thưởng trừ tiền (Redeem Loyalty Points):** Hệ thống đã hiện thực trọn vẹn cơ chế tự động tích lũy điểm thưởng (*Earn Points*) cho khách hàng sau khi thanh toán thành công ($1.000$ VNĐ $= 1$ điểm) và thiết kế sẵn sàng cấu trúc dữ liệu cho việc chi tiêu/hoàn điểm (`LoyaltyTransactionType.SPENT`, `REFUNDED`). Tuy nhiên, tính năng cho phép nhập điểm để trừ trực tiếp vào giá trị đơn hàng mới (*Redeem Points*) được phân loại vào phạm vi mở rộng sau MVP (Post-MVP Backlog) để đảm bảo an toàn tuyệt đối cho luồng thanh toán và nghiệp vụ hoàn điểm khi hủy đơn.

- **Giao diện Pha chế (Barista Web UI):** Mặc dù hệ thống đã hiện thực toàn diện Máy trạng thái đơn hàng (Order State Machine) với đầy đủ 9 chuyển tiếp nghiệp vụ (bao gồm các trạng thái `PREPARING`, `READY`, `COMPLETED`), giao diện web chuyên dụng dành riêng cho nhân viên pha chế tại quầy chưa nằm trong phạm vi giao diện của bản phát hành MVP này.

- **Quản trị viên nâng cao (Advanced Admin Portal):** Hệ thống tập trung vào trải nghiệm đặt hàng của khách hàng (B2C); các chức năng quản trị kho nâng cao và báo cáo doanh thu sẽ được triển khai ở pha tiếp theo.

## 1.3 Ngăn xếp công nghệ (Technology Stack)

Hệ thống BrewLite được xây dựng trên nền tảng kiến trúc hiện đại, phân tách độc lập giữa Client và Server (Bảng 1.1):

Bảng 1.1: Ngăn xếp công nghệ sử dụng trong dự án BrewLite

| Thành phần | Công nghệ | Vai trò và Đặc tính kỹ thuật |
|---|---|---|
| **Frontend** | Next.js 15 | Framework React hiện đại sử dụng App Router, hỗ trợ Server Components và Client Components tối ưu SEO và thời gian phản hồi. |
| | TypeScript | Đảm bảo an toàn kiểu dữ liệu (Type Safety) đồng nhất từ giao diện đến API. |
| | TailwindCSS | Thiết kế giao diện phản hồi nhanh (Responsive UI) theo chuẩn thiết kế hiện đại. |
| | Zustand / React Hooks | Quản lý trạng thái giỏ hàng và dữ liệu cục bộ phía Client. |
| **Backend** | NestJS 11 | Framework Node.js kiến trúc module hướng dịch vụ (Service-Oriented Architecture), hỗ trợ Dependency Injection mạnh mẽ. |
| | Prisma ORM 6 | Ánh xạ quan hệ đối tượng hiện đại, cung cấp Type-safe Database Client và migration tự động. |
| | Bcrypt | Mã hóa mật khẩu người dùng với cơ chế salt rounds an toàn (10 rounds). |
| | Passport & JWT | Xác thực và phân quyền bằng cơ chế JSON Web Token không trạng thái (Stateless). |
| **Database** | PostgreSQL 16 | Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, đảm bảo tính toàn vẹn ACID, hỗ trợ JSONB và lập chỉ mục tối ưu. |
| **Triển khai** | Docker & Compose | Đóng gói toàn bộ ứng dụng thành 3 container chuẩn hóa (`frontend`, `backend`, `postgres`), đảm bảo môi trường đồng nhất. |

## 1.4 Quy trình phát triển phần mềm

Dự án áp dụng mô hình phát triển phần mềm linh hoạt **Agile/Scrum tinh gọn (Lean Agile)**. Quy trình được tổ chức thành các chu kỳ phát triển linh hoạt theo từng giai đoạn thực tế của dự án:

- **Sprint Planning:** Nhóm họp đầu mỗi giai đoạn để phân tích Product Backlog, làm rõ User Story, phân rã yêu cầu thành danh mục công việc cụ thể (Task Backlog) theo lát cắt dọc (*Vertical Slice*), thống nhất các quyết định kỹ thuật cốt lõi (D1–D9) và cam kết phạm vi bàn giao thay vì ước lượng bằng Story Points trừu tượng.
- **Daily Scrum / Sync:** Các thành viên đồng bộ tiến độ thường xuyên, cập nhật trạng thái các task (TODO, IN PROGRESS, DONE) trên bảng quản lý công việc, rà soát mã nguồn (Code Review qua Pull Request) và tháo gỡ kịp thời các trở ngại kỹ thuật (blockers).
- **Sprint Review:** Nghiệm thu và kiểm thử các tính năng phần mềm đã hoàn tất đạt tiêu chuẩn chấp nhận (*Definition of Done*: vượt qua bộ kiểm thử tự động, tích hợp hoàn chỉnh và triển khai thực tế trên môi trường live).

- **Sprint Retrospective:** Đánh giá hiệu quả phối hợp, rút kinh nghiệm về thiết kế API, tính toàn vẹn dữ liệu đồng thời và đề ra các giải pháp cải tiến quy trình kỹ thuật cho giai đoạn tiếp theo.

Ghi nhận tiến độ và số liệu thực tế của Nhóm

- **Mô hình tổ chức:** Mỗi thành viên sở hữu trọn một feature theo chiều dọc (Vertical Slice: phân tích ca sử dụng, đặc tả hợp đồng API, kiểm thử và hỗ trợ tích hợp giao diện), vận hành theo mô hình Development Team đa chức năng (Cross-functional Team).
- **Vai trò Scrum:** Nhóm áp dụng Scrum tinh gọn (Lean Agile). **Phạm Minh Chiến** (Trưởng nhóm) kiêm nhiệm vai trò **Scrum Master** (điều phối tiến độ, quản lý Git branch, review Pull Request) và **Product Owner đại diện** (thống nhất hợp đồng API và bộ 9 Quyết định kỹ thuật D1–D9). Toàn bộ 7 thành viên đóng vai trò **Development Team** chủ động tự quản.
- **Chu kỳ phát triển:**
  - **Giai đoạn 1 (Hiện thực Backend, 12 task GD1-01 – GD1-12):** Từ 16/10/2026 đến 23/10/2026 (hoàn thành kiến trúc NestJS, Prisma ORM, kiểm soát tranh chấp tồn kho bằng Optimistic Locking, thanh toán Idempotency và kiểm thử Race-Condition tự động).
  - **Giai đoạn 2 (Hiện thực Frontend & Tích hợp, 9 task GD2-01 – GD2-09):** Từ 23/10/2026 đến 25/10/2026 (xây dựng giao diện Next.js 15 App Router, kết nối Zustand Store, hoàn thiện luồng E2E từ chọn món, giỏ hàng, áp mã ưu đãi đến thanh toán QR và tra cứu lịch sử đơn).
- **Theo dõi tiến độ:** Nhóm không đo lường bằng Story Points truyền thống mà quản lý dựa trên tỷ lệ hoàn thành công việc cụ thể: đạt **21/21 task cam kết (tỷ lệ 100%)** qua 2 giai đoạn, được nghiệm thu thông qua bộ kiểm thử tự động toàn diện (84 unit tests, 5 E2E tests, 44 ca kiểm thử API tự động, kịch bản Concurrency và Idempotency) và bản triển khai trực tiếp trên Vercel (`https://brew-lite.vercel.app/`) / Render.
- **Retrospective (Bài học kinh nghiệm rút ra):**
  a) *Tiếp cận API-First giảm thiểu xung đột:* Thống nhất đặc tả Hợp đồng API (API Contract) và phong cách đóng gói phản hồi chuẩn (`ApiResponse<T>`) từ đầu giúp các thành viên độc lập phát triển frontend/backend mà không bị vướng mắc giao tiếp.
  b) *Nghiệp vụ tài chính & tồn kho đòi hỏi khóa đa tầng:* Không thể phụ thuộc vào kiểm tra dữ liệu ở phía Client; bắt buộc phải kết hợp Transaction cơ sở dữ liệu với khóa lạc quan (*Optimistic Locking*) và *Idempotency-Key* ở Backend để ngăn chặn triệt để hiện tượng đặt âm hàng hoặc trừ tiền hai lần.
  c) *Đồng bộ trạng thái Client-Server thông minh:* Quản lý giỏ hàng phía Client qua LocalStorage giúp trải nghiệm liền mạch, nhưng cần cơ chế tính chữ ký giỏ hàng (`pendingCartSignature`) để hủy kịp thời đơn hàng cũ nếu khách đổi món trước khi thanh toán.
  d) *Tự động hóa kiểm thử và container hóa:* Thiết lập Docker Compose chuẩn hóa môi trường PostgreSQL và chạy kiểm thử tự động ngay trên Git giúp nhóm phát hiện và sửa triệt để các lỗi logic phức tạp trước khi bàn giao.

Hình 1.1: Tổng kết tiến độ, vai trò phân công và bài học cải tiến quy trình của nhóm

# Chương 2

# YÊU CẦU HỆ THỐNG

## 2.1 Các tác nhân hệ thống (Stakeholders & Actors)

Dựa trên phân tích phạm vi nghiệp vụ của hệ thống BrewLite trong Giai đoạn 1 và 2, các tác nhân tương tác trực tiếp với hệ thống được xác định bao gồm:

- **Khách vãng lai (Guest):** Người dùng truy cập hệ thống qua trình duyệt web mà chưa thực hiện đăng nhập. Tác nhân này có thể duyệt danh mục thực đơn, xem chi tiết món ăn, lựa chọn các tùy biến đồ uống và lưu trữ giỏ hàng tạm thời trên trình duyệt. Khi thực hiện thanh toán hoặc đặt hàng, hệ thống yêu cầu xác thực tài khoản.
- **Khách hàng đã đăng ký (Registered Customer):** Khách hàng sở hữu tài khoản định danh trên hệ thống. Tác nhân này có đầy đủ quyền lợi: đặt hàng chính thức, áp dụng mã voucher ưu đãi, tích lũy điểm thưởng thành viên (Loyalty Points), thanh toán trực tuyến và tra cứu toàn bộ lịch sử đơn hàng cá nhân.
- **Cổng thanh toán giả lập (Mock Payment Gateway):** Tác nhân hệ thống bên ngoài mô phỏng trung gian tài chính. Cổng tiếp nhận yêu cầu thanh toán kèm mã định danh bất biến (`Idempotency-Key`), kiểm tra thông tin và trả về kết quả giao dịch (`SUCCESS` hoặc `FAILED`).
- **Hệ thống Quản trị dữ liệu (PostgreSQL Database Engine):** Đóng vai trò thực thi các ràng buộc toàn vẹn, đảm bảo tính bền vững (Durability) và cách ly (Isolation) cho các giao dịch đồng thời.

## 2.2 Danh mục yêu cầu chức năng (Functional Requirements)

Hệ thống BrewLite được phân rã thành 6 nhóm yêu cầu chức năng chính (FR-01 đến FR-06), tương ứng với các phân hệ nghiệp vụ cốt lõi:

Bảng 2.1: Bảng tổng hợp Yêu cầu Chức năng (Functional Requirements)

| Mã FR | Tên yêu cầu | Mô tả chi tiết |
|---|---|---|
| **FR-01** | Quản lý Xác thực & Tài khoản | Cho phép người dùng đăng ký tài khoản với email duy nhất, họ tên, mật khẩu băm an toàn; Đăng nhập nhận JWT Token hợp lệ; Kiểm soát truy cập bảo vệ các endpoint nghiệp vụ. |
| **FR-02** | Duyệt thực đơn & Chi tiết món | Cung cấp danh mục sản phẩm phân nhóm theo phân loại (Cà phê, Trà...); Tìm kiếm món; Xem thông tin chi tiết về kích cỡ, giá phụ thu, mức đường, mức đá và danh sách topping khả dụng. |
| **FR-03** | Quản lý Giỏ hàng | Cho phép thêm sản phẩm kèm tùy biến vào giỏ hàng cục bộ; Điều chỉnh số lượng món; Xóa từng món hoặc làm rỗng giỏ; Tạm tính tổng chi phí tự động. |
| **FR-04** | Đặt hàng & Xử lý Chiết khấu | Khởi tạo đơn hàng từ giỏ hàng; Kiểm tra tồn kho sản phẩm với Optimistic Locking; Áp dụng mã giảm giá hợp lệ (`promotionCode`); Tự động tính điểm tích lũy thành viên dự kiến; Lưu vết snapshot giá tại thời điểm đặt. |
| **FR-05** | Thanh toán Trực tuyến | Khởi tạo giao dịch thanh toán; Kiểm tra trùng lặp giao dịch bằng cơ chế `Idempotency-Key` và băm `request_hash`; Cập nhật trạng thái đơn hàng sang `PAID` khi giao dịch thành công. |
| **FR-06** | Tra cứu Lịch sử Đơn hàng | Hiển thị danh sách các đơn hàng đã đặt của người dùng theo thứ tự thời gian giảm dần; Xem chi tiết thông tin thanh toán, danh mục món và trạng thái xử lý hiện tại. |

## 2.3 Danh mục yêu cầu phi chức năng (Non-Functional Requirements)

Các yêu cầu phi chức năng đặt ra các tiêu chuẩn kỹ thuật nhằm đảm bảo chất lượng vận hành của hệ thống:

- **NFR-01 (Hiệu năng - Performance):** Thời gian phản hồi trung bình (API Response Time) của các tác vụ truy vấn danh mục thực đơn và chi tiết sản phẩm không vượt quá 500ms dưới điều kiện tải thông thường.
- **NFR-02 (Bảo mật - Security):**
  - Toàn bộ mật khẩu người dùng phải được mã hóa một chiều bằng thuật toán `bcrypt` với độ phức tạp 10 salt rounds trước khi lưu trữ vào cơ sở dữ liệu.
  - Hệ thống sử dụng giao thức xác thực JSON Web Token (JWT) không trạng thái (*Stateless*). Tuyệt đối không để lộ trường nhạy cảm `password_hash` trong bất kỳ phản hồi API nào.
- **NFR-03 (Tính toàn vẹn & Tin cậy - Reliability & Idempotency):**
  - Toàn bộ thao tác tạo đơn hàng và trừ kho phải được bọc trong một Interactive Transaction (`$transaction`) của Prisma, đảm bảo nguyên lý ACID.

  - Nghiệp vụ thanh toán phải đảm bảo tính bất biến (*Idempotency*). Các yêu cầu trùng lặp hoặc gửi đồng thời với cùng một `Idempotency-Key` phải được nhận diện và xử lý an toàn, loại trừ hoàn toàn nguy cơ thanh toán lặp (double charging).

- **NFR-04 (Bảo toàn lịch sử giá - Data Immutability):** Giá bán của từng sản phẩm tại thời điểm tạo đơn hàng phải được ghi nhận độc lập vào trường `price_at_order` của bảng chi tiết đơn hàng (`order_items`), đảm bảo lịch sử thanh toán không bị sai lệch khi giá niêm yết trong bảng sản phẩm (`products`) thay đổi trong tương lai.

## 2.4 User Story tiêu biểu (US-05: Đặt hàng và áp dụng ưu đãi)

Để minh họa mối liên kết giữa yêu cầu của người dùng và thiết kế kỹ thuật, User Story tiêu biểu về quy trình đặt hàng được đặc tả theo chuẩn quốc tế:

**User Story US-05: Đặt hàng trực tuyến với mã ưu đãi**

**Là một:** Khách hàng đã đăng nhập vào hệ thống BrewLite,
**Tôi muốn:** Thực hiện đặt các món trong giỏ hàng, áp dụng mã ưu đãi hợp lệ và được tích lũy điểm thưởng thành viên sau khi thanh toán,
**Để:** Mua được đồ uống yêu thích với chi phí tiết kiệm nhất và tích lũy ưu đãi cho các đơn hàng tương lai.

**Tiêu chí chấp nhận (Acceptance Criteria - Gherkin Syntax):**

**Kịch bản 1:** *Đặt hàng thành công với mã giảm giá hợp lệ và ghi nhận tích lũy điểm thưởng:*

- **Given:** Khách hàng có các món đồ uống trong giỏ hàng với tổng giá trị tạm tính là 120.000 VNĐ và nhập mã voucher `WELCOME10`.
- **When:** Khách hàng chọn áp dụng mã voucher và gửi yêu cầu tạo đơn hàng (`POST /api/v1/orders`).
- **Then:** Hệ thống áp dụng chiết khấu 10% (12.000 VNĐ), tính tổng số tiền thanh toán chính xác là 108.000 VNĐ, kiểm tra đủ tồn kho với Optimistic Locking, tạo đơn hàng ở trạng thái `PENDING`, đồng thời tính toán điểm thưởng dự kiến tích lũy là +108 điểm khi thanh toán thành công và chuyển hướng sang trang thanh toán.

**Kịch bản 2:** *Xử lý ngoại lệ khi số lượng tồn kho không đủ (Out of Stock):*

- **Given:** Khách hàng đặt món Cà phê Sữa Đá với số lượng 2 ly, nhưng tồn kho khả dụng trong cơ sở dữ liệu chỉ còn 1 ly.
- **When:** Khách hàng nhấn ”Xác nhận đặt hàng”.
- **Then:** Hệ thống phát hiện vi phạm số lượng tồn kho trong transaction, hủy bỏ thao tác tạo đơn, trả về mã lỗi `400 Bad Request` kèm thông báo lỗi rõ ràng và giữ nguyên trạng thái giỏ hàng.

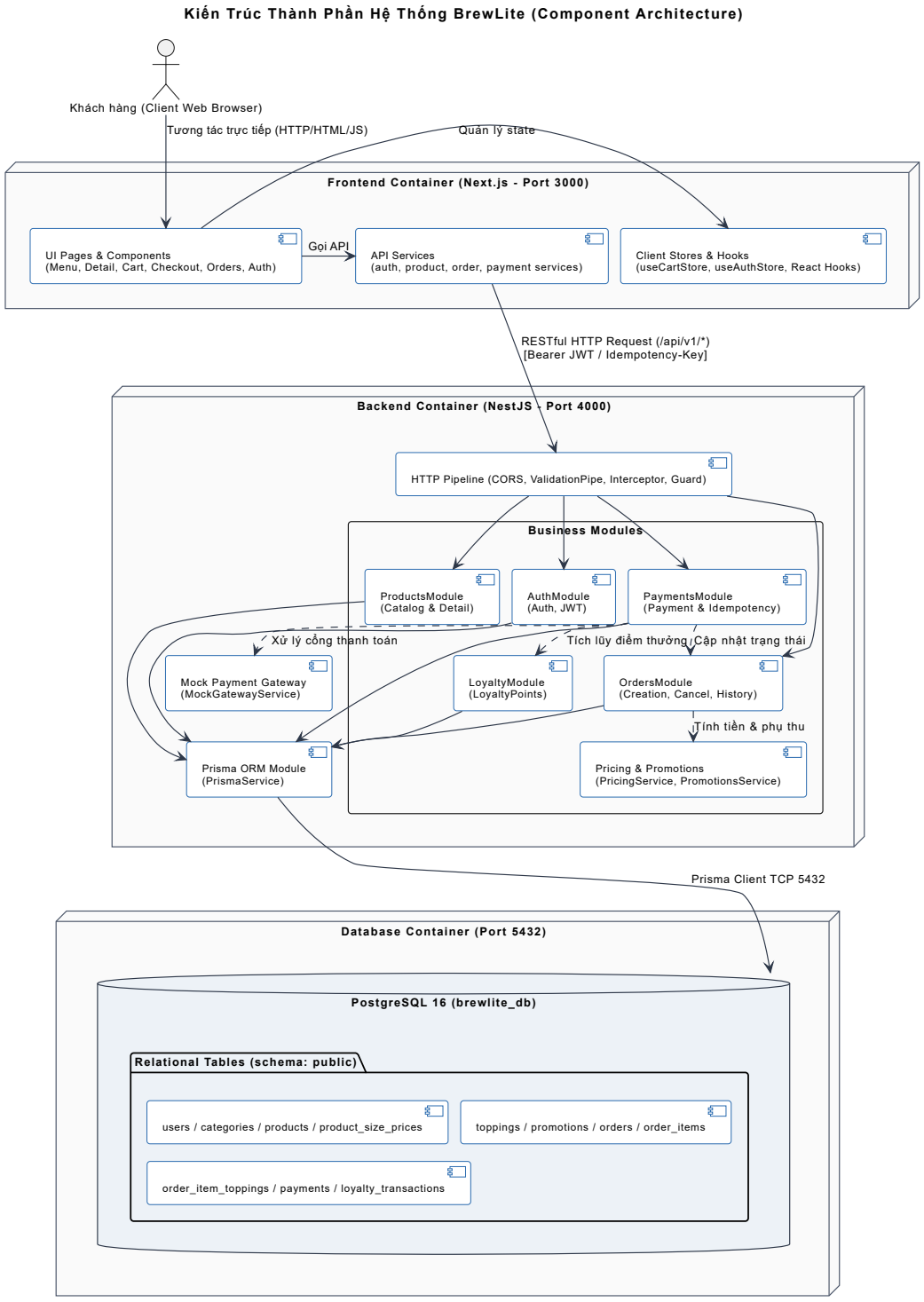
# Chương 3

# KIẾN TRÚC HỆ THỐNG

## 3.1 Kiến trúc thành phần (Component Architecture)

Hệ thống BrewLite được thiết kế theo mô hình kiến trúc phân tầng dịch vụ (*Layered Service-Oriented Architecture*), đảm bảo tính phân tách trách nhiệm cao (*Separation of Concerns*) và khả năng mở rộng linh hoạt giữa tầng giao diện người dùng, tầng xử lý nghiệp vụ và tầng dữ liệu bền vững.

Sơ đồ kiến trúc thành phần tổng thể của BrewLite được biểu diễn tại Hình 3.1.




Hình 3.1: Sơ đồ Kiến trúc thành phần tổng thể hệ thống BrewLite

Hệ thống bao gồm các thành phần cốt lõi:

1. **Tầng Trình bày (Frontend Presentation Layer - Next.js 15):**
    - Xây dựng bằng Next.js App Router, sử dụng TypeScript và TailwindCSS.
    - Phân chia thành các module giao diện tương ứng với các trang: Xác thực (`/auth/login`, `/auth/register`), Thực đơn (`/menu`), Chi tiết món (`/products/[id]`), Giỏ hàng

(`/cart`), Đặt hàng (`/checkout`), Thanh toán (`/checkout/payment`) và Lịch sử (`/orders`).

- Quản lý trạng thái cục bộ của giỏ hàng thông qua Custom Hook / Zustand và đồng bộ token xác thực lưu trong bộ nhớ an toàn.

2. **Tầng Xử lý Nghiệp vụ (Backend Application Layer - NestJS 11):** Tổ chức theo các Module chuyên biệt, tận dụng cơ chế Dependency Injection (DI):

   - **AuthModule:** Đảm nhận đăng ký, băm mật khẩu bằng `bcrypt`, đăng nhập và cấp phát JSON Web Token; bảo vệ hệ thống thông qua `JwtAuthGuard`.
   - **ProductsModule:** Quản lý danh mục (`categories`), sản phẩm (`products`), các tùy chọn size/đường/đá/topping và kiểm tra tính khả dụng của kho hàng.
   - **OrdersModule:** Trung tâm điều phối quy trình đặt hàng, tích hợp chặt chẽ với máy trạng thái đơn hàng (`OrderStateMachine`) và kiểm soát giao dịch tạo đơn.
   - **PricingModule, PromotionsModule & LoyaltyModule:** Các động cơ tính toán độc lập: `PricingModule` tính toán phụ thu kích cỡ/topping; `PromotionsModule` kiểm tra tính hợp lệ và chiết khấu của mã giảm giá (`WELCOME10`); `LoyaltyModule` đảm nhận cơ chế tự động tích lũy điểm thưởng thành viên theo tỷ lệ thực trả sau khi thanh toán thành công.
   - **PaymentsModule:** Xử lý thanh toán trực tuyến, quản lý bảng idempotency (`PaymentIdempoter`) và kết nối giao tiếp với cổng thanh toán.
   - **Common / Core Services:** Bộ lọc ngoại lệ toàn cục (`AllExceptionsFilter`) và bộ chuẩn hóa phản hồi (`TransformInterceptor`) đảm bảo mọi giao tiếp API luôn tuân thủ cấu trúc đồng nhất `{success, data, message, timestamp}`.

3. **Tầng Cơ sở dữ liệu (Data Access Layer):** Sử dụng **Prisma ORM 6** cung cấp Type-safe Client kết nối trực tiếp đến hệ quản trị cơ sở dữ liệu quan hệ **PostgreSQL 16**.

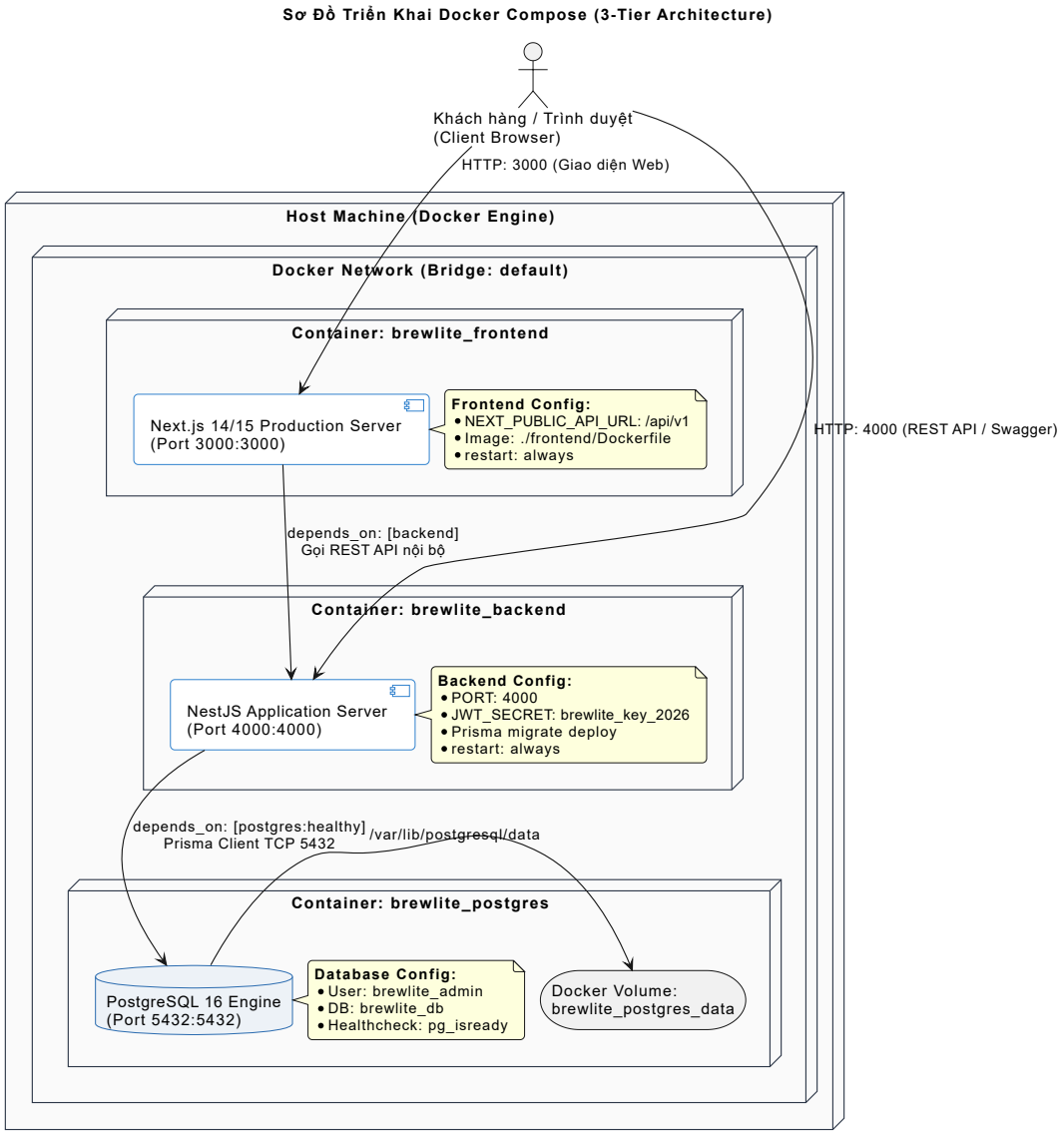
4. **Hệ thống ngoại vi (External Service): Mock Payment Gateway** giả lập cổng thanh toán trực tuyến của bên thứ ba, cung cấp endpoint xử lý thẻ/ví điện tử và phản hồi kết quả giao dịch về backend.

## 3.2 Kiến trúc triển khai hệ thống (Deployment Architecture)

### 3.2.1 Triển khai cục bộ đa container với Docker Compose

Toàn bộ môi trường ứng dụng BrewLite được đóng gói và cấu hình tự động thông qua **Docker** và **Docker Compose**, giúp loại bỏ sự sai lệch giữa môi trường phát triển (Development) và môi trường kiểm thử/triển khai.

Sơ đồ kiến trúc triển khai container được minh họa tại Hình 3.2.




Hình 3.2: Sơ đồ Triển khai hệ thống đa container với Docker Compose

Hệ thống bao gồm 3 container độc lập kết nối thông qua mạng nội bộ Docker bridge (`brewlite-network`

- **Frontend Container (`brewlite-frontend`):**
  - Base image: `node:20-alpine`.
  - Port ánh xạ: `3000:3000`.
  - Biến môi trường: `NEXT_PUBLIC_API_BASE_URL=http://localhost:4000/api/v1`.
- **Backend Container (`brewlite-backend`):**
  - Base image: `node:20-alpine`.
  - Port ánh xạ: `4000:4000`.
  - Biến môi trường: `DATABASE_URL=postgresql://user:pass@postgres:5432/brewlite`, `JWT_SECRET=...`, `JWT_EXPIRES_IN=7d`, `PORT=4000`.
  - Phụ thuộc (`depends_on`): Chờ container `postgres` sẵn sàng nhận kết nối trước khi khởi chạy và tự động thực thi lệnh `prisma migrate deploy`.

- **Database Container (`brewlite-postgres`):**
  - Base image: `postgres:16-alpine`.
  - Port ánh xạ: `5432:5432`.
  - Lưu trữ bền vững (`Persistent Storage`): Mount volume Docker `postgres_data` vào thư mục `/var/lib/postgresql/data`, đảm bảo dữ liệu không bị thất thoát khi container khởi động lại hoặc dừng hoạt động.

### 3.2.2 Triển khai trực tuyến trên nền tảng Đám mây (Cloud Production Deployment)

Bên cạnh môi trường đóng gói cục bộ bằng Docker Compose phục vụ phát triển và kiểm thử nội bộ, hệ thống BrewLite được triển khai thực tế trên hạ tầng đám mây phân tán theo mô hình tích hợp và phân phối liên tục (CI/CD) từ nhánh `main` trên GitHub:

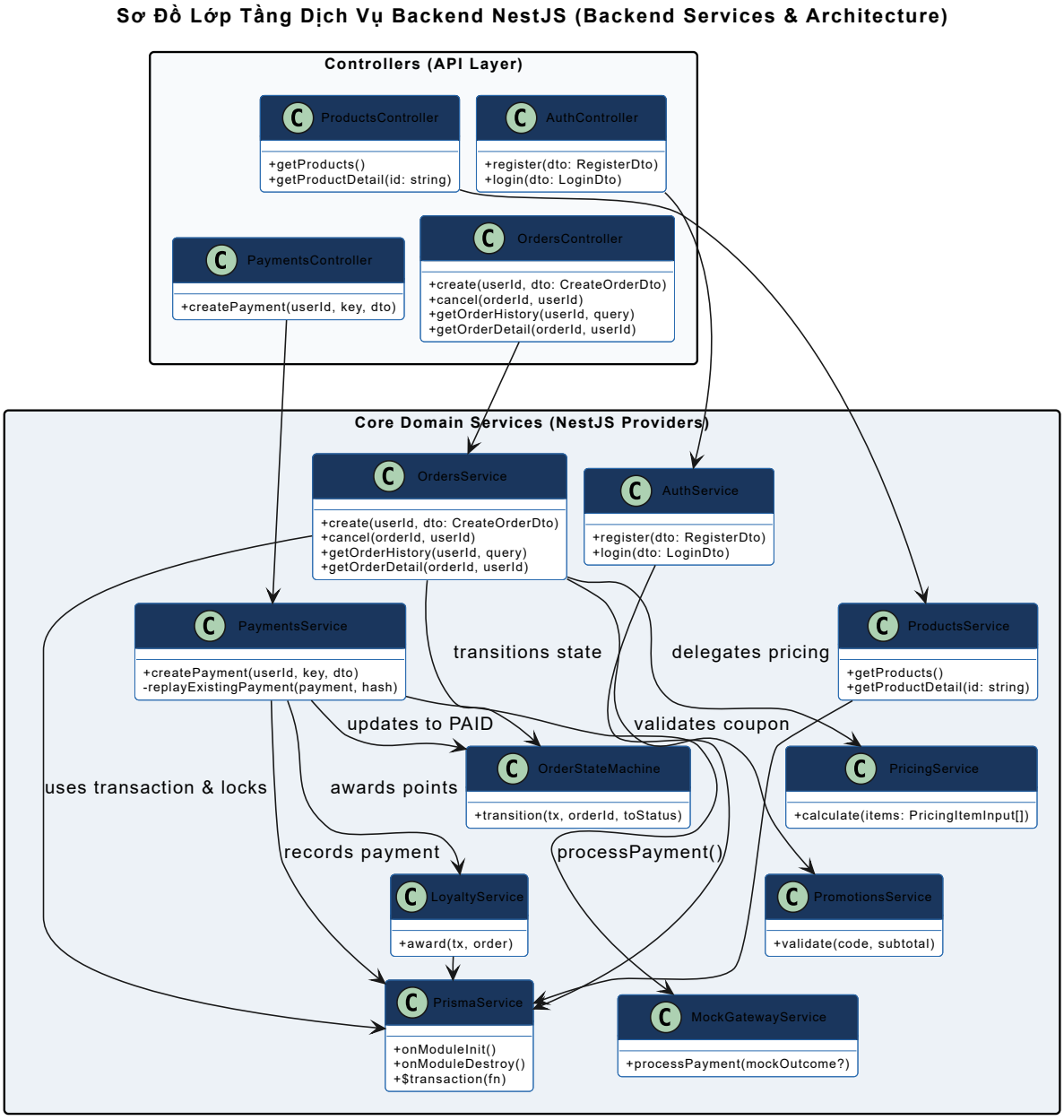
- **Frontend Web App (Vercel):** Ứng dụng Next.js 15 được triển khai tự động lên nền tảng máy chủ Serverless của Vercel tại địa chỉ chính thức `https://brew-lite.vercel.app/`, tận dụng mạng phân phối nội dung toàn cầu (Edge Network CDN) để tối ưu hóa tốc độ tải trang và hỗ trợ Server-Side Rendering (SSR).
- **Backend REST API (Render):** Dịch vụ NestJS được đóng gói dưới dạng Web Service trên Render, cung cấp endpoint công khai an toàn qua giao thức HTTPS, tự động kích hoạt tiến trình triển khai lại (Auto-deploy) mỗi khi có Pull Request được hợp nhất vào nhánh chính.
- **Cơ sở dữ liệu Đám mây (Supabase):** Cung cấp cụm máy chủ PostgreSQL 16 được quản trị hoàn toàn (Managed Cloud Database). Để tối ưu hóa hiệu năng và tránh cạn kiệt tài nguyên kết nối khi ứng dụng backend chạy trên môi trường đám mây, hệ thống cấu hình song song hai chuỗi kết nối thông qua Prisma ORM:
  - `DATABASE_URL`: Kết nối thông qua bộ điều phối kết nối **PgBouncer (Port 6543)** ở chế độ Transaction Pooling phục vụ các truy vấn nghiệp vụ hàng ngày.
  - `DIRECT_URL`: Kết nối trực tiếp đến PostgreSQL **(Port 5432)** dành riêng cho tiến trình đồng bộ cấu trúc cơ sở dữ liệu (`prisma migrate deploy`).

Bảng 3.1: Bảng tổng hợp hạ tầng và biến môi trường triển khai Cloud Production

| **Thành phần** | **Nền tảng / Địa chỉ** | **Cấu hình biến môi trường chính** |
|---|---|---|
| **Frontend** | Vercel (Edge CDN) https://brew-lite.vercel.app | NEXT_PUBLIC_API_URL = https://brewlite-api.onrender.com/api/v1<br>NEXT_PUBLIC_DEMO_MODE = false |
| **Backend** | Render (Web Service Container) | PORT = 4000<br>NODE_ENV = production<br>CORS_ORIGIN = https://brew-lite.vercel.app<br>DATABASE_URL = postgresql://postgres:***@...pooler.supabase.com:6543/...<br>DIRECT_URL = postgresql://postgres:***@...supabase.com:5432/...<br>JWT_SECRET = [Khoá bảo mật chuẩn sản xuất]<br>JWT_EXPIRES_IN = 7d |
| **Database** | Supabase (Managed PostgreSQL 16) | Cụm máy chủ PostgreSQL 16 tại khu vực Đông Nam Á (Singapore), hỗ trợ kết nối bảo mật SSL (sslmode=require) và tự động sao lưu dữ liệu. |

## 3.3 Sơ đồ lớp Backend (Backend Class Diagram)

Cấu trúc hướng đối tượng của hệ thống Backend được thể hiện thông qua Sơ đồ lớp tại Hình 3.3. Sơ đồ mô tả rõ nét mối liên kết giữa các tầng Controller, Service, DTO và các Entity của Prisma Client.




Hình 3.3: Sơ đồ Lớp (Class Diagram) tầng Backend NestJS

Các lớp dịch vụ chính bao gồm:

- `AuthService`: Xử lý logic mã hóa mật khẩu, kiểm tra trùng lặp email và sinh mã xác thực JWT.
- `ProductsService`: Chịu trách nhiệm truy vấn danh mục, lọc món và thực thi lệnh cập nhật phiên bản tồn kho (`version`) khi có giao dịch.

- `OrdersService`: Lớp điều phối chính của quy trình mua hàng. Quản lý việc tạo đơn trong transaction, phối hợp cùng `PricingService`, `PromotionsService` và `LoyaltyService` để xác định số tiền chính xác, sau đó kiểm tra và trừ tồn kho.
- `PaymentsService`: Khởi tạo yêu cầu thanh toán, quản lý và kiểm tra bảng `PaymentIdempotency`, gọi Mock Gateway và cập nhật trạng thái đơn hàng.

## 3.4 Luồng tương tác nghiệp vụ tổng thể (Overall Business Flow)

Luồng tương tác nghiệp vụ từ đầu đến cuối (End-to-End) của một khách hàng trên hệ thống diễn ra tuần tự qua các giai đoạn:

1. **Giai đoạn 1 - Khám phá và Chọn món:** Khách hàng duyệt danh mục thực đơn tại trang (`/menu`), chọn đồ uống để xem chi tiết (`/products/[id]`), tùy chọn kích cỡ (Size S/M/L), mức đường, mức đá và các món topping đi kèm. Món ăn sau khi cấu hình được đưa vào giỏ hàng cục bộ (`Cart`) quản lý bằng Zustand.
2. **Giai đoạn 2 - Xác thực tài khoản:** Khi người dùng chọn tiến hành đặt hàng (`/checkout`), nếu chưa đăng nhập, hệ thống sẽ tự động điều hướng sang trang Đăng nhập (`/auth/login`) hoặc Đăng ký (`/auth/register`) để xác thực danh tính qua JWT.
3. **Giai đoạn 3 - Đặt hàng và Tính giá:** Khách hàng xác nhận danh sách món trong giỏ hàng và áp dụng mã ưu đãi `WELCOME10` (nếu có). Backend thực thi kiểm tra tồn kho đồng thời qua cơ chế Optimistic Locking (khóa lạc quan theo phiên bản `version`), tính toán đơn giá snapshot và tạo bản ghi đơn hàng ở trạng thái `PENDING`.
4. **Giai đoạn 4 - Thanh toán Trực tuyến:** Hệ thống chuyển hướng khách hàng sang màn hình thanh toán (`/checkout/payment`). Khách hàng chọn phương thức (Thẻ ngân hàng hoặc Ví điện tử). Lệnh thanh toán được gửi kèm `Idempotency-Key`. Mock Gateway xác nhận giao dịch thành công.
5. **Giai đoạn 5 - Cập nhật và Hoàn tất:** Đơn hàng được chuyển trạng thái sang `PAID`, hệ thống tự động tích lũy điểm thưởng thành viên vào tài khoản người dùng, xóa các món đã đặt khỏi giỏ hàng cục bộ và lưu vết vào Lịch sử đơn hàng (`/orders`).

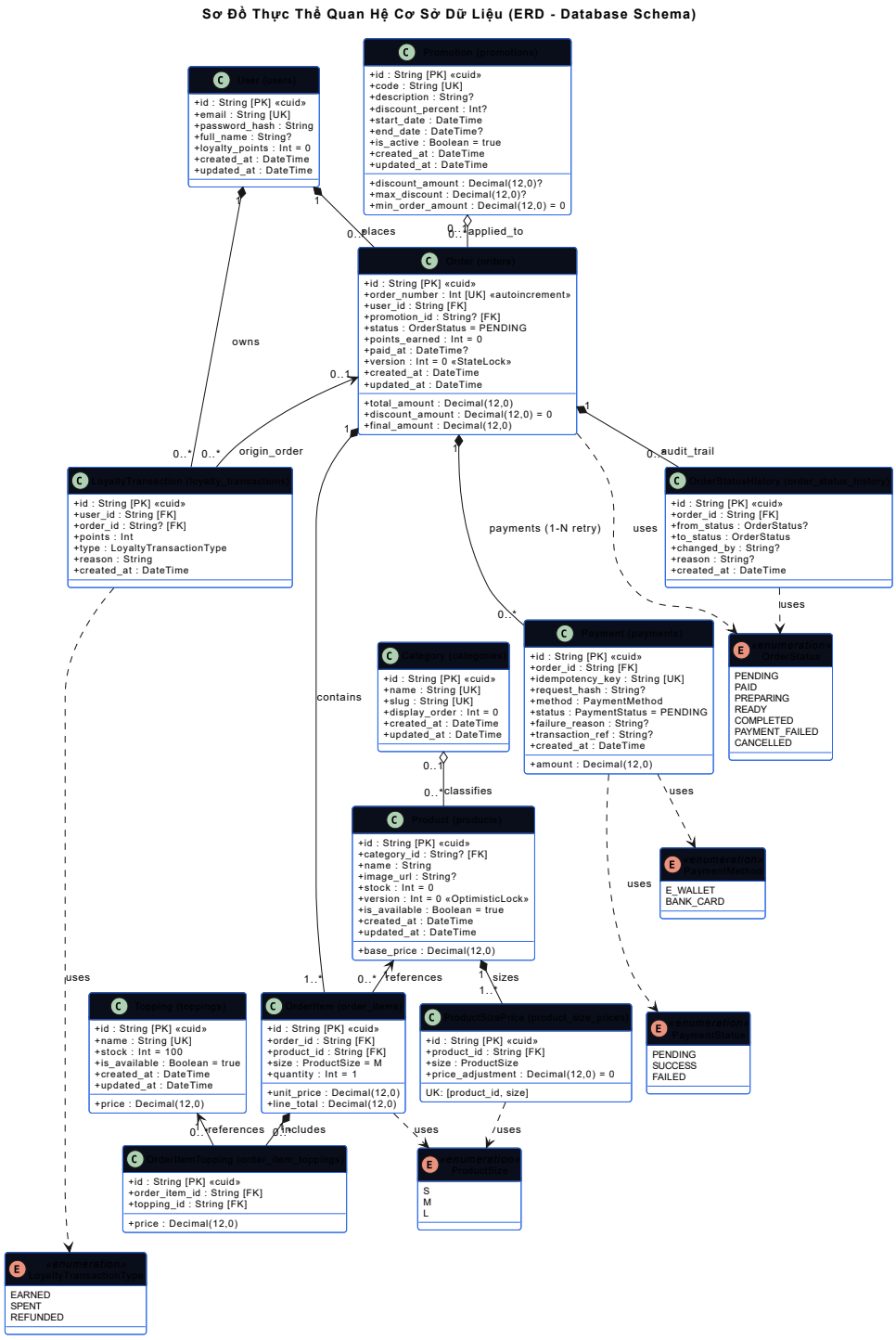
# Chương 4

# THIẾT KẾ CƠ SỞ DỮ LIỆU

## 4.1 Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống

Cơ sở dữ liệu của hệ thống BrewLite được chuẩn hóa theo dạng chuẩn 3NF, thiết kế bám sát tập tin mô hình dữ liệu `backend/prisma/schema.prisma`. Toàn bộ hệ thống bao gồm 12 thực thể quan hệ, đảm bảo tính toàn vẹn dữ liệu, hiệu năng truy vấn và hỗ trợ chuyên sâu cho các nghiệp vụ phức tạp như kiểm soát đồng thời (Optimistic Locking) và thanh toán bất biến (Idempotency).

Sơ đồ ERD toàn hệ thống được trình bày tại Hình 4.1.




Hình 4.1: Sơ đồ Thực thể Liên kết (ERD) toàn hệ thống BrewLite

## 4.2 Chi tiết 12 thực thể trong Cơ sở dữ liệu

Mô hình dữ liệu bao gồm 12 bảng được phân chia theo các nhóm nghiệp vụ chính:

1. **Nhóm Người dùng và Định danh:**
    - **`users`:** Lưu trữ thông tin định danh khách hàng. Gồm các trường: `id` (PK, cuid), `email` (Unique, chuỗi định danh duy nhất), `password_hash` (mật khẩu băm Bcrypt 10 rounds), `full_name`, `loyalty_points` (điểm tích lũy hiện có, mặc định 0), `created_at`, `updated_at`.
2. **Nhóm Danh mục và Sản phẩm:**
    - **`categories`:** Phân loại thực đơn (Cà phê, Trà...). Gồm: `id` (PK), `name` (Unique), `slug` (Unique), `display_order` (thứ tự hiển thị trên tab giao diện), `created_at`, `updated_at`.
    - **`products`:** Thông tin đồ uống. Gồm: `id` (PK), `category_id` (FK → `categories.id`), `name`, `base_price` (Decimal(12,0) tiền VNĐ nguyên chẵn), `image_url`, `stock` (số lượng tồn kho khả dụng), `version` (Int, default 0 - trường phiên bản phục vụ Optimistic Locking), `is_available` (Boolean), `created_at`, `updated_at`.
    - **`product_size_prices`:** Cấu hình phụ thu theo từng kích cỡ (S, M, L) cho từng sản phẩm. Gồm: `id` (PK), `product_id` (FK → `products.id`, Cascade), `size` (Enum `ProductSize`: S, M, L), `price_adjustment` (Decimal(12,0)). Có ràng buộc duy nhất `@@unique([product_id, size])`.
    - **`toppings`:** Danh mục topping dùng kèm. Gồm: `id` (PK), `name` (Unique), `price` (Decimal(12,0)), `stock` (Int, default 100), `is_available` (Boolean), `created_at`, `updated_at`.
3. **Nhóm Khuyến mãi (Promotions):**
    - **`promotions`:** Quản lý mã giảm giá voucher. Gồm: `id` (PK), `code` (Unique, ví dụ: `WELCOME10`, `GIAM15K`), `description`, `discount_percent` (tỷ lệ giảm theo %), `discount_amount` (tiền giảm cố định), `max_discount` (mức giảm trần tối đa, ví dụ 20.000 VNĐ trong mã `WELCOME10`), `min_order_amount` (giá trị đơn tối thiểu, ví dụ 40.000 VNĐ trong mã `WELCOME10` hoặc 60.000 VNĐ trong mã `GIAM15K`), `start_date`, `end_date`, `is_active` (Boolean).
4. **Nhóm Đơn hàng và Chi tiết:**
    - **`orders`:** Bảng trung tâm lưu trữ thông tin đơn hàng. Gồm: `id` (PK), `order_number` (Int, Unique, autoincrement - mã hiển thị cho khách dạng #1042), `user_id` (FK → `users.id`), `promotion_id` (FK → `promotions.id`, có thể null), `status` (Enum `OrderStatus`), `total_amount` (tạm tính trước giảm), `discount_amount` (số tiền được giảm), `final_amount` (số tiền thực trả), `points_earned` (số điểm thưởng tích được từ đơn), `paid_at` (thời điểm thanh toán), `version` (Int, default 0 - Optimistic Locking khi chuyển trạng thái), `created_at`, `updated_at`.
    - **`order_items`:** Chi tiết từng món trong đơn. Gồm: `id` (PK), `order_id` (FK → `orders.id`, Cascade), `product_id` (FK → `products.id`), `size` (Enum `ProductSize`), `quantity` (Int), `unit_price` (Decimal(12,0) - đơn giá snapshot đã cộng phụ thu size), `line_total` (Decimal(12,0) = (unitPrice + toppings) × quantity).

- **order_item_toppings:** Lưu trữ topping kèm theo từng món. Gồm: id (PK), order_item_id (FK → order_items.id, Cascade), topping_id (FK → toppings.id), price (Decimal(12,0) - snapshot giá topping tại thời điểm đặt).
- **order_status_history:** Lưu vết lịch sử chuyển đổi trạng thái đơn hàng (phục vụ giám sát 9 chuyển tiếp của máy trạng thái). Gồm: id (PK), order_id (FK → orders.id, Cascade), from_status (OrderStatus?), to_status (OrderStatus), changed_by (String?), reason (String?), created_at.

5. **Nhóm Thanh toán và Điểm thành viên:**

- **payments:** Lưu thông tin các lần thanh toán. Gồm: id (PK), order_id (FK → orders.id, quan hệ 1-N hỗ trợ retry khi thanh toán thất bại), idempotency_key (Unique, chuỗi UUID chống trùng lặp), request_hash (mã băm SHA-256 nội dung request), amount (Decimal(12,0)), method (Enum: BANK_CARD, E_WALLET), status (Enum: PENDING, SUCCESS, FAILED), failure_reason, transaction_ref, created_at.
- **loyalty_transactions:** Nhật ký biến động điểm thưởng. Gồm: id (PK), user_id (FK → users.id), order_id (FK → orders.id?), points (Int), type (Enum: EARNED, SPENT, REFUNDED), reason, created_at.

## 4.3 Ràng buộc toàn vẹn và Chiến lược đánh chỉ mục

- **Ràng buộc Khóa ngoại (Referential Integrity):** Sử dụng cơ chế onDelete: Cascade cho các quan hệ cha-con gắn kết chặt chẽ: khi xóa đơn hàng (Order), hệ thống tự động dọn dẹp các dòng tương ứng trong order_items, order_item_toppings, order_status_history và payments. Ngược lại, quan hệ giữa Product và OrderItem áp dụng cơ chế bảo vệ (Restrict) để ngăn ngừa việc xóa mất dấu sản phẩm đã có trong hóa đơn.
- **Ràng buộc Duy nhất (Unique Constraints):** Thiết lập chỉ mục duy nhất trên users(email), categories(slug), categories(name), promotions(code), orders(order_number), payments(idempotency_key) và bộ đôi product_size_prices(product_id, size).
- **Chỉ mục Tối ưu truy vấn (Query Optimization Indexes):** Đánh chỉ mục B-tree trên các trường khóa ngoại và các trường thường xuyên xuất hiện trong mệnh đề lọc/sắp xếp:
  - orders(user_id), orders(status): tăng tốc độ truy vấn danh sách lịch sử đơn hàng.
  - order_items(order_id): tăng tốc độ nạp dữ liệu hóa đơn chi tiết.
  - payments(order_id): hỗ trợ tra cứu các giao dịch theo đơn hàng.
  - loyalty_transactions(user_id): tối ưu hóa việc tính toán lịch sử điểm thưởng.

## 4.4 Thiết kế CSDL hỗ trợ nghiệp vụ nâng cao

Cơ sở dữ liệu của BrewLite được tính toán kỹ lưỡng ngay từ thiết kế schema để đáp ứng các bài toán kỹ thuật phức tạp:

1. **Kiểm soát Đồng thời (Optimistic Locking):** Trường `version` trên bảng `products` và `orders` cho phép phát hiện xung đột dữ liệu mà không cần khóa bi quan (Pessimistic lock) gây nghẽn cổ chai. Câu lệnh cập nhật kho luôn đi kèm điều kiện `WHERE id = :id AND version = :currentVersion`, tự động tăng `version = version + 1`.
2. **Thanh toán Bất biến (Idempotent Payment):** Trường `idempotency_key` có tính duy nhất toàn cục kết hợp cùng `request_hash` trong bảng `payments` giúp hệ thống ngay lập tức nhận diện các request gửi lại từ client hoặc lỗi mạng, trả về kết quả đã xử lý mà không thực hiện gọi lại cổng thanh toán.
3. **Snapshot Lịch sử Giá (Price Immutability):** Các trường `unit_price` trong `order_items` và `price` trong `order_item_toppings` sao lưu chính xác giá trị tại thời điểm khách hàng đặt, đảm bảo giá trị hóa đơn luôn cố định bất biến theo thời gian.

# Chương 5

# THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG

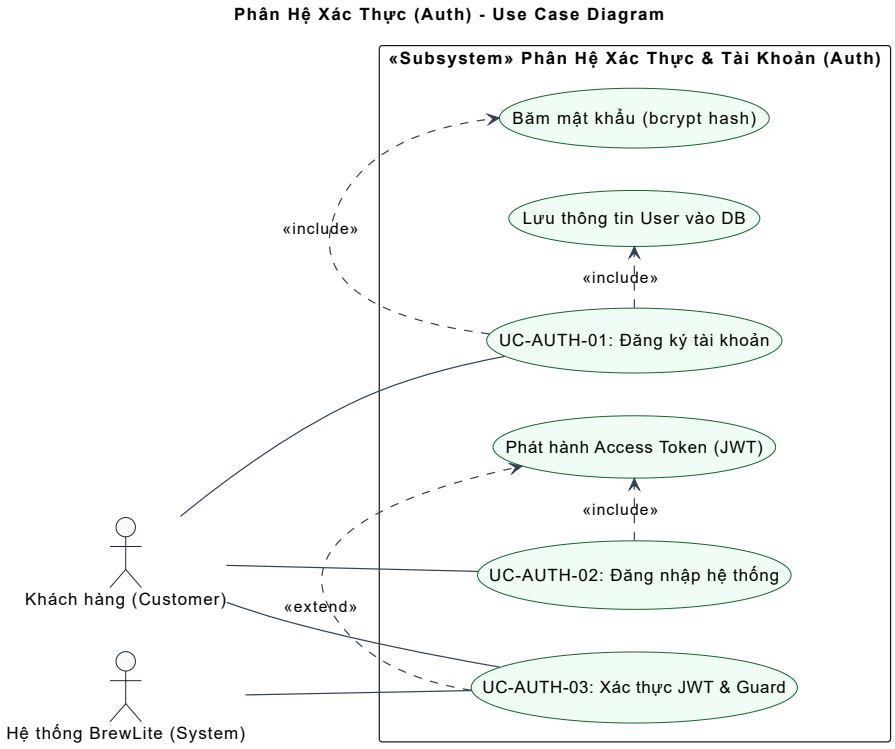
Chương này trình bày chi tiết thiết kế kỹ thuật cho toàn bộ 7 phân hệ chức năng cốt lõi của hệ thống BrewLite. Mỗi phân hệ được chuẩn hóa theo cấu trúc toàn diện bao gồm:

1. **Đặc tả Use Case:** Sơ đồ Use Case và bảng đặc tả nghiệp vụ chi tiết (Tiền điều kiện, Hậu điều kiện, Luồng chính, Luồng ngoại lệ).
2. **Sơ đồ Hoạt động (Activity Diagram):** Mô tả luồng xử lý quyết định nghiệp vụ.
3. **Sơ đồ Tuần tự (Sequence Diagram):** Ánh xạ tương tác chuẩn xác từ Người dùng → Frontend Component → Controller → Service → Prisma/DB.
4. **Hợp đồng API (API Contract):** Đặc tả Endpoint, phương thức HTTP, DTO validation và cấu trúc phản hồi.
5. **Ánh xạ Cơ sở dữ liệu (Database Mapping):** Thao tác bảng, trường dữ liệu và ràng buộc ACID.
6. **Kịch bản Kiểm thử (Test Cases):** Bảng các ca kiểm thử nghiệp vụ và giao diện.
7. **Giao diện Người dùng (User Interface):** Vị trí lưu giữ minh chứng cài đặt giao diện.

## 5.1 Phân hệ Xác thực Người dùng (Auth)

### 5.1.1 Đặc tả Use Case

Phân hệ Xác thực quản lý vòng đời tài khoản của người dùng, cung cấp cơ chế đăng ký và đăng nhập bảo mật bằng JSON Web Token (JWT). Sơ đồ Use Case được minh họa tại Hình 5.1.



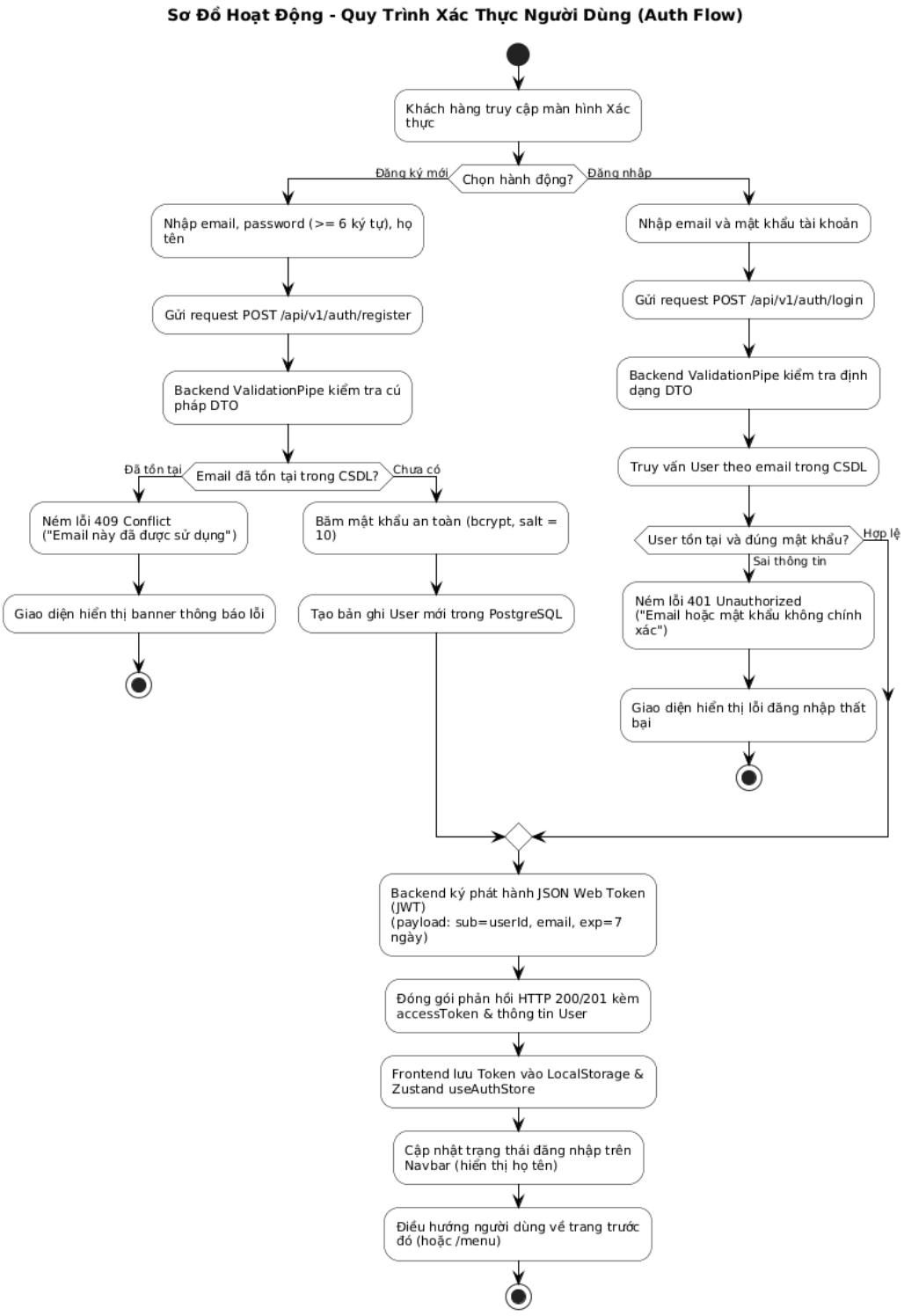

Hình 5.1: Sơ đồ Use Case - Phân hệ Xác thực

Bảng 5.1: Đặc tả Use Case: Đăng ký và Đăng nhập người dùng

| **Mục** | **Đăng ký (UC-AUTH-01)** | **Đăng nhập (UC-AUTH-02)** |
|---|---|---|
| **Actor** | Khách vãng lai (Guest) | Khách hàng đã có tài khoản |
| **Tiền điều kiện** | Chưa đăng nhập; địa chỉ email chưa tồn tại trên hệ thống. | Đã có tài khoản trên hệ thống; tài khoản ở trạng thái hoạt động. |
| **Hậu điều kiện** | Bản ghi mới được tạo trong bảng `users`; mật khẩu được băm Bcrypt 10 rounds; khởi tạo 0 điểm loyalty. | Cấp phát mã thông báo JWT `accessToken`; lưu trữ tại client để xác thực các request kế tiếp. |
| **Luồng chính** | 1. Người dùng nhập email, mật khẩu ($\geq$ 6 ký tự), họ tên.<br>2. Nhấn ”Đăng ký”.<br>3. Hệ thống kiểm tra hợp lệ DTO và tính duy nhất của email.<br>4. Băm mật khẩu bằng `bcrypt.hash(password, 10)`.<br>5. Tạo người dùng trong DB và trả về thông tin (loại trừ `password_hash`). | 1. Người dùng nhập email và mật khẩu.<br>2. Nhấn ”Đăng nhập”.<br>3. Hệ thống tìm người dùng theo email.<br>4. So sánh mật khẩu bằng `bcrypt.compare(password, hash)`.<br>5. Sinh mã JWT chứa payload `{sub, email}`.<br>6. Trả về token và thông tin hồ sơ người dùng. |
| **Luồng ngoại lệ** | - DTO không hợp lệ: Trả về mã `400 Bad Request`.<br>- Trùng email: Trả về `409 Conflict` (”Email đã được sử dụng”). | - Email không tồn tại hoặc sai mật khẩu: Trả về `401 Unauthorized` (”Email hoặc mật khẩu không chính xác”). |

### 5.1.2 Sơ đồ Hoạt động (Activity Diagram)

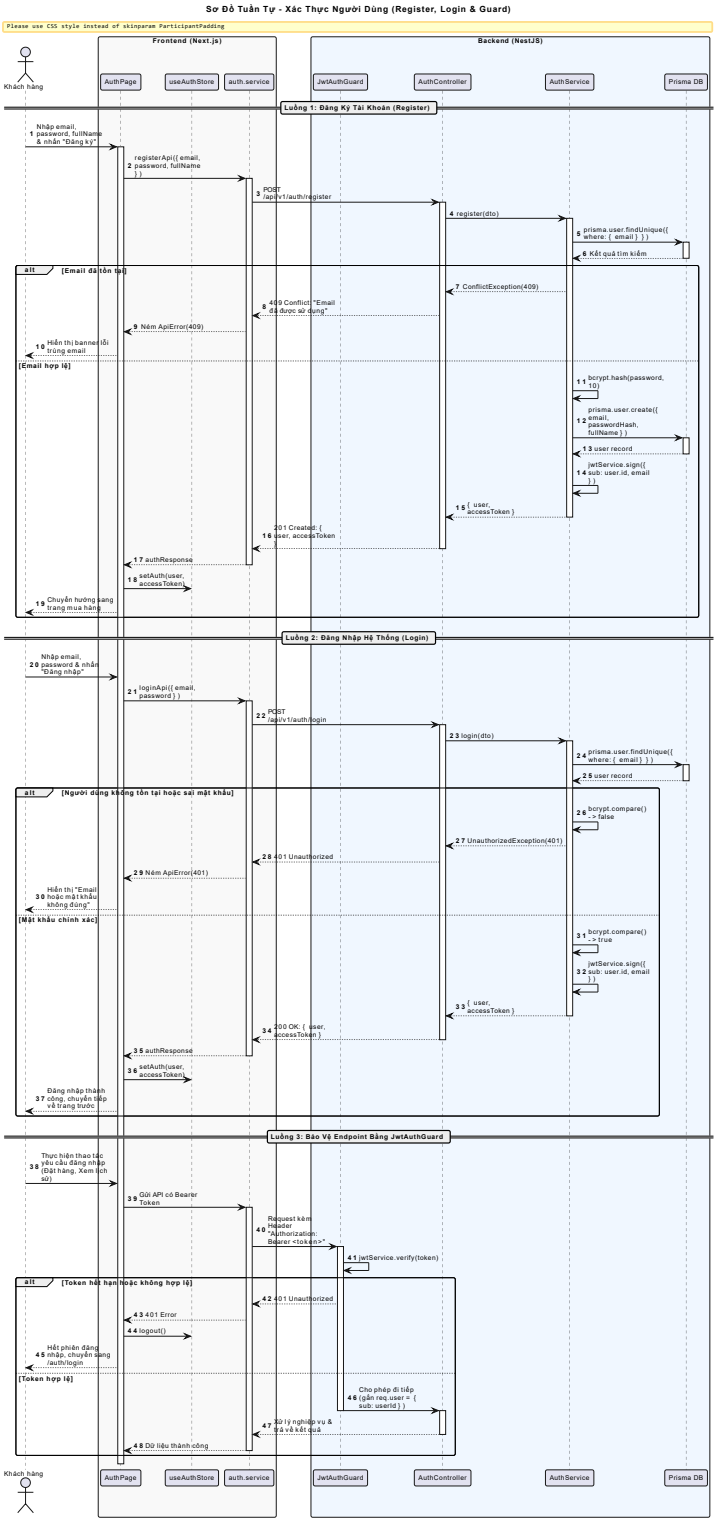
Sơ đồ Hoạt động mô tả chi tiết quy trình xử lý luồng đăng nhập và kiểm tra xác thực người dùng được biểu diễn tại Hình 5.2.




Hình 5.2: Sơ đồ Hoạt động - Phân hệ Xác thực

### 5.1.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh chuẩn xác các lời gọi hàm từ giao diện qua Controller đến Service và Prisma DB tại Hình 5.3.




Hình 5.3: Sơ đồ Tuần tự - Phân hệ Xác thực (Happy path & Exception paths)

### 5.1.4 Hợp đồng API (API Contract)

Bảng 5.2: Danh mục API Endpoints - Phân hệ Xác thực

| Phương thức | Endpoint | Mô tả Request Body | Phản hồi chính |
|---|---|---|---|
| `POST` | `/api/v1/auth/register` | `{email, password, fullName}` (Email hợp lệ, password $\geq$ 6 ký tự) | `201 Created`: Trả về thông tin người dùng mới (loại trừ passwordHash). Lỗi: `400`, `409`. |
| `POST` | `/api/v1/auth/login` | `{email, password}` | `200 OK`: Trả về `accessToken` và thông tin người dùng. Lỗi: `400`, `401`. |

### 5.1.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

Mọi thao tác của AuthModule tác động trực tiếp lên bảng `users`:

- **Đăng ký:** Gọi `prisma.user.findUnique({ where: { email } })`. Nếu tồn tại $\rightarrow$ throw `ConflictException`. Nếu chưa, thực thi `bcrypt.hash(password, 10)` và gọi `prisma.user.create({ data: { email, passwordHash, fullName, loyaltyPoints: 0 } })`.
- **Đăng nhập:** Gọi `prisma.user.findUnique({ where: { email } })`. Nếu không tìm thấy hoặc `bcrypt.compare()` trả về false $\rightarrow$ throw `UnauthorizedException`.
- **Bảo mật dữ liệu:** Trường `passwordHash` tuyệt đối được lược bỏ trước khi gán vào đối tượng trả về thông qua destructuring: `const { passwordHash, ...userWithoutPassword } = user`.

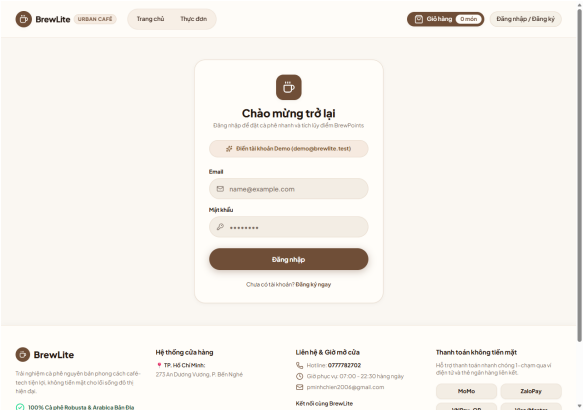
### 5.1.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.3: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Xác thực

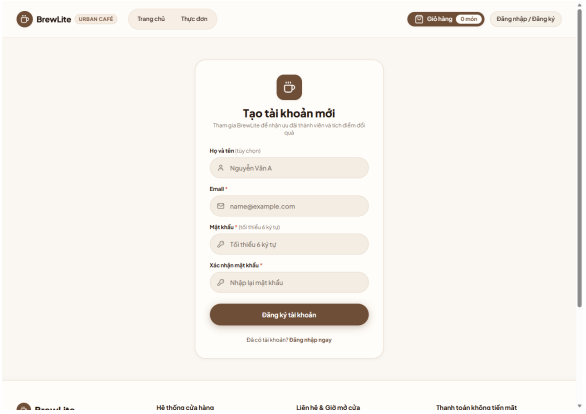
| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| `TC-AUTH-01` | Hợp lệ | Email chưa tồn tại | `POST /auth/register` | HTTP 201, tạo user mới, băm mật khẩu Bcrypt 10 rounds | Đạt |
| `TC-AUTH-03` | Lỗi | Email đã tồn tại | `POST /auth/register` | HTTP 409 Conflict, thông báo ”Email này đã được sử dụng” | Đạt |
| `TC-AUTH-04` | Hợp lệ | Đã có tài khoản | `POST /auth/login` | HTTP 200, cấp Bearer JWT `accessToken` hợp lệ | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Xác thực được trình bày tại Chương 8 và Phụ lục D.

### 5.1.7 Giao diện Người dùng




(a) Màn hình Đăng nhập



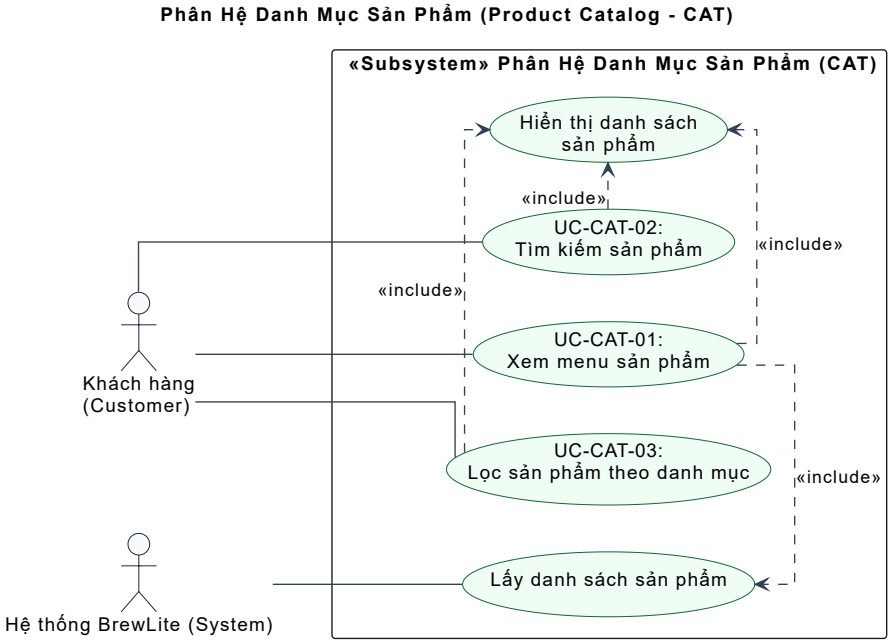

(b) Màn hình Đăng ký tài khoản

Hình 5.4: Giao diện Đăng nhập và Đăng ký người dùng

## 5.2 Phân hệ Danh mục Sản phẩm (Product Catalog)

### 5.2.1 Đặc tả Use Case

Phân hệ Danh mục Sản phẩm cho phép người dùng khám phá toàn bộ thực đơn của BrewLite, tìm kiếm đồ uống theo tên và lọc theo các phân loại như Cà phê hoặc Trà. Sơ đồ Use Case được biểu diễn tại Hình 5.5.



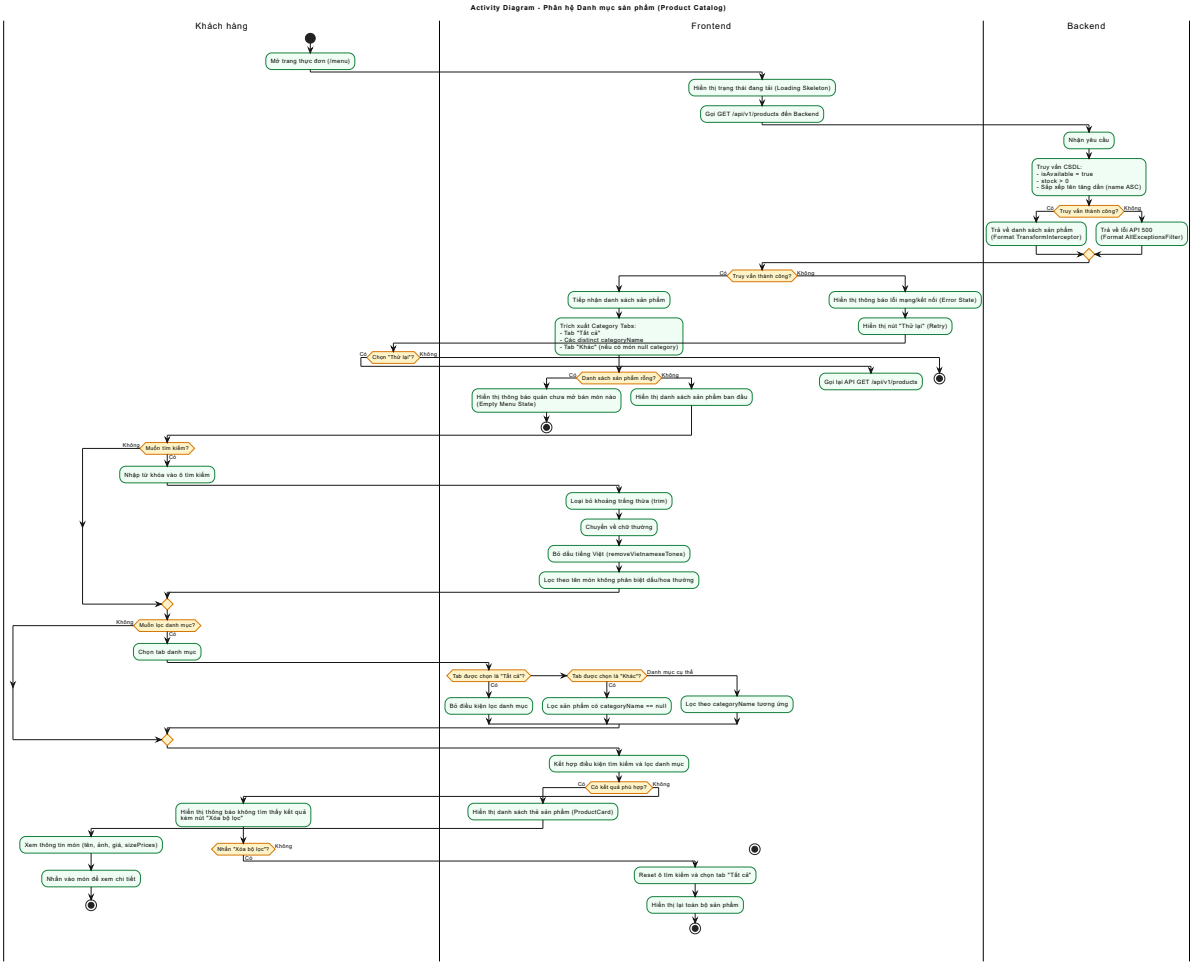

Hình 5.5: Sơ đồ Use Case - Phân hệ Danh mục Sản phẩm

Bảng 5.4: Đặc tả Use Case: Xem và Tìm kiếm Danh mục Sản phẩm (UC-CAT-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách vãng lai, Khách hàng đăng nhập |
| **Tiền điều kiện** | Hệ thống có dữ liệu sản phẩm và danh mục đã được kích hoạt trong CSDL. |
| **Hậu điều kiện** | Danh sách sản phẩm tương ứng hiển thị trực quan trên giao diện thực đơn. |
| **Luồng chính** | 1. Người dùng truy cập trang Thực đơn (`/menu`).<br>2. Giao diện gửi request `GET /api/v1/products`.<br>3. Hệ thống trả về danh sách 10 sản phẩm (6 món cà phê, 4 món trà theo seed data) kèm phân loại danh mục (`categoryName`).<br>4. Khách hàng nhấp chọn tab phân loại (ví dụ ”Cà phê”) hoặc nhập từ khóa tìm kiếm (ví dụ ”Latte”).<br>5. Hook `useProductCatalog` lọc lại danh sách hiển thị tương ứng kèm nhãn giá cơ bản và trạng thái còn hàng/hết hàng. |
| **Luồng ngoại lệ** | - Lỗi kết nối mạng hoặc máy chủ backend: Hiển thị thông báo tải dữ liệu thất bại và cung cấp nút thử lại.<br>- Không tìm thấy sản phẩm phù hợp: Hiển thị thông báo ”Không tìm thấy sản phẩm nào phù hợp”. |

### 5.2.2 Sơ đồ Hoạt động (Activity Diagram)

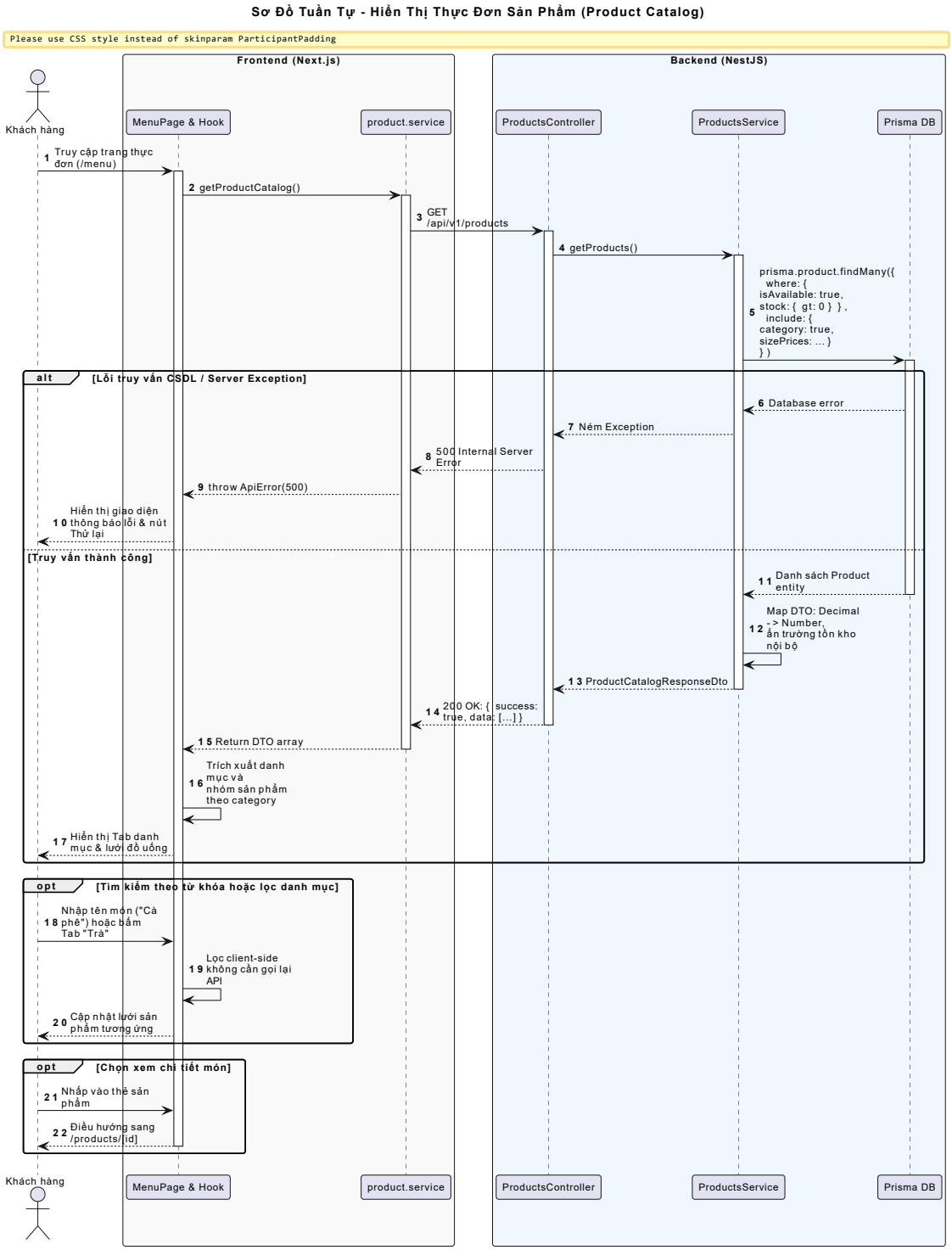
Quy trình nạp và lọc danh mục sản phẩm được mô tả tại Hình 5.6.




Hình 5.6: Sơ đồ Hoạt động - Phân hệ Danh mục Sản phẩm

### 5.2.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh quá trình truy vấn danh mục và sản phẩm từ Frontend đến Backend Controller, Service và Prisma tại Hình 5.7.




Hình 5.7: Sơ đồ Tuần tự - Phân hệ Danh mục Sản phẩm

### 5.2.4 Hợp đồng API (API Contract)

Bảng 5.5: Danh mục API Endpoints - Phân hệ Danh mục Sản phẩm

| Phương thức | Endpoint | Tham số Query | Phản hồi chính |
|---|---|---|---|
| `GET` | `/api/v1/products` | `categoryId`, `search` | `200 OK`: Danh sách sản phẩm kèm thông tin giá cơ bản, phân loại danh mục (`categoryName`), hình ảnh và trạng thái khả dụng. |

### 5.2.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- Truy vấn sản phẩm và danh mục: Backend gọi `prisma.product.findMany({ include: { sizePrices: true, category: true } })`, trả về danh sách sản phẩm mang đầy đủ tên phân loại (`categoryName`) giúp Frontend hiển thị và lọc trực tiếp mà không cần gọi thêm endpoint danh mục độc lập.
- Dữ liệu seed chuẩn gồm 2 danh mục chính (”Cà phê” - 6 món, ”Trà” - 4 món), tổng cộng 10 sản phẩm mẫu được nạp sẵn vào bảng `categories` và `products`.

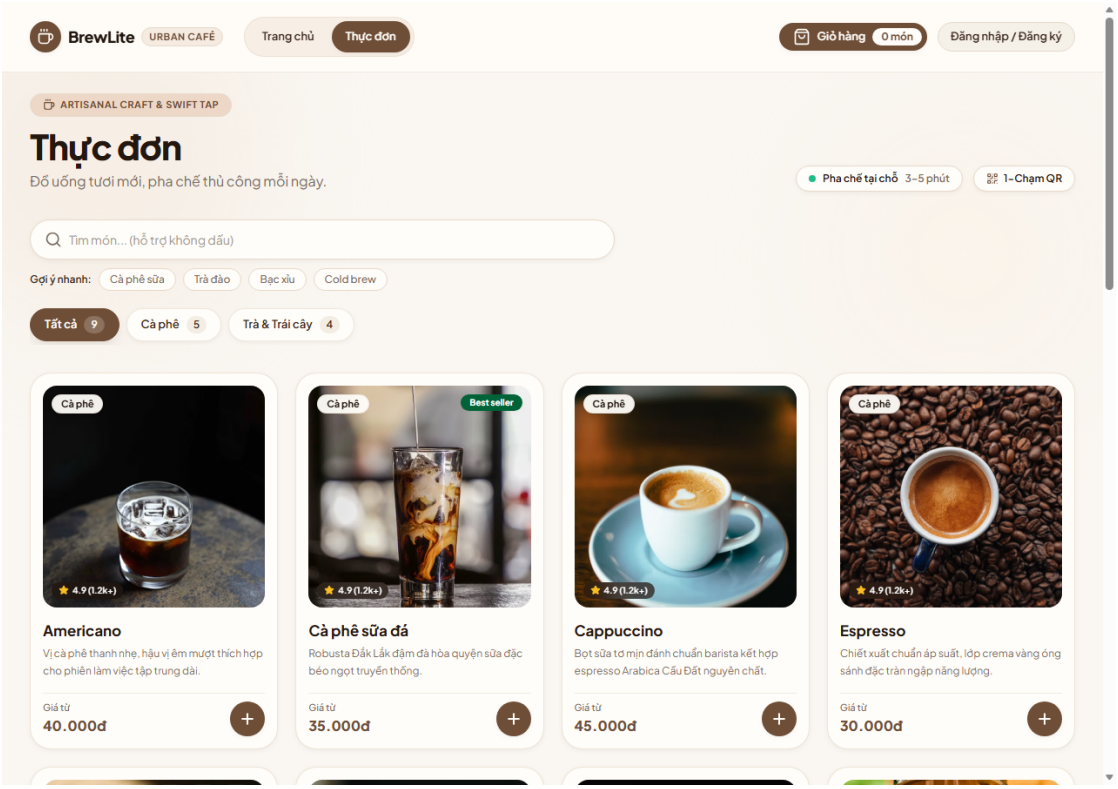
### 5.2.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.6: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Danh mục

| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| `TC-CAT-01` | Hợp lệ | DB có 10 sản phẩm | `GET /products` | Trả về đủ 10 món kèm phân loại danh mục ”Cà phê”, ”Trà & Trái cây” | Đạt |
| `TC-CAT-02` | Hợp lệ | DB có 10 sản phẩm | `GET /products` | Trả về mảng sản phẩm chứa `basePrice`, không lộ `stock` | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Danh mục sản phẩm được trình bày tại Chương 8 và Phụ lục D.

### 5.2.7 Giao diện Người dùng

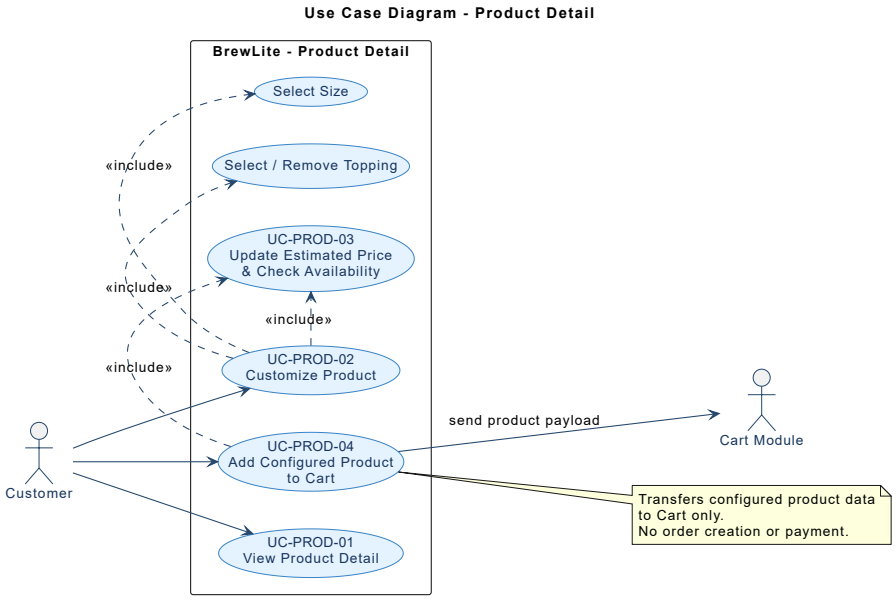



Hình 5.8: Giao diện Thực đơn sản phẩm và Bộ lọc danh mục BrewLite

## 5.3 Phân hệ Chi tiết Sản phẩm (Product Detail)

### 5.3.1 Đặc tả Use Case

Phân hệ Chi tiết Sản phẩm cung cấp thông tin toàn diện về đồ uống, bao gồm mô tả, thành phần dinh dưỡng/nguyên liệu và cho phép khách hàng cá nhân hóa món đồ uống theo sở thích cá nhân. Sơ đồ Use Case được minh họa tại Hình 5.9.



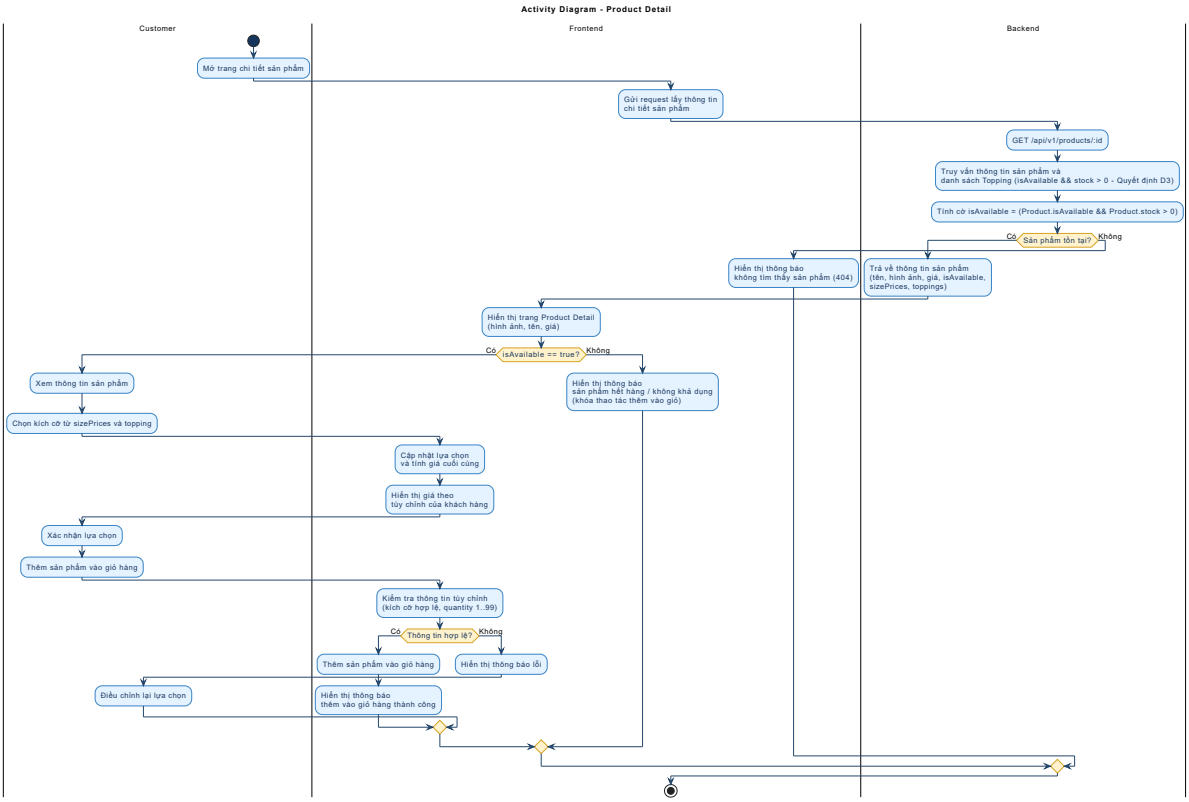

Hình 5.9: Sơ đồ Use Case - Phân hệ Chi tiết Sản phẩm

Bảng 5.7: Đặc tả Use Case: Xem chi tiết và Tùy biến món (UC-DET-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách vãng lai, Khách hàng đăng nhập |
| **Tiền điều kiện** | Sản phẩm tồn tại trên hệ thống và có trạng thái `is_available = true`. |
| **Hậu điều kiện** | Thông tin món, các lựa chọn kích cỡ, mức ngọt, mức đá, danh sách topping và đơn giá tính toán động hiển thị đầy đủ trên màn hình. |
| **Luồng chính** | 1. Khách hàng nhấp vào thẻ món tại trang thực đơn.<br>2. Trình duyệt điều hướng sang `/products/[id]`.<br>3. Frontend gửi request `GET /api/v1/products/:id`.<br>4. Hệ thống phản hồi chi tiết món ăn, bảng giá kích cỡ (`sizePrices`) và danh sách topping khả dụng (`toppings`) trong cùng một phản hồi. Giao diện hiển thị: tên món, hình ảnh, nút ”Quay lại thực đơn”, lựa chọn Size (Mặc định: M), mức đường (0%, 30%, 50%, 70%, 100%), mức đá (0%, 30%, 50%, 70%, 100%) và danh sách Topping kèm giá phụ thu.<br>5. Khi người dùng thay đổi kích cỡ hoặc chọn thêm topping, giao diện tự động tính lại tổng đơn giá của món ăn theo thời gian thực.<br>6. Khách hàng nhấn ”Thêm vào giỏ hàng” để lưu món vào giỏ. |
| **Luồng ngoại lệ** | - Sản phẩm không tồn tại: API trả về mã lỗi `404 Not Found`, giao diện hiển thị thông báo sản phẩm không tìm thấy và nút quay về thực đơn.<br>- Sản phẩm hết hàng (`stock = 0`): Nút ”Thêm vào giỏ hàng” bị vô hiệu hóa (`disabled`). |

### 5.3.2 Sơ đồ Hoạt động (Activity Diagram)

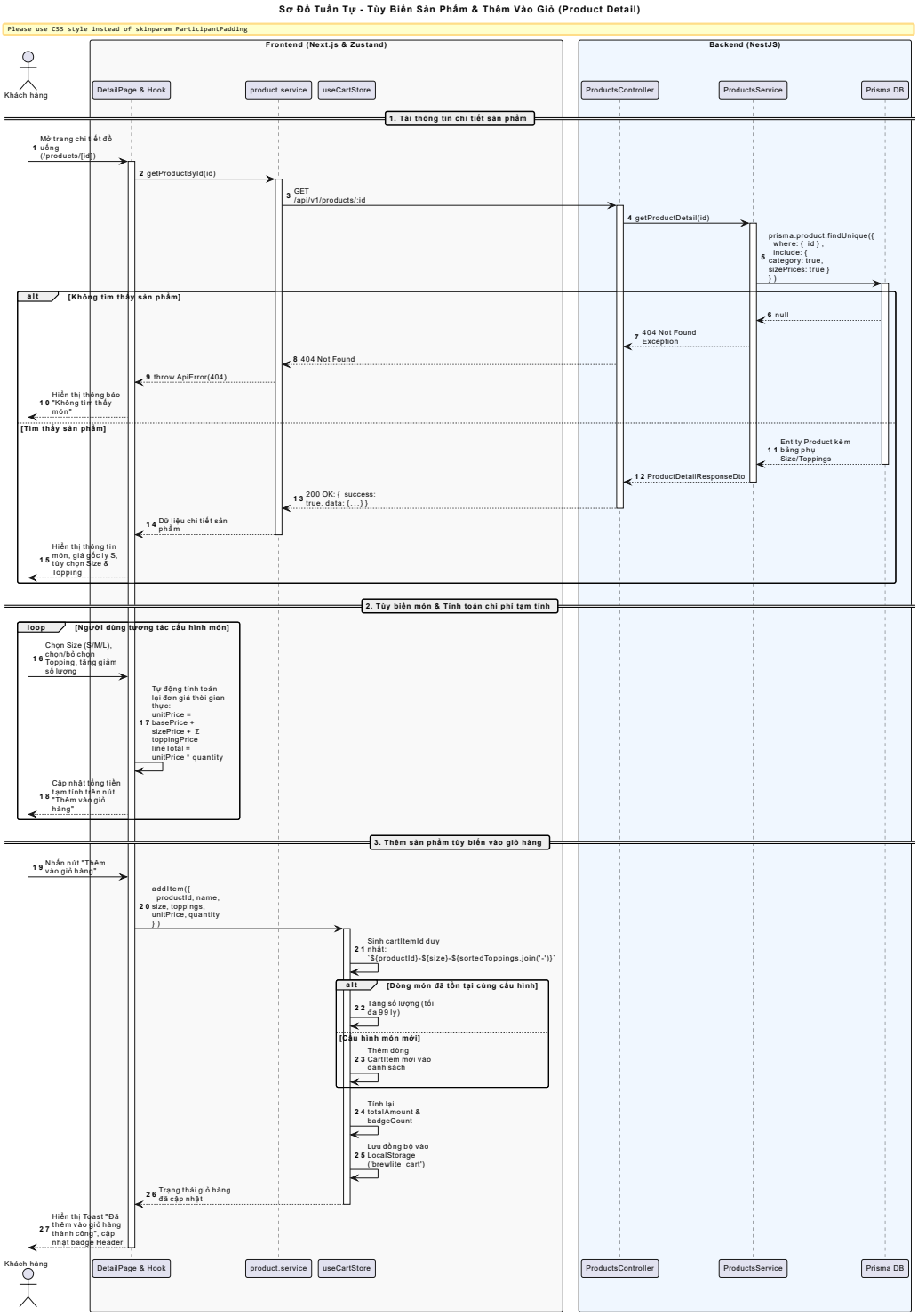
Quy trình người dùng tương tác, chọn lựa tùy biến và kiểm tra trạng thái tồn kho được biểu diễn tại Hình 5.10.




Hình 5.10: Sơ đồ Hoạt động - Phân hệ Chi tiết Sản phẩm

### 5.3.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự thể hiện các tương tác truy vấn dữ liệu chi tiết sản phẩm và danh mục topping tại Hình 5.11.




Hình 5.11: Sơ đồ Tuần tự - Phân hệ Chi tiết Sản phẩm

### 5.3.4 Hợp đồng API (API Contract)

Bảng 5.8: Danh mục API Endpoints - Phân hệ Chi tiết Sản phẩm

| Phương thức | Endpoint | Tham số Đường dẫn | Phản hồi chính |
|---|---|---|---|
| `GET` | `/api/v1/products/:id` | `id` (CUID của sản phẩm) | `200 OK`: Chi tiết sản phẩm kèm mảng cấu hình kích cỡ (`sizePrices`), danh sách `toppings` khả dụng và thông tin `category`. Lỗi `404` nếu không tìm thấy. |

### 5.3.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- Truy vấn chi tiết sản phẩm và topping: `ProductDetailService` thực hiện truy vấn món cùng các bảng quan hệ qua `prisma.product.findUnique({ where: { id }, include: { sizePrices: true, category: true } })`, đồng thời kết hợp danh sách topping đang kinh doanh từ bảng `toppings` (`isAvailable: true`) để đóng gói hoàn chỉnh dữ liệu phản hồi cho client.

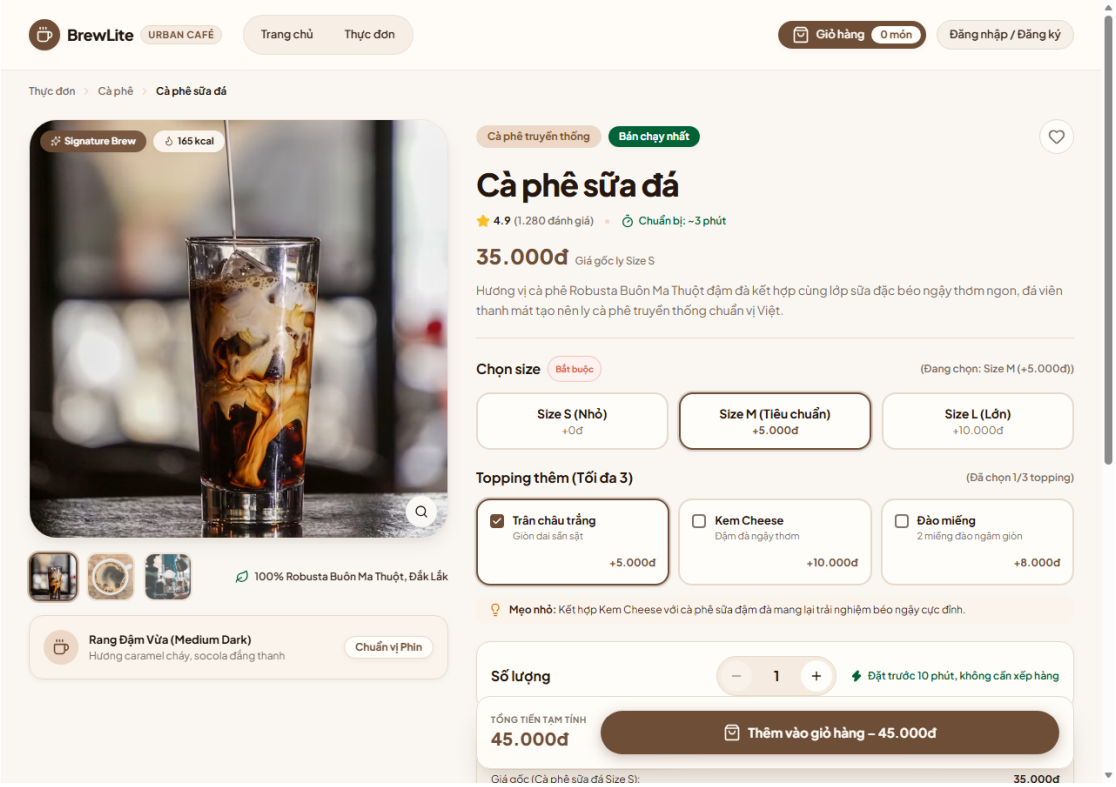
### 5.3.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.9: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Chi tiết Sản phẩm

| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| `TC-DET-01` | Hợp lệ | ID sản phẩm tồn tại | `GET /products/:id` | HTTP 200, nạp đủ chi tiết món, `sizePrices` (S, M, L) và toppings | Đạt |
| `TC-DET-03` | Lỗi | ID không tồn tại | `GET /products/:id` | HTTP 404 Not Found, thông báo ”Không tìm thấy sản phẩm” | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Chi tiết sản phẩm được trình bày tại Chương 8 và Phụ lục D.

### 5.3.7 Giao diện Người dùng

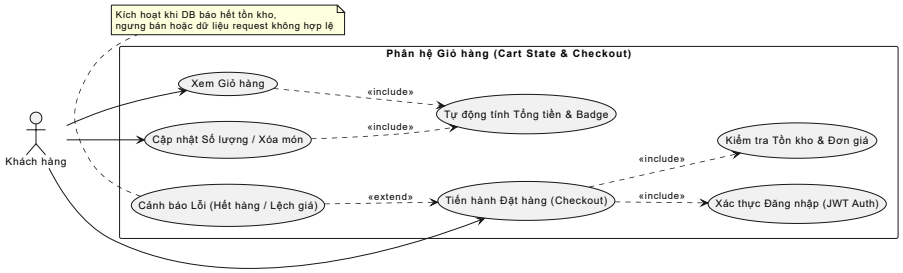



Hình 5.12: Giao diện Chi tiết món và Tùy biến Size / Topping

## 5.4 Phân hệ Giỏ hàng (Cart)

### 5.4.1 Đặc tả Use Case

Phân hệ Giỏ hàng quản lý danh sách các món đồ uống mà khách hàng đã lựa chọn kèm theo các tùy biến cụ thể trước khi tiến hành tạo đơn hàng chính thức. Để tối ưu hóa tài nguyên máy chủ và mang lại phản hồi tức thì cho người dùng, phân hệ này được thiết kế theo kiến trúc lưu trữ phía máy khách (*Client-side State Management*). Sơ đồ Use Case được minh họa tại Hình 5.13.



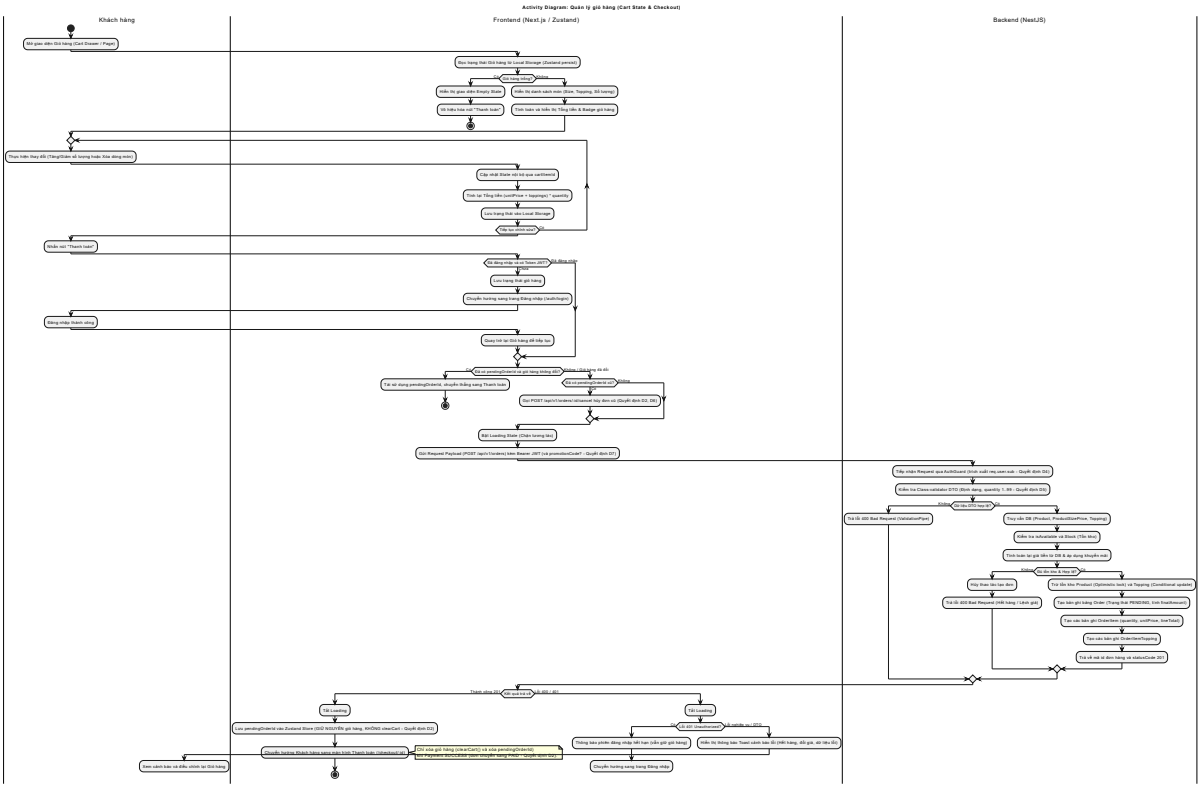

Hình 5.13: Sơ đồ Use Case - Phân hệ Giỏ hàng

Bảng 5.10: Đặc tả Use Case: Quản lý Giỏ hàng (UC-CART-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách vãng lai, Khách hàng đăng nhập |
| **Tiền điều kiện** | Người dùng đã truy cập ứng dụng trên trình duyệt web. |
| **Hậu điều kiện** | Danh sách món trong giỏ hàng, số lượng và tổng tiền tạm tính được cập nhật đồng bộ trong bộ nhớ máy khách (`localStorage` / Zustand state). |
| **Luồng chính** | 1. Người dùng thêm món từ trang chi tiết hoặc nhấn vào biểu tượng giỏ hàng tại thanh điều hướng (`Navbar`).<br>2. Giao diện điều hướng sang trang `/cart`.<br>3. Hệ thống nạp danh sách các món trong giỏ từ bộ nhớ cục bộ.<br>4. Người dùng có thể tăng/giảm số lượng từng món, xóa từng món hoặc xóa toàn bộ giỏ.<br>5. Bộ tính giá phía client tự động tính toán lại mục ”Tạm tính” cho từng dòng và tổng giá trị đơn hàng.<br>6. Khi người dùng nhấn ”Thanh toán” (hoặc ”Tiến hành đặt hàng”), hệ thống kiểm tra điều kiện đăng nhập trước khi chuyển sang bước đặt hàng. |
| **Luồng ngoại lệ** | - Giỏ hàng không có sản phẩm nào: Giao diện hiển thị thông báo ”Giỏ hàng của bạn đang trống” và nút điều hướng ”Xem thực đơn”. |

### 5.4.2 Sơ đồ Hoạt động (Activity Diagram)

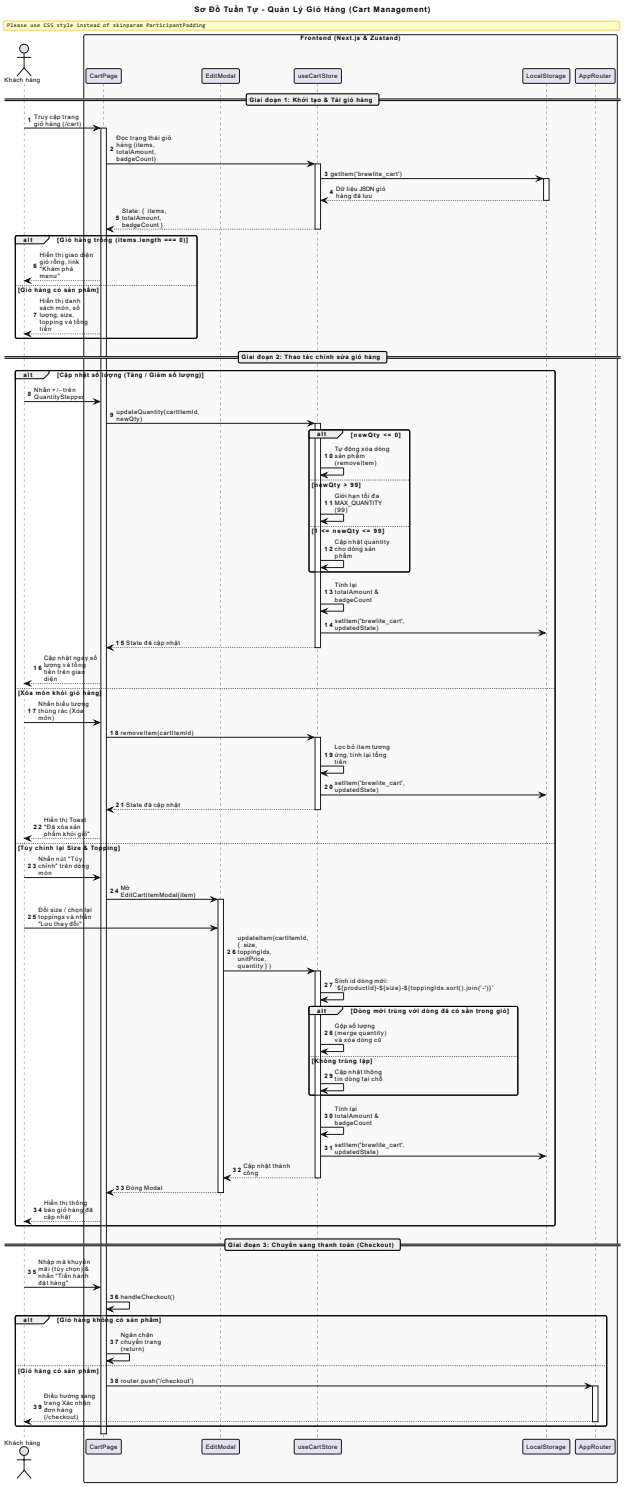
Quy trình quản lý thêm, sửa, xóa và tính toán giá trị giỏ hàng được thể hiện tại Hình 5.14.




Hình 5.14: Sơ đồ Hoạt động - Phân hệ Giỏ hàng

### 5.4.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự minh họa luồng xử lý hoàn toàn khép kín ở phía Frontend giữa giao diện, Store quản lý trạng thái và LocalStorage tại Hình 5.15.




Hình 5.15: Sơ đồ Tuần tự - Phân hệ Giỏ hàng (Client-side Architecture)

### 5.4.4 Hợp đồng API và Ánh xạ Cơ sở dữ liệu

- **Đặc tính kỹ thuật (FE-Only):** Giỏ hàng trong BrewLite hoạt động hoàn toàn ở phía máy khách (*Frontend-only*). Không có API endpoint riêng trên Backend và không tạo bản ghi trong cơ sở dữ liệu cho giỏ hàng tạm thời.
- **Cấu trúc đối tượng lưu trữ (`CartItem`):** Mỗi phần tử giỏ hàng được định danh bởi một khóa duy nhất sinh ra từ tổ hợp thuộc tính (`productId`, `size`, `sweetness`, `ice`, danh sách `toppings`):
  - `id`: Mã duy nhất của món trong giỏ.
  - `productId`: ID sản phẩm tham chiếu.
  - `name`, `image`: Thông tin hiển thị.
  - `size`, `sweetness`, `ice`: Các tùy biến đã chọn.
  - `toppings`: Danh sách topping kèm giá đơn vị.
  - `quantity`: Số lượng đặt ($\geq 1$).
  - `unitPrice`: Đơn giá đã bao gồm phụ thu kích cỡ.
  - `subtotal`: Tổng tiền của dòng món = $(unitPrice + \sum toppingPrice) \times quantity$.

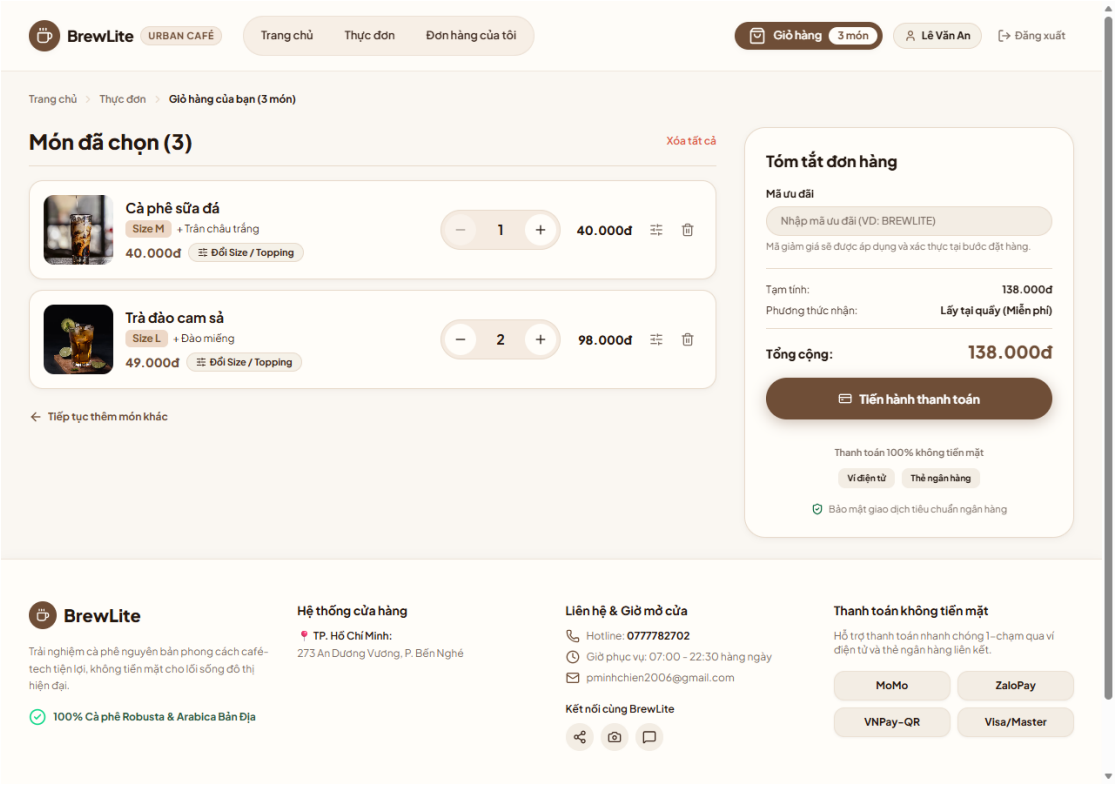
### 5.4.5 Kịch bản Kiểm thử (Test Cases)

Bảng 5.11: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Giỏ hàng

| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| `TC-CART-UI-01` | Giao diện | Giỏ chưa có món | Màn hình `/cart` | Hiển thị thông báo giỏ hàng trống và nút chuyển sang thực đơn | Chưa chạy (sẽ kiểm thử thủ công) |
| `TC-CART-UI-02` | Nghiệp vụ | Đã thêm 1 món | Màn hình `/cart` | Hiển thị chuẩn xác tên món, size, đường, đá và danh sách top-ping | Chưa chạy (sẽ kiểm thử thủ công) |
| `TC-CART-UI-03` | Nghiệp vụ | Đang có món | Nút tăng/giảm | Cập nhật số lượng tức thời và tự động tính lại tổng tiền giỏ hàng | Chưa chạy (sẽ kiểm thử thủ công) |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Giỏ hàng được trình bày tại Chương 8 và Phụ lục D.

### 5.4.6 Giao diện Người dùng

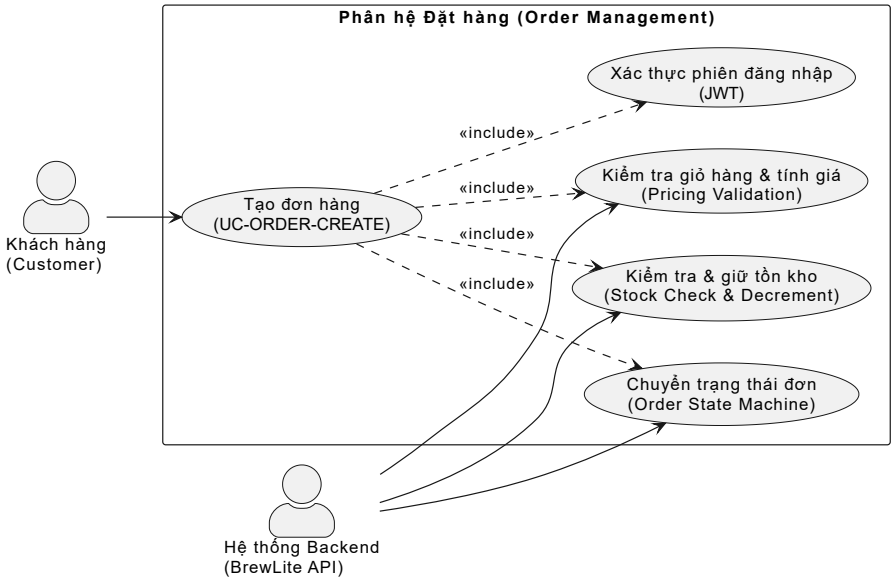



Hình 5.16: Giao diện Quản lý Giỏ hàng và Điều chỉnh số lượng BrewLite

## 5.5 Phân hệ Đặt hàng (Order)

### 5.5.1 Đặc tả Use Case

Phân hệ Đặt hàng là hạt nhân nghiệp vụ của hệ thống BrewLite, chịu trách nhiệm chuyển hóa các sản phẩm trong giỏ hàng thành đơn hàng chính thức. Phân hệ tích hợp các dịch vụ tính giá (`PricingService`), kiểm tra và áp dụng khuyến mãi (`PromotionsService`), tích lũy điểm thành viên (`LoyaltyService`) và kiểm soát kho bằng khóa lạc quan (`Optimistic Locking`). Sơ đồ Use Case được minh họa tại Hình 5.17.



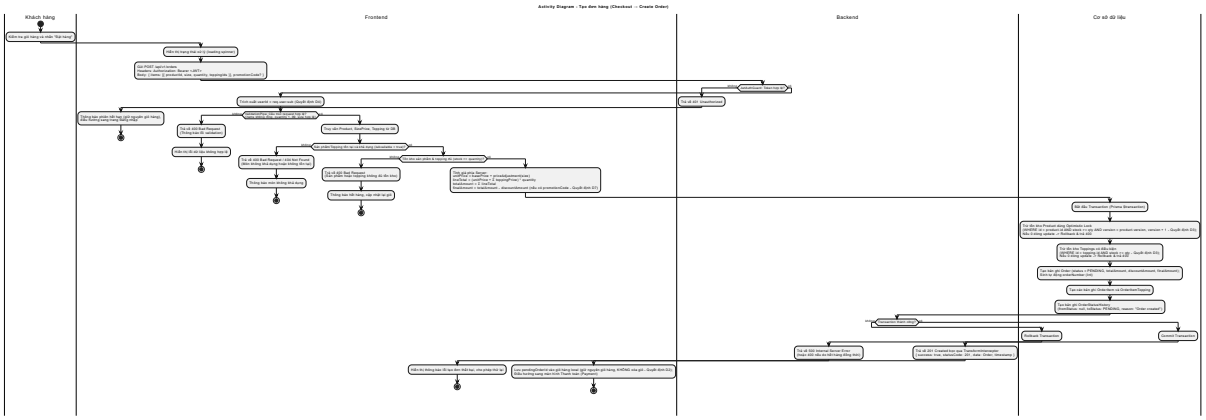

Hình 5.17: Sơ đồ Use Case - Phân hệ Đặt hàng

Bảng 5.12: Đặc tả Use Case: Tạo đơn hàng trực tuyến (UC-ORD-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách hàng đã đăng nhập (Registered Customer) |
| **Tiền điều kiện** | Khách hàng đã đăng nhập (mang Bearer JWT Token); giỏ hàng có ít nhất 1 sản phẩm. |
| **Hậu điều kiện** | Bản ghi đơn hàng mới được tạo ở trạng thái `PENDING`; tồn kho sản phẩm được cập nhật; voucher được áp dụng; bản ghi lịch sử trạng thái được khởi tạo. |
| **Luồng chính** | 1. Khách hàng từ giỏ hàng nhấn ”Tiến hành đặt hàng” đến trang Checkout (`/checkout`).<br>2. Khách hàng nhập mã giảm giá (ví dụ `WELCOME10`) nếu có.<br>3. Nhấn nút ”Xác nhận đặt hàng”.<br>4. Giao diện gửi request `POST /api/v1/orders` kèm Bearer Token.<br>5. Backend khởi động Interactive Transaction (`prisma.$transaction`):<br>a. Kiểm tra từng sản phẩm, đối chiếu `version` và số lượng tồn kho `stock`.<br>b. Tính giá cơ bản, phụ thu size và giá topping qua `PricingService`.<br>c. Thẩm định mã giảm giá và tính số tiền giảm trừ qua `PromotionsService`.<br>d. Tính toán số điểm thưởng dự kiến tích lũy (`pointsEarned`).<br>e. Trừ tồn kho và tăng `version = version + 1` của sản phẩm.<br>f. Tạo bản ghi `orders`, các `order_items` và `order_item_toppings`.<br>g. Thêm bản ghi vào `order_status_history` (chuyển trạng thái sang `PENDING`).<br>6. Transaction commit thành công, trả về thông tin đơn hàng và mã số tự tăng `orderNumber` (ví dụ #1042).<br>7. Giao diện xóa giỏ hàng và điều hướng sang trang thanh toán (`/payment/[orderId]`). |
| **Luồng ngoại lệ** | - Hết hàng hoặc tồn kho không đủ: Báo lỗi `400 Bad Request` (”Sản phẩm ... không đủ tồn kho”), rollback transaction.<br>- Thiếu tồn kho hoặc xung đột phiên bản (Optimistic Lock Conflict): Báo lỗi `400 Bad Request` (`Insufficient stock or concurrent update...`), tự động rollback transaction.<br>- Mã voucher không hợp lệ hoặc hết hạn: Báo lỗi `400 Bad Request` (”Mã giảm giá không hợp lệ”).<br>- Chưa đăng nhập: Báo lỗi `401 Unauthorized`, điều hướng về trang đăng nhập. |

### 5.5.2 Sơ đồ Hoạt động (Activity Diagram)

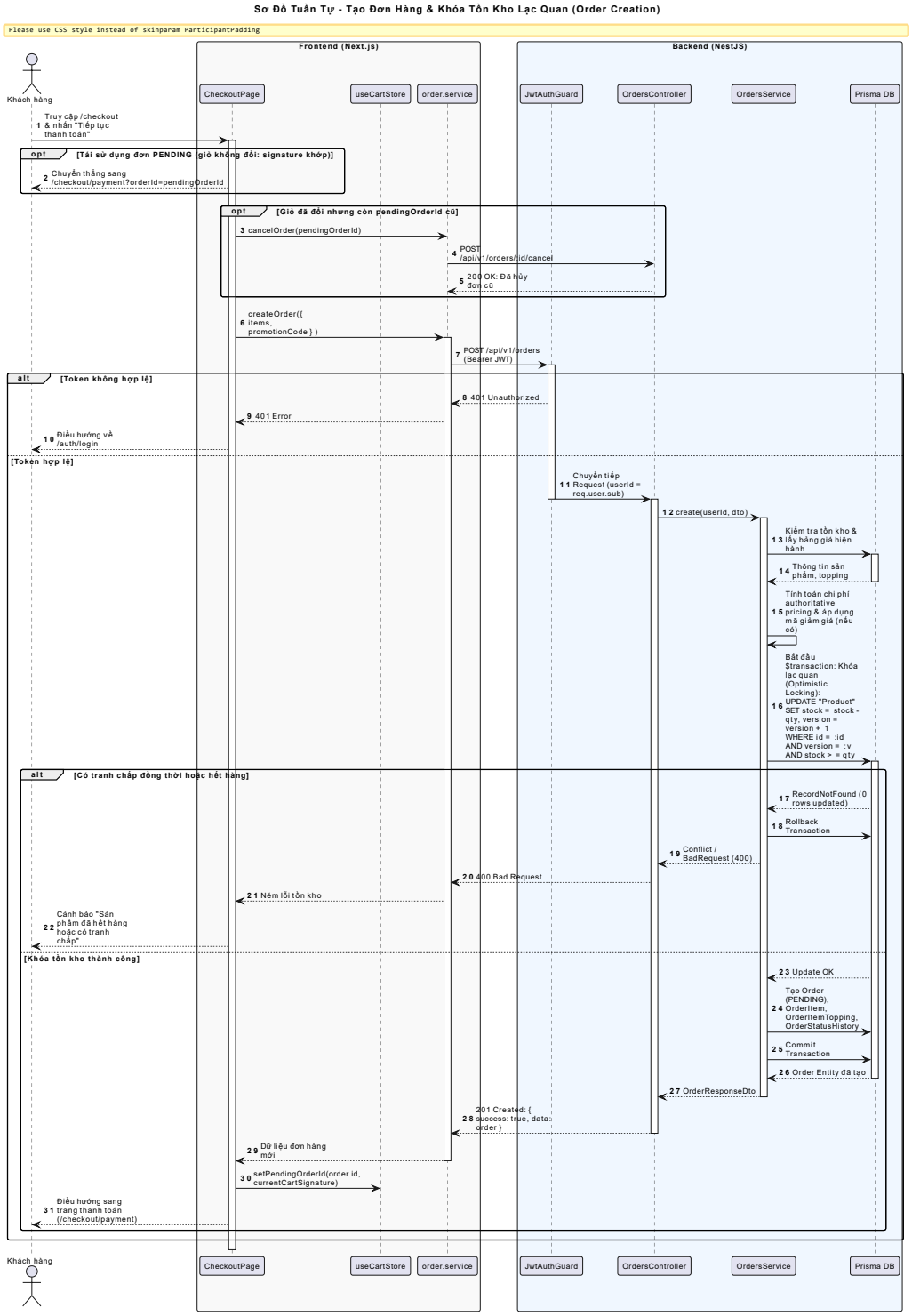
Luồng hoạt động kiểm tra điều kiện tồn kho, tính giá và cam kết giao dịch tạo đơn được biểu diễn tại Hình 5.18.




Hình 5.18: Sơ đồ Hoạt động - Phân hệ Đặt hàng

### 5.5.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự minh họa chi tiết toàn bộ các lời gọi dịch vụ phối hợp trong một transaction tạo đơn hàng tại Hình 5.19.




Hình 5.19: Sơ đồ Tuần tự - Phân hệ Đặt hàng (Interactive Transaction & Pricing Engine)

### 5.5.4 Hợp đồng API (API Contract)

Bảng 5.13: Danh mục API Endpoints - Phân hệ Đặt hàng

| Phương thức | Endpoint | Mô tả Request Payload | Phản hồi chính |
|---|---|---|---|
| POST | /api/v1/orders | {items: [{productId, size, quantity, toppings}], promotionCode?} (Header: Authorization: Bearer <JWT>) | 201 Created: Trả về đơn hàng mới tạo với cấu trúc bọc chuẩn Transform Interceptor. Lỗi: 400 Bad Request, 401 Unauthorized. |
| GET | /api/v1/orders/:id | Tham số id của đơn hàng (Header: Bearer Token) | 200 OK: Chi tiết đơn hàng, danh sách món, trạng thái xử lý và các lần thử thanh toán. |
| POST | /api/v1/orders/:id/cancel | Tham số id của đơn hàng (Header: Bearer Token) | 200 OK: Hủy đơn hàng khi còn ở trạng thái hợp lệ (PENDING/ PAYMENT_ FAILED), hoàn trả tồn kho và cập nhật lịch sử trạng thái. |

### 5.5.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

Quá trình tạo đơn hàng thực thi nguyên tử trong một prisma.$transaction:

```
-- 1. Kiem tra va cap nhat ton kho voi Optimistic Locking
UPDATE products
SET stock = stock - :quantity, version = version + 1
WHERE id = :productId AND version = :currentVersion AND stock >= :quantity;

-- 2. Tao ban ghi don hang chinh thuc
INSERT INTO orders (id, order_number, user_id, status, total_amount,
    discount_amount, final_amount, points_earned, version, created_at,
    updated_at)
VALUES (cuid(), nextval('orders_order_number_seq'), :userId, 'PENDING', :
    totalAmount, :discountAmount, :finalAmount, :pointsEarned, 0, NOW(), NOW
    ());

-- 3. Tao cac chi tiet don hang va luu snapshot gia
INSERT INTO order_items (id, order_id, product_id, size, quantity,
    unit_price, line_total)
VALUES (cuid(), :orderId, :productId, :size, :quantity, :unitPrice, :
    lineTotal);
```

```
13
14 -- 4. Ghi nhat ky trang thai ban dau
15 INSERT INTO order_status_history (id, order_id, from_status, to_status,
       changed_by, created_at)
16 VALUES (cuid(), :orderId, NULL, 'PENDING', 'system', NOW());
```

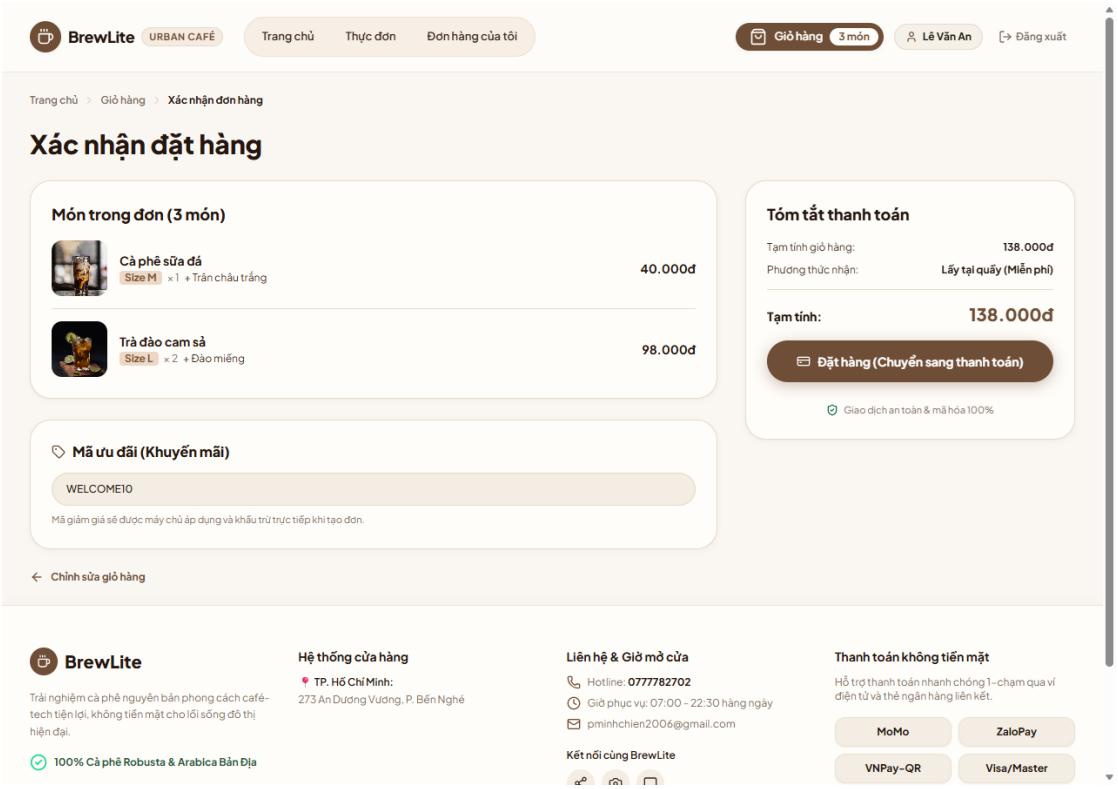
### 5.5.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.14: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Đặt hàng

| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| TC-CO-01 | Hợp lệ | Đủ tồn kho, có Bearer JWT | POST /orders | HTTP 201 Created, tạo đơn ở trạng thái PENDING, trừ tồn kho chính xác | Đạt |
| TC-CO-13 | Lỗi | Tồn kho không đủ | POST /orders | HTTP 400 Bad Request, rollback transaction, tồn kho không bị âm | Đạt |
| TC-CO-18 | Hợp lệ | Áp mã WELCOME10 | POST /orders | Giảm 10% (tối đa 20.000đ), tính toán finalAmount chính xác | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Đặt hàng được trình bày tại Chương 8 và Phụ lục D.

### 5.5.7 Giao diện Người dùng

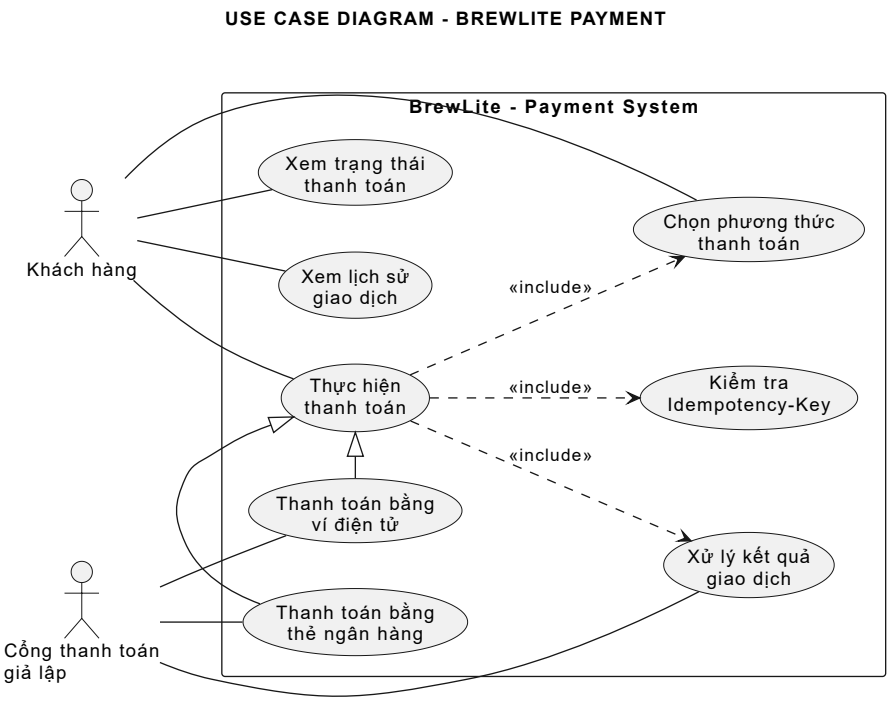



Hình 5.20: Giao diện Xác nhận Đặt hàng và Áp dụng Khuyến mại

## 5.6 Phân hệ Thanh toán (Payment)

### 5.6.1 Đặc tả Use Case

Phân hệ Thanh toán chịu trách nhiệm tiếp nhận và xử lý giao dịch tài chính cho các đơn hàng đang ở trạng thái chờ. Phân hệ được thiết kế với cơ chế an toàn cấp cao: bảo vệ giao dịch bất biến (*Idempotency*) để chống trùng lặp tiền khi người dùng nhấp đúp hoặc xảy ra lỗi chập chờn mạng, đồng thời hỗ trợ nhiều lượt thử lại (*Retry*) cho cùng một đơn hàng thông qua quan hệ 1-N giữa `Order` và `Payment`. Sơ đồ Use Case được minh họa tại Hình 5.21.



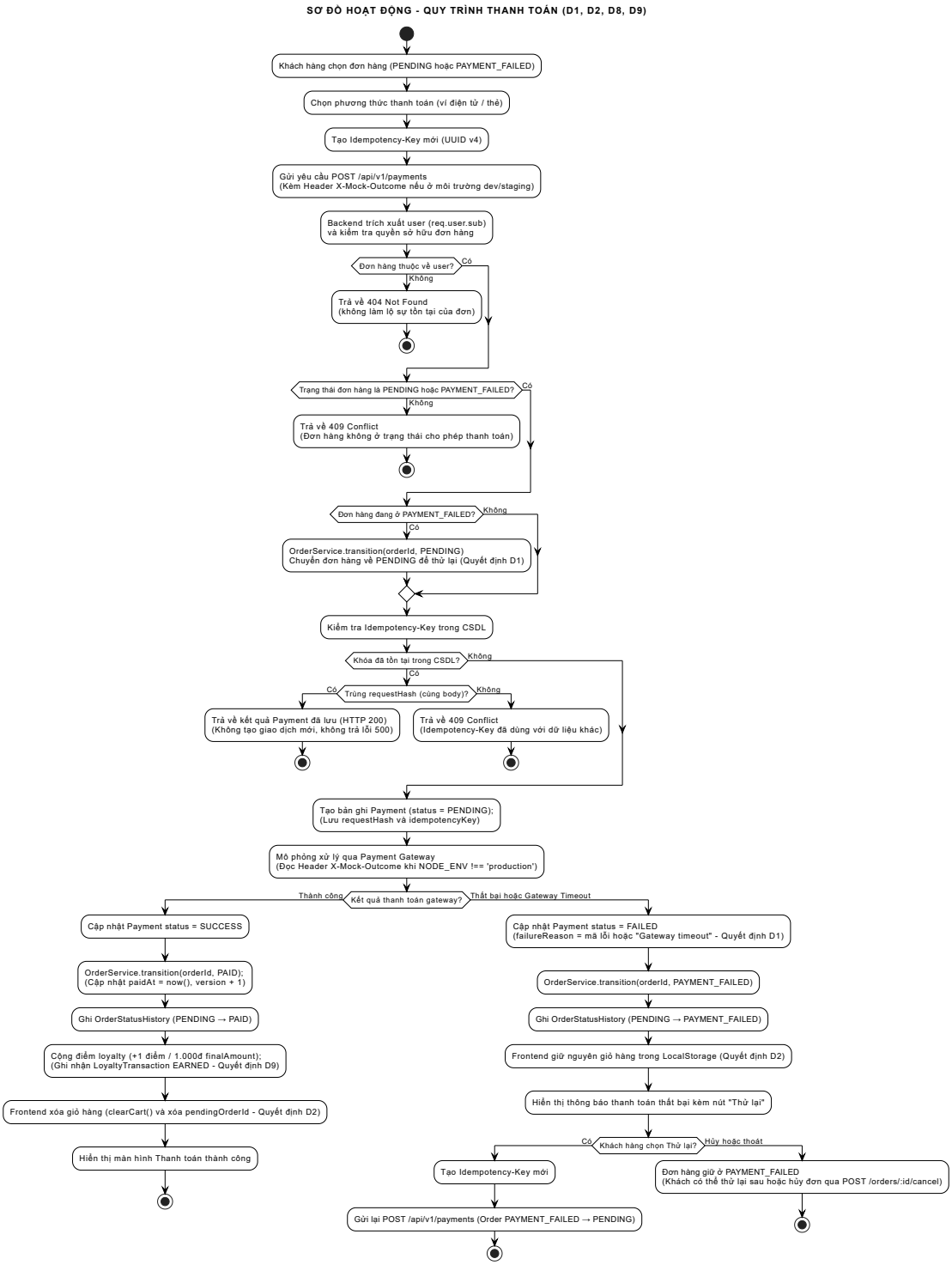

Hình 5.21: Sơ đồ Use Case - Phân hệ Thanh toán

Bảng 5.15: Đặc tả Use Case: Thực hiện Thanh toán Trực tuyến (UC-PAY-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách hàng đã đăng nhập, Cổng thanh toán giả lập (Mock Gateway) |
| **Tiền điều kiện** | Đơn hàng tồn tại ở trạng thái `PENDING`; chưa được thanh toán thành công. |
| **Hậu điều kiện** | Bản ghi thanh toán được lưu trữ; trạng thái đơn hàng chuyển thành `PAID`; điểm loyalty được cộng vào tài khoản khách hàng. |
| **Luồng chính** | 1. Khách hàng truy cập trang thanh toán `/checkout/payment` (hoặc mở Modal xác nhận thanh toán).<br>2. Chọn phương thức thanh toán: Thẻ ngân hàng (`BANK_CARD`) hoặc Ví điện tử (`E_WALLET`).<br>3. Nhấn "Thanh toán ngay".<br>4. Trình duyệt tự động sinh mã UUID `Idempotency-Key` duy nhất và gửi request `POST /api/v1/payments`.<br>5. PaymentsService kiểm tra `idempotencyKey`:<br>a. Nếu khóa đã tồn tại trong DB: Gọi `replayExistingPayment()` đối chiếu mã băm SHA-256 trên `{orderId, method}`. Nếu trùng khớp, trả lại kết quả đã lưu mà không gọi lại Mock Gateway; nếu sai lệch tải trọng, ném ngoại lệ `409 Conflict`.<br>b. Nếu là khóa mới: Khởi tạo giao dịch `payments` trong `prisma.$transaction` với trạng thái `PENDING` và lưu mã băm `requestHash`.<br>c. Xử lý tranh chấp đồng thời: Nếu hai request cùng gửi một khóa, lỗi trùng khóa Unique (`P2002`) được bắt trong khối `catch`, tự động truy vấn lại giao dịch vừa tạo và phát lại (*replay*) kết quả an toàn.<br>6. Backend gọi Mock Payment Gateway để xác thực giao dịch.<br>7. Khi Gateway phản hồi thành công (`SUCCESS`):<br>a. Cập nhật bản ghi `payments` thành `SUCCESS` kèm `transactionRef`.<br>b. Cập nhật đơn hàng: `status = PAID`, `paidAt = NOW()`.<br>c. Cộng điểm tích lũy mới vào `users.loyalty_points`.<br>d. Ghi nhận giao dịch cộng điểm vào `loyalty_transactions` (type: `EARNED`).<br>e. Ghi nhật ký vào `order_status_history` (PENDING → PAID).<br>8. Giao diện thông báo thanh toán thành công và điều hướng sang trang Lịch sử đơn hàng. |
| **Luồng ngoại lệ** | - Gateway từ chối hoặc giao dịch thất bại: Cập nhật bản ghi payment thành `FAILED`, đơn hàng chuyển sang `PAYMENT_FAILED`, cho phép khách hàng nhấn thử lại (tạo lượt payment mới).<br>- Đơn hàng đã được thanh toán trước đó hoặc ở trạng thái không cho phép: Báo lỗi `409 Conflict` ("Trạng thái đơn hàng không cho phép thanh toán").<br>- Trùng Idempotency-Key nhưng khác payload: Báo lỗi `409 Conflict` (Ïdempotency-Key đã được sử dụng với nội dung khác)̈. |

### 5.6.2 Sơ đồ Hoạt động (Activity Diagram)

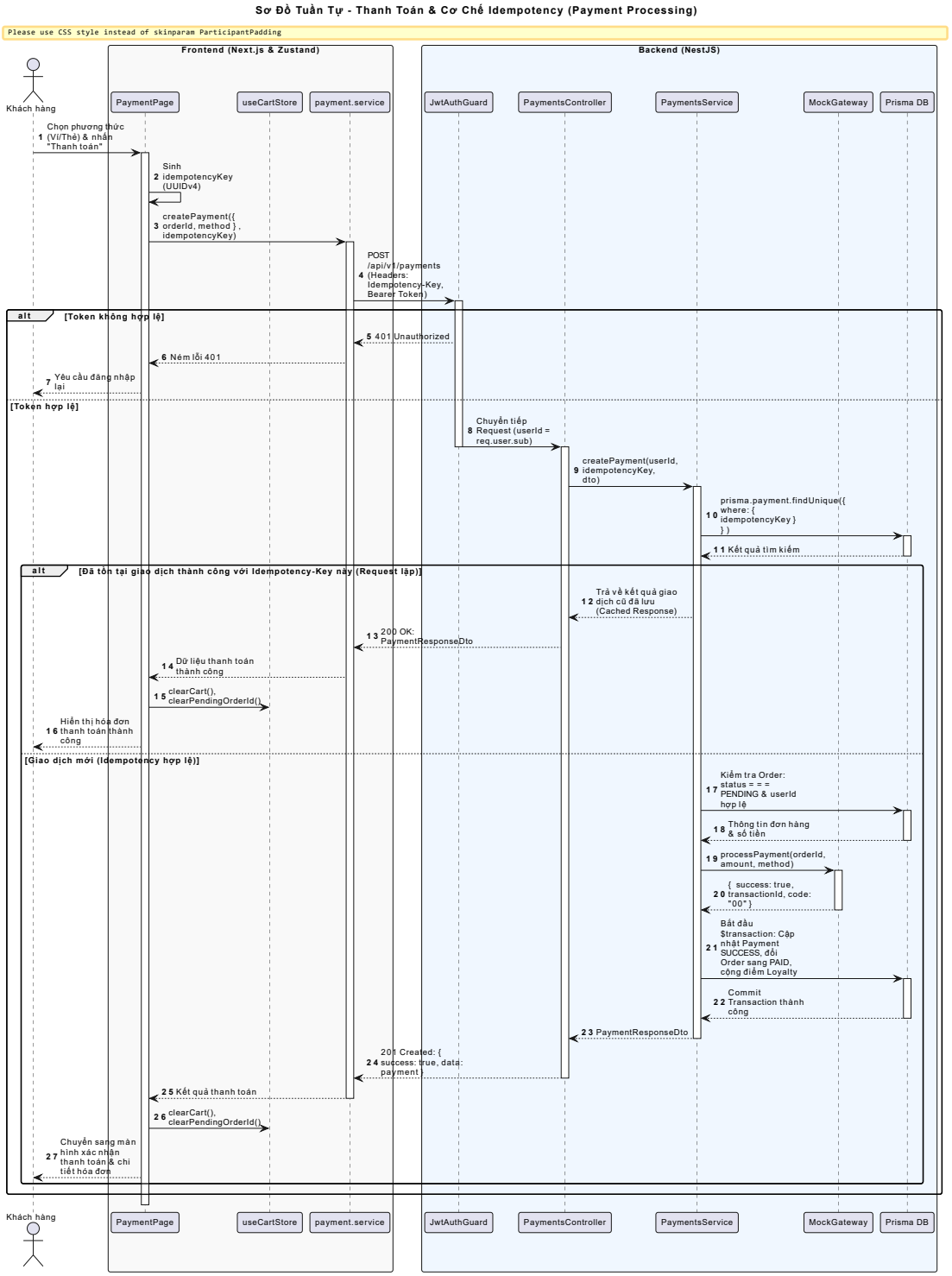
Quy trình thẩm định Idempotency, gọi cổng thanh toán và xử lý kết quả giao dịch được mô tả tại Hình 5.22.




Hình 5.22: Sơ đồ Hoạt động - Phân hệ Thanh toán

### 5.6.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự phản ánh toàn diện các luồng tương tác giữa Frontend, PaymentsController, PaymentsService, Prisma DB và Mock Payment Gateway tại Hình 5.23.




Hình 5.23: Sơ đồ Tuần tự - Phân hệ Thanh toán (Idempotent Flow & Gateway Integration)

### 5.6.4 Hợp đồng API (API Contract)

Bảng 5.16: Danh mục API Endpoints - Phân hệ Thanh toán

| Phương thức | Endpoint | Headers & Body | Phản hồi chính |
|---|---|---|---|
| POST | /api/v1/payments | Header: Idempotency-Key: <UUID>, Bearer Token. Body: {orderId, method} | 201 Created: Trả về chi tiết thanh toán thành công (hoặc kết quả phát lại an toàn). Lỗi: 400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict (sai lệch requestHash hoặc trạng thái đơn không cho phép). |

### 5.6.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- **Cơ chế Idempotency:** Bảng payments sử dụng ràng buộc @unique trên trường idempotency_key kết hợp cùng request_hash sinh từ thuật toán SHA-256 trên nội dung JSON của body request:

```
SELECT id, status, request_hash, transaction_ref
FROM payments
WHERE idempotency_key = :idempotencyKey;
```

- **Cập nhật sau thanh toán thành công:** Thực thi trong transaction cập nhật đồng thời 4 bảng:

```
UPDATE payments SET status = 'SUCCESS', transaction_ref = :ref WHERE id = :paymentId;
UPDATE orders SET status = 'PAID', paid_at = NOW() WHERE id = :orderId;
UPDATE users SET loyalty_points = loyalty_points + :pointsEarned WHERE id = :userId;
INSERT INTO loyalty_transactions (id, user_id, order_id, points, type, reason, created_at)
VALUES (cuid(), :userId, :orderId, :pointsEarned, 'EARNED', 'Tich diem tu don hang', NOW());
INSERT INTO order_status_history (id, order_id, from_status, to_status, changed_by, created_at)
VALUES (cuid(), :orderId, 'PENDING', 'PAID', 'payment_gateway', NOW());
```

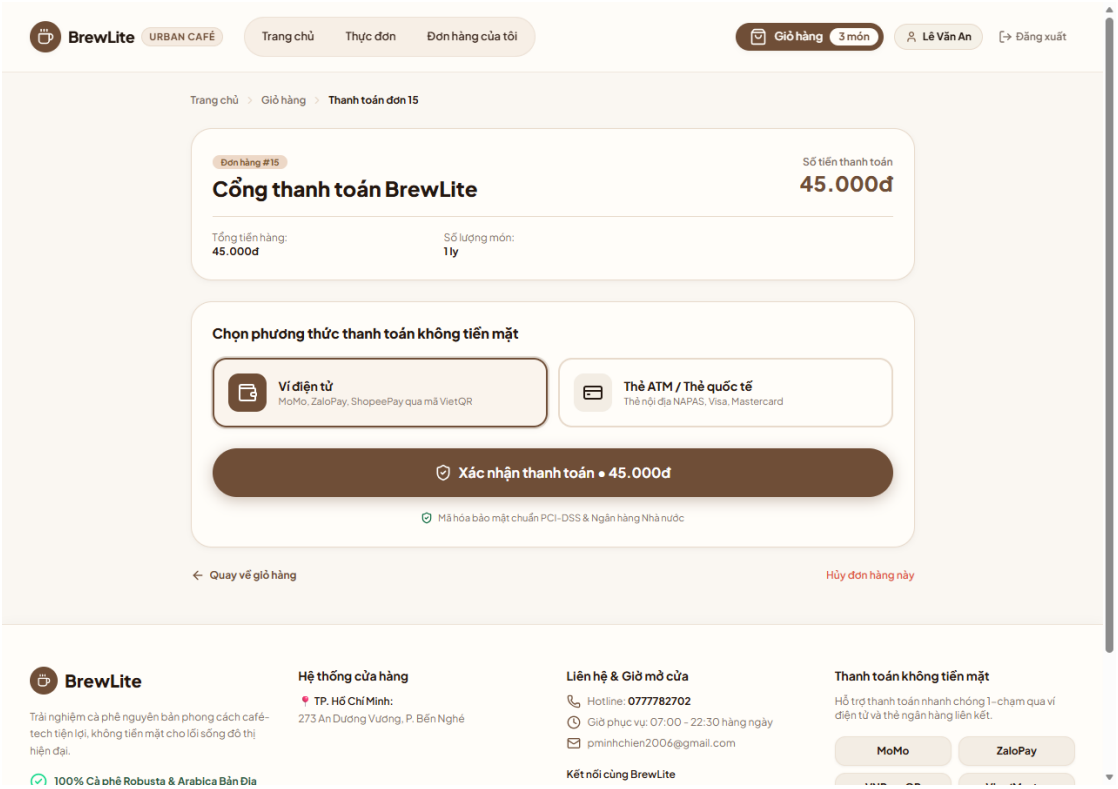
### 5.6.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.17: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Thanh toán

| **Mã TC** | **Loại** | **Tiền điều kiện** | **Endpoint / Màn hình** | **Kết quả mong đợi** | **Trạng thái** |
|---|---|---|---|---|---|
| `TC-PAY-01` | Hợp lệ | Đơn PENDING, key mới | `POST /payments` | HTTP 201 Created, status SUCCESS, đơn chuyển sang PAID, tích lũy loyalty | Đạt |
| `TC-PAY-10` | Idempotent | Gửi lại cùng key | `POST /payments` | Trả về kết quả cũ đã lưu (replay an toàn, HTTP 201 Created), không phát sinh trùng lặp | Đạt |
| `TC-PAY-09` | Lỗi | Đơn đã thanh toán | `POST /payments` | HTTP 409 Conflict, message ”Trạng thái đơn hàng không cho phép thanh toán” | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Thanh toán được trình bày tại Chương 8 và Phụ lục D.

### 5.6.7 Giao diện Người dùng

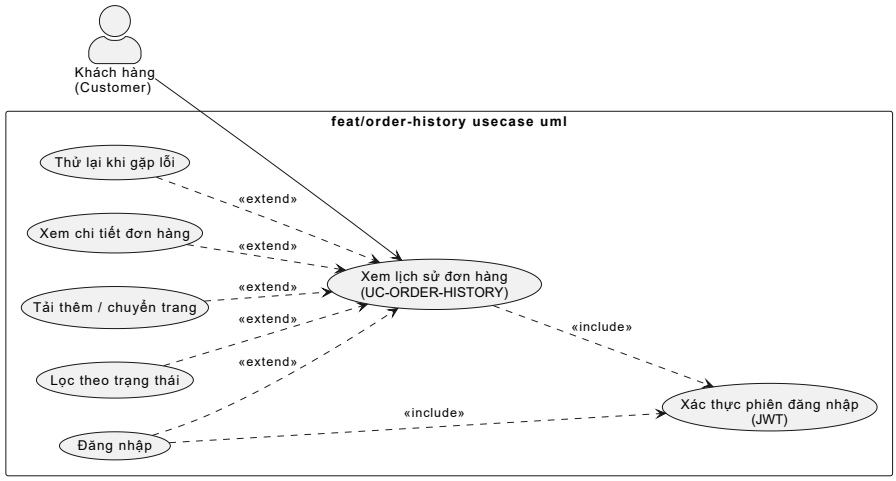



Hình 5.24: Giao diện Cổng thanh toán không tiền mặt đa phương thức

## 5.7 Phân hệ Lịch sử Đơn hàng (Order History)

### 5.7.1 Đặc tả Use Case

Phân hệ Lịch sử Đơn hàng cho phép khách hàng tra cứu toàn bộ các đơn hàng đã đặt trong quá khứ, theo dõi trạng thái pha chế và kiểm tra hóa đơn chi tiết. Sơ đồ Use Case được minh họa tại Hình 5.25.



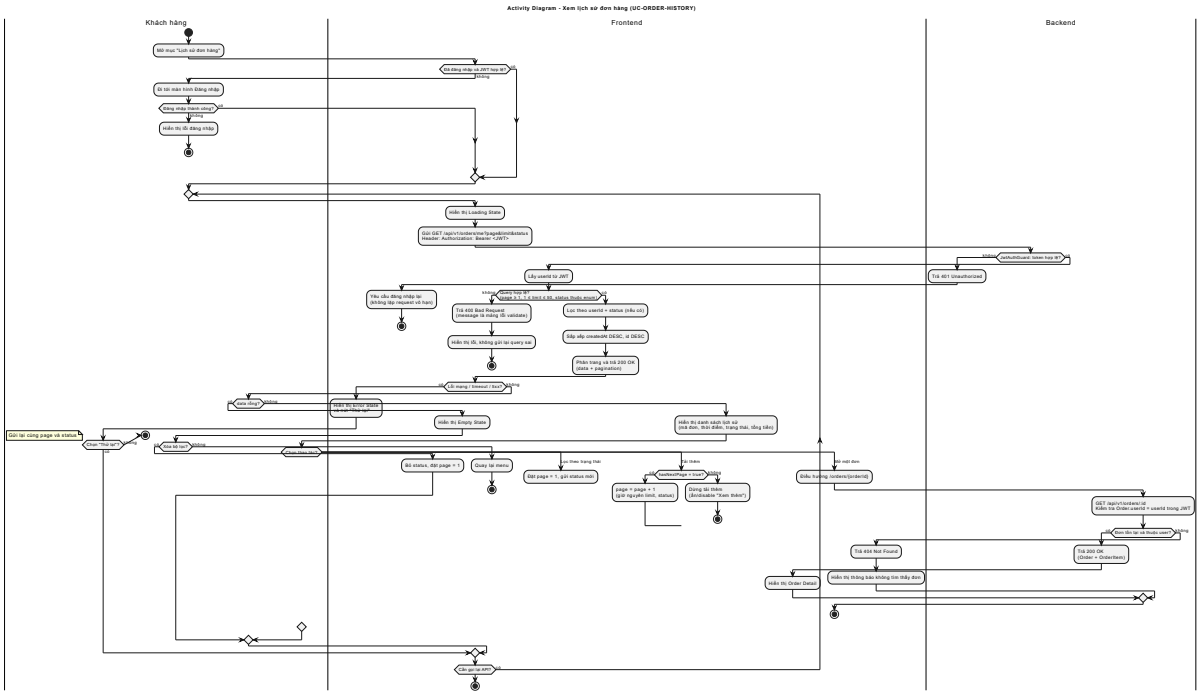

Hình 5.25: Sơ đồ Use Case - Phân hệ Lịch sử Đơn hàng

Bảng 5.18: Đặc tả Use Case: Tra cứu Lịch sử và Chi tiết Đơn hàng (UC-HIS-01)

| **Mục** | **Chi tiết Đặc tả** |
|---|---|
| **Actor** | Khách hàng đã đăng nhập |
| **Tiền điều kiện** | Khách hàng đã đăng nhập vào hệ thống. |
| **Hậu điều kiện** | Danh sách các đơn hàng và trạng thái chi tiết hiển thị trực quan. |
| **Luồng chính** | 1. Khách hàng nhấp vào mục ”Lịch sử đơn hàng” trên thanh điều hướng hoặc truy cập `/orders`.<br>2. Giao diện gửi request `GET /api/v1/orders/me` kèm Bearer Token.<br>3. Backend truy vấn các đơn hàng thuộc về `userId` hiện tại, sắp xếp giảm dần theo thời gian tạo (`createdAt desc`).<br>4. Màn hình hiển thị danh sách các thẻ đơn hàng: mã hiển thị `#orderNumber`, ngày giờ đặt, tổng tiền, huy hiệu trạng thái (ví dụ ”Đã thanh toán”, ”Đã hủy”).<br>5. Khách hàng nhấp vào một đơn hàng để xem chi tiết hóa đơn (`/orders/[id]`): danh sách từng món, size, topping, số tiền giảm giá và phương thức thanh toán. |
| **Luồng ngoại lệ** | - Người dùng chưa có đơn hàng nào: Giao diện hiển thị thông báo ”Bạn chưa có đơn hàng nào” và nút liên kết ”Bắt đầu đặt món”.<br>- Phiên đăng nhập hết hạn: Báo lỗi `401 Unauthorized`, tự động chuyển hướng về trang đăng nhập. |

### 5.7.2 Sơ đồ Hoạt động (Activity Diagram)

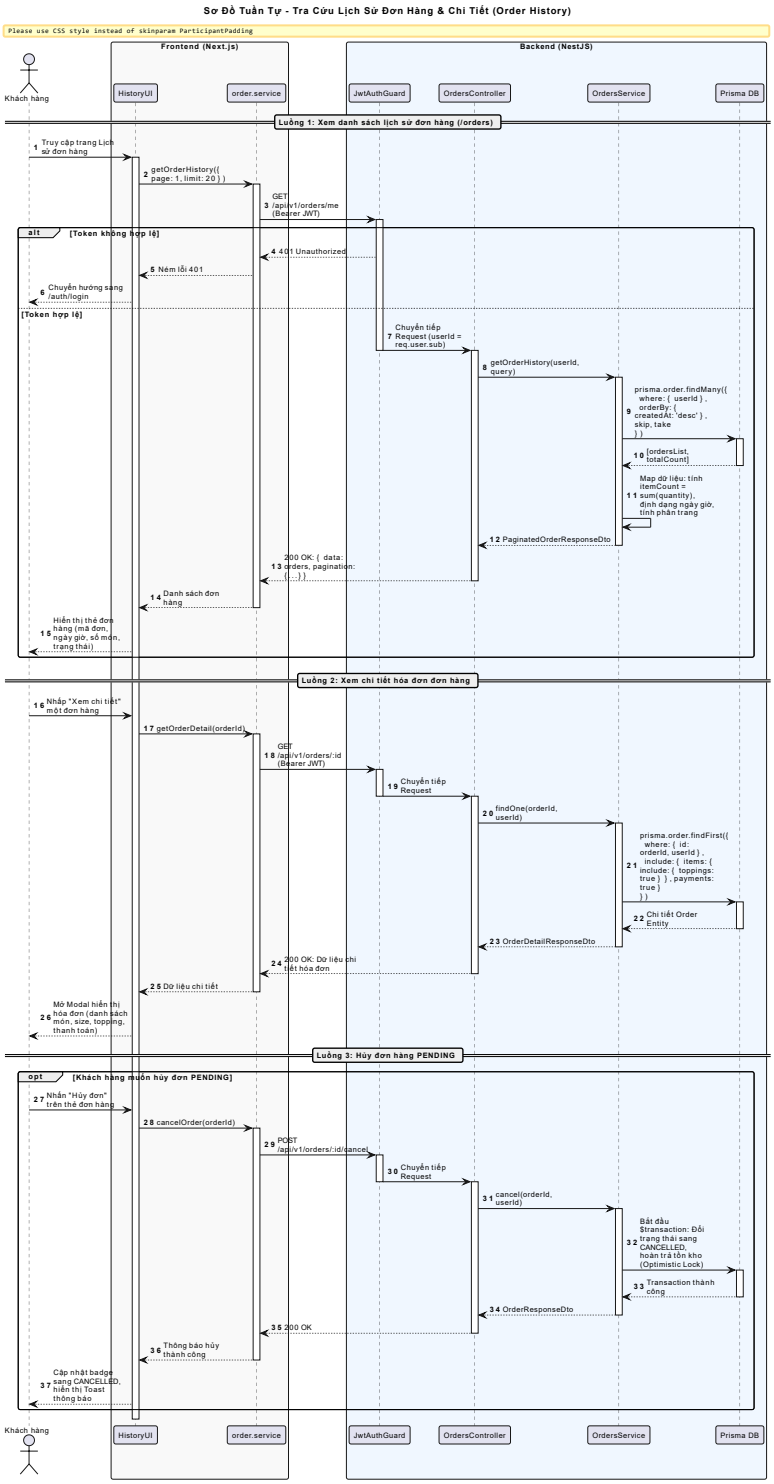
Quy trình truy vấn danh sách và xem chi tiết trạng thái đơn hàng được thể hiện tại Hình 5.26.




Hình 5.26: Sơ đồ Hoạt động - Phân hệ Lịch sử Đơn hàng

### 5.7.3 Sơ đồ Tuần tự (Sequence Diagram)

Sơ đồ Tuần tự minh họa tương tác nạp dữ liệu lịch sử đơn hàng từ Frontend đến Controller, Service và Prisma tại Hình 5.27.




Hình 5.27: Sơ đồ Tuần tự - Phân hệ Lịch sử Đơn hàng

### 5.7.4 Hợp đồng API (API Contract)

Bảng 5.19: Danh mục API Endpoints - Phân hệ Lịch sử Đơn hàng

| Phương thức | Endpoint | Header | Phản hồi chính |
|---|---|---|---|
| GET | /api/v1/orders/me | Authorization: Bearer <JWT> | 200 OK: Danh sách các đơn hàng của user đang đăng nhập, kèm tổng tiền và trạng thái. |
| GET | /api/v1/orders/:id | Authorization: Bearer <JWT> | 200 OK: Chi tiết đơn hàng gồm các món (items), topping, giao dịch thanh toán và lịch sử trạng thái. Lỗi 404 nếu không tìm thấy hoặc không thuộc sở hữu. |

### 5.7.5 Ánh xạ Cơ sở dữ liệu (Database Mapping)

- Truy vấn danh sách đơn hàng cá nhân:

```
SELECT id, order_number, status, total_amount, discount_amount,
    final_amount, created_at
FROM orders
WHERE user_id = :userId
ORDER BY created_at DESC;
```

- Tối ưu hóa: Chỉ mục `@@index([userId])` và `@@index([status])` trên bảng `orders` đảm bảo câu truy vấn thực thi với độ phức tạp $\mathcal{O}(\log N)$, đáp ứng yêu cầu phản hồi nhanh của hệ thống.

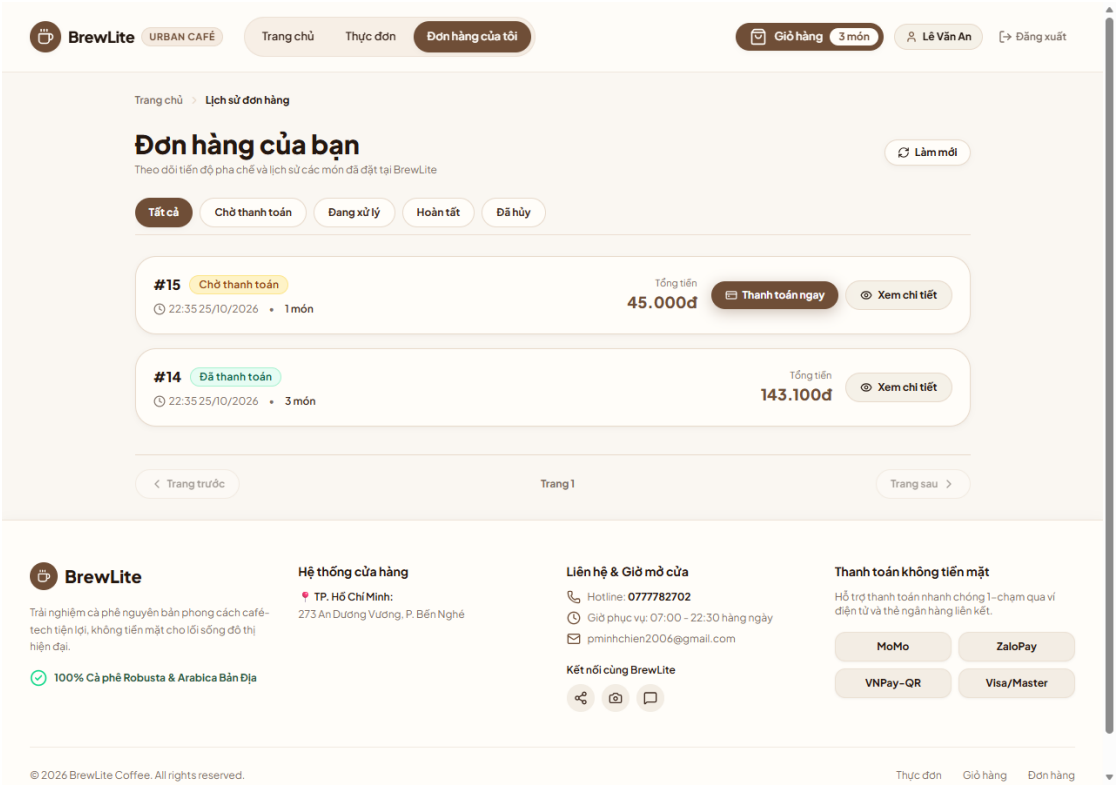
### 5.7.6 Kịch bản Kiểm thử (Test Cases)

Bảng 5.20: Tóm lược các Ca kiểm thử tiêu biểu - Phân hệ Lịch sử

| Mã TC | Loại | Tiền điều kiện | Endpoint / Màn hình | Kết quả mong đợi | Trạng thái |
|---|---|---|---|---|---|
| TC-HIS-01 | Hợp lệ | User đã có đơn hàng | GET /orders/me | HTTP 200, trả về danh sách đơn của chính người dùng sắp xếp mới nhất | Đạt |
| TC-HIS-06 | Bảo mật | Xem đơn của user khác | GET /orders/:id | HTTP 404 Not Found, cô lập dữ liệu an toàn, không làm lộ đơn người khác | Đạt |
| TC-CX-01 | Nghiệp vụ | Đơn ở trạng thái PENDING | POST /orders/:id/cancel | HTTP 200 OK, đơn chuyển sang CANCELLED, hoàn trả tồn kho nguyên tử | Đạt |

*Ghi chú:* Chi tiết toàn bộ kết quả kiểm thử phân hệ Lịch sử đơn hàng được trình bày tại Chương 8 và Phụ lục D.

### 5.7.7 Giao diện Người dùng




Hình 5.28: Giao diện Lịch sử và Trạng thái Đơn hàng của Khách hàng

# Chương 6

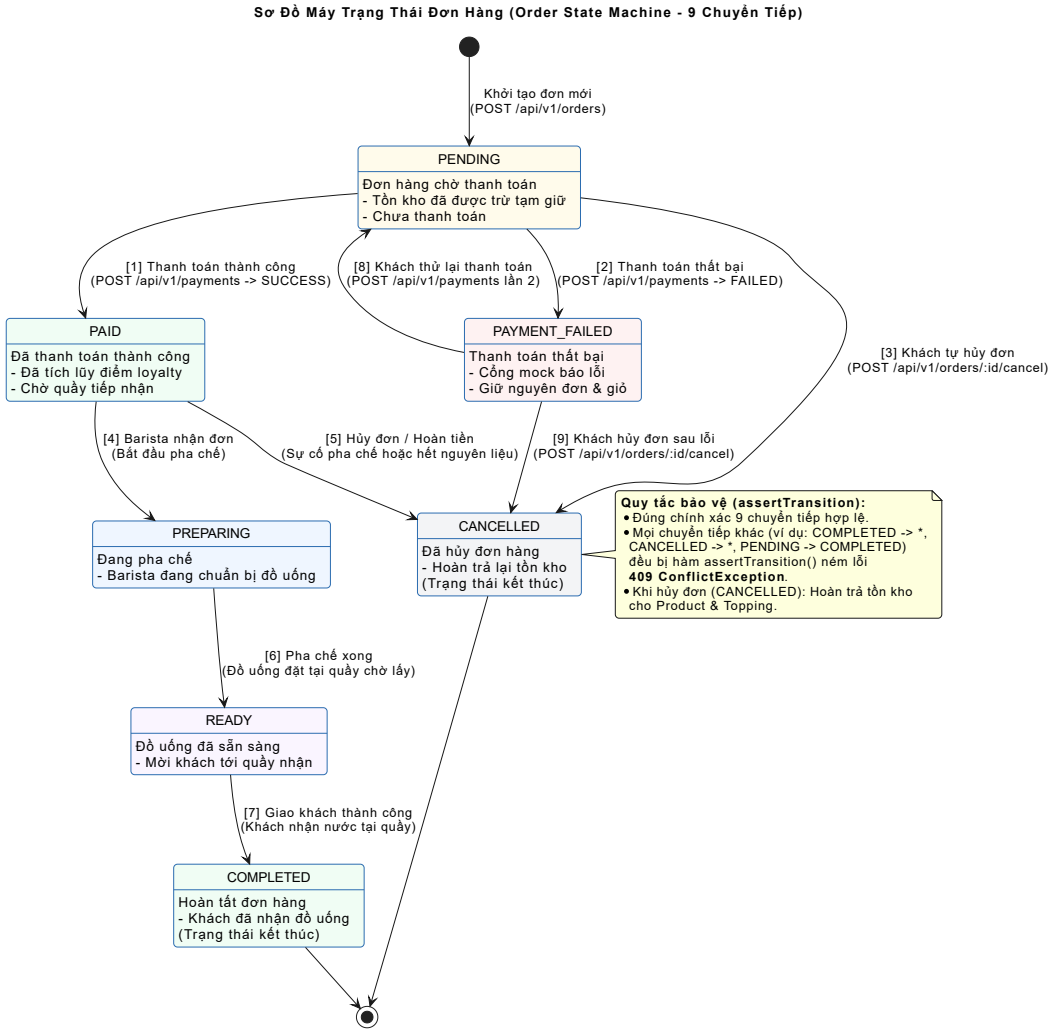
# HIỆN THỰC CÁC NGHIỆP VỤ NÂNG CAO

Chương này đi sâu vào các giải pháp kỹ thuật phức tạp đã được thiết kế và hiện thực hóa trong hệ thống BrewLite nhằm giải quyết các bài toán hóc búa về tính toàn vẹn dữ liệu, kiểm soát đồng thời và độ tin cậy giao dịch trong môi trường đa người dùng.

## 6.1 Máy trạng thái Đơn hàng (Order State Machine)

Quy trình luân chuyển trạng thái của một đơn hàng trong BrewLite được quản lý chặt chẽ thông qua mô hình Máy trạng thái hữu hạn (*Finite State Machine*), định nghĩa nguyên văn trong tập tin mã nguồn `backend/src/modules/orders/order-state.ts`.

Sơ đồ máy trạng thái đơn hàng được trình bày chi tiết tại Hình 6.1.




Hình 6.1: Sơ đồ Máy trạng thái Đơn hàng BrewLite (9 chuyển tiếp hợp lệ)

Hệ thống quy định chính xác **9 chuyển tiếp trạng thái hợp lệ** giữa 7 trạng thái nghiệp vụ:

1. `PENDING` → `PAID`: Chuyển đổi khi khách hàng thực hiện thanh toán trực tuyến thành công qua cổng thanh toán.
2. `PENDING` → `PAYMENT_FAILED`: Chuyển đổi khi giao dịch thanh toán bị từ chối, thẻ hết hạn hoặc phát sinh lỗi mạng.
3. `PENDING` → `CANCELLED`: Khách hàng chủ động hủy đơn khi chưa thanh toán hoặc đơn hàng hết thời gian chờ (timeout).
4. `PAID` → `PREPARING`: Quầy pha chế (Barista) tiếp nhận đơn hàng đã thanh toán và bắt đầu quy trình làm đồ uống.
5. `PAID` → `CANCELLED`: Đơn hàng đã thanh toán bị hủy do sự cố bất khả kháng (ví dụ hết nguyên liệu đột xuất), kích hoạt luồng hoàn tiền.
6. `PREPARING` → `READY`: Nhân viên pha chế hoàn tất đồ uống, đơn hàng sẵn sàng chờ khách hàng đến nhận tại quầy.
7. `READY` → `COMPLETED`: Khách hàng nhận đồ uống thành công. Đây là một trong hai trạng thái kết thúc (*Terminal State*).
8. `PAYMENT_FAILED` → `PENDING`: Khách hàng nhấn thử lại (*Retry Payment*), chọn phương thức thanh toán mới để tiến hành trả tiền lại cho đơn hàng.
9. `PAYMENT_FAILED` → `CANCELLED`: Đơn hàng bị hủy vĩnh viễn sau khi thanh toán thất bại và khách hàng không có nhu cầu thử lại.

Hai trạng thái `COMPLETED` và `CANCELLED` là trạng thái kết thúc vĩnh viễn (*Terminal States*), danh sách chuyển tiếp tiếp theo là mảng rỗng (`[]`). Bất kỳ nỗ lực chuyển đổi sai quy tắc nào đều bị chặn ngay lập tức bởi hàm kiểm tra `assertTransition()` và ném ngoại lệ `409 ConflictException`:

```
export function assertTransition(fromStatus: OrderStatus, toStatus:
    OrderStatus): void {
  const allowedStatuses = ALLOWED_TRANSITIONS[fromStatus];
  if (!allowedStatuses?.includes(toStatus)) {
    throw new ConflictException(
      `Invalid order status transition: ${fromStatus} -> ${toStatus}`
    );
  }
}
```

## 6.2 Xử lý Thanh toán Bất biến (Idempotent Payment)

Trong môi trường mạng không ổn định, khách hàng thường có xu hướng nhấp đúp vào nút "Thanh toán" hoặc trình duyệt tự động gửi lại request khi mạng chập chờn (*Network Timeout*). Nếu không có cơ chế bảo vệ, người dùng sẽ bị trừ tiền nhiều lần cho cùng một đơn hàng (*Double Charge*).

BrewLite giải quyết triệt để vấn đề này bằng cơ chế **Idempotency** cấp độ doanh nghiệp:

1. **Sinh khóa định danh duy nhất (`Idempotency-Key`):** Mỗi khi người dùng bấm xác nhận thanh toán, client tự động sinh một mã UUIDv4 duy nhất và đính kèm vào HTTP Header: `Idempotency-Key: <UUID>`.

2. **Mã hóa toàn vẹn dữ liệu (`request_hash`):** Backend nhận request và tính toán mã băm SHA-256 trên chuỗi JSON chứa thông tin đơn hàng và phương thức thanh toán: `{ orderId, method }`. Mã băm này đại diện cho tính toàn vẹn của giao dịch.

3. **Kiểm tra và Điều phối giao dịch:**

   - **Trường hợp 1 (Khóa đã tồn tại & Trùng khớp nội dung):** Nếu `idempotency_key` đã tồn tại trong cơ sở dữ liệu và `request_hash` trùng khớp, hệ thống gọi hàm `replayExistingPayment()` để phát lại (*replay*) ngay kết quả thanh toán đã lưu trước đó mà không gọi lại Mock Gateway.
   - **Trường hợp 2 (Sai lệch tải trọng - Payload Tampering):** Nếu `idempotency_key` đã được sử dụng nhưng `request_hash` khác nhau (ví dụ cố tình gửi cùng key nhưng đổi phương thức thanh toán hoặc ID đơn hàng), hệ thống ném ngoại lệ `409 ConflictException("Idempotency-Key đã được sử dụng với nội dung khác")`.
   - **Trường hợp 3 (Tranh chấp đồng thời - Concurrency Race Condition):** Khi hai request mang cùng một khóa đến gần như đồng thời, cơ sở dữ liệu chặn lại bằng ràng buộc duy nhất (`@unique`) và trả về mã lỗi `P2002`. Khối `catch` ở tầng Service nhận diện mã lỗi `P2002`, truy vấn lại bản ghi thanh toán vừa được tạo thành công và phát lại (*replay*) kết quả an toàn mà không ném lỗi `409` hay gây trừ tiền lặp.
   - **Trường hợp 4 (Giao dịch mới):** Bản ghi thanh toán được tạo với trạng thái `PENDING`, gọi cổng thanh toán giả lập, sau đó cập nhật kết quả cuối cùng (`SUCCESS` hoặc `FAILED`).

## 6.3 Kiểm soát Đồng thời Tồn kho (Concurrency Control)

Một trong những thách thức lớn nhất của các ứng dụng thương mại điện tử là bài toán xung đột tồn kho (*Race Condition*) khi nhiều khách hàng cùng đặt mua sản phẩm cuối cùng vào cùng một thời điểm.

BrewLite áp dụng kỹ thuật **Khóa lạc quan (Optimistic Locking)** kết hợp cùng **Interactive Transaction (`prisma.$transaction`)**:

- Trên bảng `products`, mỗi dòng sản phẩm có một trường số nguyên `version` với giá trị khởi tạo là `0`.
- Khi giao dịch tạo đơn hàng diễn ra, câu lệnh cập nhật số lượng tồn kho bắt buộc phải thỏa mãn điều kiện phiên bản không bị thay đổi:

```
UPDATE products
SET stock = stock - :quantity,
    version = version + 1
WHERE id = :productId
  AND version = :currentVersion
  AND stock >= :quantity;
```

- Nếu hai giao dịch đồng thời cùng đọc phiên bản $version = 0$, giao dịch thứ nhất thực thi thành công sẽ đẩy phiên bản lên $version = 1$. Giao dịch thứ hai khi cập nhật sẽ nhận số dòng bị ảnh hưởng là 0 (`affected_rows = 0`).

- Khi phát hiện số dòng cập nhật khác 1 (`result.count !== 1`), hệ thống ngay lập tức ném ngoại lệ `400 BadRequestException` (`"Insufficient stock or concurrent update for product <id>."`), đồng thời tự động rollback toàn bộ giao dịch tạo đơn, đảm bảo tồn kho tuyệt đối không bao giờ bị âm.

## 6.4 Nghiệp vụ Khuyến mãi và Điểm tích lũy Thành viên

Hệ thống tích hợp bộ đôi động cơ khuyến mãi và khách hàng thân thiết hoạt động nhịp nhàng nhằm gia tăng giá trị đơn hàng:

### 6.4.1 Cơ chế Mã giảm giá (Promotions Engine)

Hệ thống hỗ trợ cấu hình mã giảm giá linh hoạt được quản lý trong bảng `promotions`, khởi tạo sẵn qua `prisma/seed.ts` phục vụ kiểm thử và trải nghiệm:

1. **Mã `WELCOME10` (Chiết khấu phần trăm có mức trần):**
   - **Mã code:** `WELCOME10`.
   - **Tỷ lệ chiết khấu:** Giảm 10% trên tổng giá trị tạm tính (`totalAmount`).
   - **Điều kiện áp dụng:** Đơn hàng tối thiểu (`minOrderAmount`) đạt từ 40.000 VNĐ trở lên.
   - **Mức giảm tối đa:** Giới hạn mức giảm trần (`maxDiscount`) là 20.000 VNĐ.
   - Công thức tính trong `PromotionsService`:

$$\text{discountAmount} = \min\left(\left\lfloor \frac{\text{totalAmount} \times 10}{100} \right\rfloor, 20.000\right)$$

2. **Mã `GIAM15K` (Chiết khấu số tiền cố định):**
   - **Mã code:** `GIAM15K`.
   - **Mức chiết khấu:** Giảm trực tiếp 15.000 VNĐ (`discountAmount = 15.000`).
   - **Điều kiện áp dụng:** Đơn hàng tối thiểu đạt từ 60.000 VNĐ trở lên.

### 6.4.2 Cơ chế Điểm thưởng Thành viên (Loyalty Points)

- **Tích lũy điểm tự động (Earn Points - Đã hiện thực trong MVP):** Nhằm tri ân và tăng tính gắn kết của khách hàng, hệ thống hiện thực cơ chế tích lũy điểm tự động được kích hoạt ngay khi đơn hàng chuyển sang trạng thái `PAID` thông qua phương thức `LoyaltyService.award()`. Tỷ lệ tích điểm thực tế trong mã nguồn là 1 điểm thưởng cho mỗi 1.000 VNĐ thanh toán thực trả (`finalAmount`):

$$\text{pointsEarned} = \left\lfloor \frac{\text{finalAmount}}{1.000} \right\rfloor$$

  Tiến trình cộng điểm được thực thi nguyên tử trong cùng cơ chế giao dịch (`tx`), tự động tăng biến `loyaltyPoints` của người dùng, cập nhật trường `pointsEarned` của đơn hàng và ghi nhận một bản ghi biến động dạng `EARNED` vào bảng `loyalty_transactions`. Cơ chế kiểm tra tồn tại giao dịch đảm bảo tính Idempotent, loại trừ nguy cơ cộng điểm trùng lặp khi có webhook gọi lại.

- **Quy đổi điểm trừ tiền (Redeem Points - Đặc tả Thiết kế phạm vi mở rộng Post-MVP):** Tại phiên bản MVP 1.0, nhằm tập trung bảo đảm an toàn tuyệt đối cho luồng thanh toán và giao dịch tài chính cốt lõi, tính năng nhập điểm để trừ trực tiếp vào đơn hàng mới được xếp vào danh mục phát triển mở rộng (*Post-MVP Backlog*). Cấu trúc cơ sở dữ liệu đã được thiết kế sẵn sàng để đón đầu tính năng này trong các Sprint tiếp theo:
  - Enum `LoyaltyTransactionType` đã hỗ trợ sẵn các loại giao dịch `SPENT` (tiêu điểm) và `REFUNDED` (hoàn trả điểm khi hủy đơn).
  - Quy tắc quy đổi dự kiến: $1$ điểm $= 1.000$ VNĐ, với trần khống chế số tiền giảm trừ tối đa không vượt quá 50% giá trị đơn hàng sau chiết khấu khuyến mãi để đảm bảo tính an toàn tài chính.
- Mọi biến động cộng điểm đều được ghi vết minh bạch trong bảng `loyalty_transactions`.

# Chương 7

# HIỆN THỰC HÓA VÀ ĐỐI CHIẾU THIẾT KẾ

Chương này trình bày ngắn gọn các minh chứng kỹ thuật nhằm xác nhận rằng toàn bộ các thiết kế Use Case, sơ đồ tương tác và mô hình dữ liệu trong Giai đoạn 1 và Giai đoạn 2 đã được cài đặt đầy đủ vào mã nguồn thực tế của dự án BrewLite.

## 7.1 Ma trận Ánh xạ Nhiệm vụ GD1 & GD2 sang Mã nguồn

Bảng 7.1 tổng hợp chi tiết mối liên kết giữa các nhiệm vụ thiết kế nghiệp vụ của Giai đoạn 1 và 2 với các tập tin và hàm cụ thể trong mã nguồn Backend (`backend/src`) và Frontend (`frontend/src`).

Bảng 7.1: Ma trận Đối chiếu Thiết kế GD1 & GD2 sang Mã nguồn thực tế

| Phân hệ | Nhiệm vụ GD1 & GD2 | Hiện thực Backend | Hiện thực Frontend |
|---|---|---|---|
| **Auth** | Đăng ký tài khoản, Đăng nhập nhận JWT | `auth.controller.ts` (`register`, `login`)<br>`auth.service.ts` | `store/useAuthStore.ts`<br>`services/auth.service.ts`<br>`app/auth/login/page.tsx`<br>`app/auth/register/page.tsx` |
| **Catalog** | Xem danh mục, Lọc món theo loại, Tìm kiếm | `products.controller.ts` (`getProducts`)<br>`products.service.ts` | `hooks/useProductCatalog.ts`<br>`services/product.service.ts`<br>`app/menu/page.tsx` |
| **Detail** | Xem chi tiết đồ uống, Chọn size, đường, đá, topping | `product-detail.controller.ts` (`getProductById`)<br>`product-detail.service.ts` | `hooks/useProductDetail.ts`<br>`services/product.service.ts`<br>`app/products/[id]/page.tsx` |
| **Cart** | Quản lý giỏ hàng cục bộ, Thêm/sửa/xóa, Tạm tính | *(FE-Only State Management)* | `store/useCartStore.ts`<br>`app/cart/page.tsx` |
| **Order** | Tạo đơn hàng, Kiểm tra tồn kho, Tính giá, Áp voucher, Hủy đơn | `orders.module.ts` (`OrdersController.create`)<br>`orders.service.ts` (`create`)<br>`pricing.service.ts`<br>`promotions.service.ts`<br>`cancel-order.controller.ts` | `services/order.service.ts`<br>`store/useCartStore.ts`<br>`app/checkout/page.tsx` |
| **Payment** | Thanh toán an toàn, Xử lý Idempotency, Gọi Gateway | `payments.controller.ts` (`createPayment`)<br>`payments.service.ts`<br>`mock-gateway.service.ts` | `services/payment.service.ts`<br>`app/checkout/payment/page.tsx`<br>`MockGatewayModal.tsx` |
| **History** | Tra cứu danh sách đơn hàng và trạng thái | `history.controller.ts` (`findAll`, `findOne`)<br>`history.service.ts` | `services/order.service.ts`<br>`app/orders/page.tsx`<br>`app/orders/[id]/page.tsx` |

## 7.2 Minh chứng Kỹ thuật Cốt lõi

Dưới đây là một số đoạn trích mã nguồn tiêu biểu chứng minh tính chính xác của các giải pháp kỹ thuật đã thiết kế:

### 7.2.1 Cơ chế Mã hóa Bcrypt và Cấp phát Token trong AuthService

```
// backend/src/modules/auth/auth.service.ts
const saltRounds = 10;
const passwordHash = await bcrypt.hash(dto.password, saltRounds);

const user = await this.prisma.user.create({
  data: {
    email: dto.email,
    passwordHash,
    fullName: dto.fullName,
    loyaltyPoints: 0,
  },
});
const { passwordHash: _, ...result } = user;
return result;
```

### 7.2.2 Interactive Transaction và Khóa Lạc quan trong OrdersService

```
// backend/src/modules/orders/orders.service.ts
// Product: optimistic locking theo version va ton kho trong tx
for (const [productId, quantity] of productQuantities) {
  const product = productById.get(productId);
  if (!product) {
    throw new BadRequestException(`Product ${productId} is unavailable.`);
  }

  const result = await tx.product.updateMany({
    where: {
      id: productId,
      isAvailable: true,
      version: product.version,
      stock: { gte: quantity },
    },
    data: {
      stock: { decrement: quantity },
      version: { increment: 1 },
    },
  });

  if (result.count !== 1) {
    throw new BadRequestException(
      `Insufficient stock or concurrent update for product ${productId}.`,
    );
  }
}
```

### 7.2.3 Kiểm tra Bất biến Idempotency và Xử lý Tranh chấp trong PaymentsService

```
// backend/src/modules/payments/payments.service.ts
// 1. Tinh requestHash tren { orderId, method }
const requestHash = createHash('sha256')
  .update(JSON.stringify({ orderId: body.orderId, method: body.method }))
  .digest('hex');

// 2. Kiem tra key da ton tai: phat lai ket qua da luu (replay)
const existingPayment = await this.prisma.payment.findUnique({
  where: { idempotencyKey: normalizedKey },
  include: { order: true },
});
if (existingPayment) {
  return this.replayExistingPayment(existingPayment, userId, requestHash);
}

// 3. Bat tranh chap dong thoi (Race Condition P2002) de replay an toan
try {
  return await this.prisma.$transaction(async (tx) => {
    // Tao payment voi unique idempotencyKey va requestHash...
  });
} catch (error) {
  if (error instanceof Prisma.PrismaClientKnownRequestError && error.code
      === 'P2002') {
    const paymentAfterRace = await this.prisma.payment.findUnique({
      where: { idempotencyKey: normalizedKey },
      include: { order: true },
    });
    if (paymentAfterRace) {
      return this.replayExistingPayment(paymentAfterRace, userId,
          requestHash);
    }
  }
  throw error;
}
```

# Chương 8

# KIỂM THỬ HỆ THỐNG

Kiểm thử phần mềm là hoạt động sống còn nhằm xác minh tính đúng đắn của logic nghiệp vụ, tính toàn vẹn dữ liệu trong môi trường đa nhiệm và sự tuân thủ nghiêm ngặt theo các hợp đồng API đã cam kết. Trong dự án BrewLite, nhóm đã thiết lập một chiến lược kiểm thử đa tầng bao gồm kiểm thử đơn vị (Unit Test), kiểm thử tích hợp (Integration Test), kiểm thử đầu-cuối (End-to-End Test), kiểm thử tương tranh (Concurrency / Race Conditions Test) và kiểm thử API trên nền tảng container hóa thực tế.

## 8.1 Chiến lược và Môi trường Kiểm thử

### 8.1.1 Chiến lược kiểm thử đa tầng (Multi-tier Testing Strategy)

Hệ thống BrewLite áp dụng mô hình kim tự tháp kiểm thử (*Test Pyramid*) với sự phân định rõ ràng giữa các mức độ kiểm chứng:

- **Kiểm thử Đơn vị (Unit Testing - Service Layer):** Kiểm thử độc lập logic nghiệp vụ cô lập tại tầng Service sử dụng Jest và cơ chế Mocking đối với Prisma Client. Tổng cộng 6 test suites bao quát 84 ca kiểm thử đơn vị tập trung vào các nghiệp vụ trọng yếu của hệ thống: kiểm thử Máy trạng thái đơn hàng (`order-state.spec.ts`), tính toán giá và khuyến mãi (`order.service.spec.ts`), quy trình tạo đơn và trừ kho (`orders.service.spec.ts`), xử lý thanh toán và idempotency (`payments.service.spec.ts`), tra cứu lịch sử đơn (`history.service.spec.ts`) và kiểm tra điểm cuối sức khỏe hệ thống (`health.spec.ts`).

- **Kiểm thử Tích hợp và E2E (End-to-End Integration Testing):** Sử dụng Jest kết hợp thư viện Supertest (`backend/test/orders-concurrency.e2e-spec.ts`) để gửi các HTTP requests trực tiếp đến cổng API của ứng dụng NestJS đang kết nối với cơ sở dữ liệu PostgreSQL thực tế. Mức kiểm thử này xác nhận toàn bộ quy trình từ Controller, ValidationPipe, Service, Transaction cho đến ràng buộc ACID của PostgreSQL.

- **Kiểm thử Tương tranh và Tranh chấp dữ liệu (Concurrency / Stress Testing):** Kịch bản kiểm thử tương tranh thực tế (`backend/test/concurrency.ts`) sử dụng `Promise.all` để bắn đồng thời hàng loạt request (10 requests đặt hàng tranh chấp sản phẩm có tồn kho bằng 1; 5 requests thanh toán đồng thời mang cùng một `Idempotency-Key`). Mục tiêu là kiểm chứng tính vững chắc của cơ chế Khóa lạc quan (*Optimistic Locking*) và cơ chế chống trùng lặp thanh toán (*Idempotency*) dưới áp lực đa luồng.

- **Kiểm thử API Endpoint:** Thực thi và đối soát toàn diện các yêu cầu API của hệ thống trên môi trường cục bộ (`http://127.0.0.1:4000/api/v1`), đối chiếu mã trạng thái HTTP, định dạng phản hồi chuẩn `ApiResponse<T>` và cấu trúc dữ liệu trả về theo đặc tả thiết kế tại thư mục `docs/`.

- **Kiểm thử Giao diện Người dùng (Manual UI Testing):** Do mã nguồn dự án chưa tích hợp runner tự động hóa cho trình duyệt (như Playwright hoặc Cypress), các ca kiểm thử giao diện người dùng được phân loại vào hình thức kiểm thử thủ công (*Manual Exploratory Testing*) theo kịch bản thiết kế để nhóm tự kiểm thử trực tiếp trên trình duyệt.

### 8.1.2 Cấu hình Môi trường Kiểm thử Thực tế

Hệ thống kiểm thử được vận hành trên hạ tầng chuẩn hóa của dự án:

- **Cơ sở dữ liệu:** PostgreSQL được khởi chạy độc lập thông qua Docker Compose (`docker-compose.yml`), mở cổng kết nối tiêu chuẩn 5432.
- **Dịch vụ Backend:** Ứng dụng NestJS giao tiếp cơ sở dữ liệu qua Prisma ORM, phục vụ các API endpoints tại cổng 4000 (tiền tố `/api/v1`).
- **Ứng dụng Frontend:** Ứng dụng Next.js phục vụ giao diện người dùng tại cổng 3000.
- **Bộ công cụ kiểm thử:** Sử dụng Jest và Supertest cho các bài kiểm thử đơn vị, kiểm thử tích hợp và kiểm thử tương tranh tự động.

### 8.1.3 Dữ liệu Khởi tạo Kiểm thử (Seed Data)

Dữ liệu thử nghiệm được nạp tự động vào PostgreSQL thông qua kịch bản `backend/prisma/seed.ts`, bao gồm:

- **Danh mục (Categories):** 2 danh mục chính gồm ”Cà phê” (slug `ca-phe`, displayOrder: 1) và ”Trà & Trái cây” (slug `tra-trai-cay`, displayOrder: 2).
- **Sản phẩm (Products):** 10 sản phẩm đồ uống với khoảng giá gốc từ 30.000 VNĐ đến 45.000 VNĐ:
  - *Nhóm Cà phê (6 món):* Cà phê sữa đá (35.000 VNĐ, tồn kho 100), Americano (40.000 VNĐ, tồn kho 80), Cappuccino (45.000 VNĐ, tồn kho 60), Latte (45.000 VNĐ, tồn kho 50), Espresso (30.000 VNĐ, tồn kho 40), Bạc xỉu (35.000 VNĐ, tồn kho 5 – cấu hình tồn kho thấp phục vụ kiểm thử tương tranh).
  - *Nhóm Trà & Trái cây (4 món):* Trà đào cam sả (39.000 VNĐ, tồn kho 75), Trà chanh (30.000 VNĐ, tồn kho 70), Trà vải (39.000 VNĐ, tồn kho 45), Trà dâu (40.000 VNĐ, tồn kho 35).
- **Quy chuẩn Phụ thu Kích cỡ (Size Prices):** 3 kích cỡ áp dụng cho toàn bộ sản phẩm theo bảng cấu hình `product_size_prices`:
  - Size S: phụ thu 0 VNĐ.
  - Size M: phụ thu +5.000 VNĐ.
  - Size L: phụ thu +10.000 VNĐ.
- **Topping đi kèm:** 3 loại topping theo bảng `toppings`:
  - Trân châu trắng: đơn giá 5.000 VNĐ, tồn kho ban đầu 200 đơn vị.
  - Kem Cheese: đơn giá 10.000 VNĐ, tồn kho ban đầu 150 đơn vị.
  - Đào miếng: đơn giá 8.000 VNĐ, tồn kho ban đầu 100 đơn vị.
- **Mã Khuyến mãi (Vouchers):** 2 mã ưu đãi tiêu chuẩn nạp sẵn trong bảng `promotions`:
  - `WELCOME10`: Giảm 10% tối đa 20.000 VNĐ cho đơn hàng có giá trị tối thiểu từ 40.000 VNĐ.

  - `GIAM15K`: Giảm trực tiếp 15.000 VNĐ cho đơn hàng có giá trị tối thiểu từ 60.000 VNĐ.

- **Tài khoản Người dùng Mẫu (Seed User):** 1 tài khoản khách hàng mặc định với email `demo@brewlite.test`, mật khẩu được băm Bcrypt `Demo@12345` (10 rounds) và số điểm tích lũy ban đầu là 120 điểm loyalty.

### 8.1.4 Các lệnh thực thi kiểm thử và Tóm tắt kết quả

Hệ thống kiểm thử tự động của BrewLite được kích hoạt thông qua các tập lệnh chuẩn trong môi trường phát triển:

1. **Bộ kiểm thử Đơn vị (Unit Tests):**
   - Lệnh thực thi: `npm test` (chạy Jest tại thư mục `backend/`).
   - Kết quả: 6/6 Test Suites Đạt; 84/84 Tests Đạt (100%).
   - Bao phủ: Kiểm thử độc lập Máy trạng thái đơn hàng (Order State Machine), logic tính giá (Pricing Engine), kiểm tra tính hợp lệ của mã giảm giá và các dịch vụ Order, Payment, History, Healthcheck.
2. **Bộ kiểm thử Tích hợp Đầu-cuối (E2E Integration Tests):**
   - Lệnh thực thi: `npm run test:e2e` (chạy Jest E2E tại thư mục `backend/`).
   - Kết quả: 1/1 Test Suite Đạt; 5/5 E2E Tests Đạt (100%).
   - Bao phủ: Kiểm thử tương tranh đặt hàng đồng thời trên sản phẩm và topping, kiểm thử Idempotency thanh toán đồng thời và chặn chuyển trạng thái sai trực tiếp qua HTTP API.
3. **Bộ kiểm thử Tương tranh Độc lập (Concurrency Stress Script):**
   - Lệnh thực thi: `npm run test:concurrency` (`ts-node test/concurrency.ts`).
   - Kết quả: 2/2 Kịch bản kiểm thử Đạt (100%).
   - Bao phủ: Bắn 10 request đặt hàng song song khi tồn kho bằng 1 (chỉ 1 đơn thành công, tồn kho về 0) và bắn 5 request thanh toán đồng thời cùng `Idempotency-Key` (đúng 1 payment trong DB, trả về cùng kết quả).

## 8.2 Bảng Tổng hợp Kết quả Kiểm thử theo Phân hệ

Bảng 8.1 tổng hợp số lượng và trạng thái chi tiết của toàn bộ các ca kiểm thử trong hệ thống BrewLite, phân định rõ ràng giữa các ca kiểm thử tự động tầng Backend đã xác minh và các ca kiểm thử giao diện người dùng.

Bảng 8.1: Tổng hợp số lượng và trạng thái Test Cases theo từng phân hệ

| **Phân hệ** | **Tổng** | **Đạt** | **Chờ XN** | **Chưa chạy** | **Ghi chú kiểm thử** |
|---|---|---|---|---|---|
| Xác thực (Auth) | 14 | 8 | 0 | 6 | 8 ca API/Backend đạt 100%; 6 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Danh mục (Catalog) | 12 | 5 | 0 | 7 | 5 ca API đạt 100%; 7 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Chi tiết món (Detail) | 8 | 3 | 0 | 5 | 3 ca API đạt 100%; 5 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Giỏ hàng (Cart) | 8 | 0 | 0 | 8 | Thuần Client Zustand + LocalStorage, 8 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Đặt hàng (Order) | 52 | 48 | 0 | 4 | 48 ca API/Unit/State Machine đạt 100%; 4 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Thanh toán (Payment) | 22 | 19 | 0 | 3 | 19 ca API/Unit đạt 100%; 3 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Lịch sử đơn (History) | 15 | 7 | 0 | 8 | 7 ca API đạt 100%; 8 ca UI chưa chạy (sẽ kiểm thử thủ công) |
| Tương tranh & Task 10 | 3 | 3 | 0 | 0 | 3 ca Concurrency / E2E đạt 100% |
| **TỔNG CỘNG** | **134** | **93** | **0** | **41** | **100% ca Backend/Unit/E2E/Concurrency ĐẠT** |

*Đánh giá tổng quan:* Toàn bộ 93 ca kiểm thử thuộc tầng Backend, API Endpoint và kịch bản Tương tranh đều được kiểm chứng thông qua các bộ kiểm thử tự động (`npm test`, `npm run test:e2e`, `npm run test:concurrency`), đạt kết quả **Đạt (Pass) tỷ lệ 100%**. 41 ca kiểm thử giao diện mang trạng thái **Chưa chạy (sẽ kiểm thử thủ công)** do repo chưa tích hợp công cụ tự động hóa UI (như Playwright/Cypress); các kịch bản này sẽ do nhóm tự kiểm thử trực tiếp trên trình duyệt.

## 8.3 Ba Kịch bản Kiểm thử Bắt buộc của Task 10

Dưới đây là phân tích chi tiết về mặt kỹ thuật, cơ chế thực thi và kết quả của 3 kịch bản kiểm thử bắt buộc theo yêu cầu kỹ thuật của dự án (Task 10).

### 8.3.1 Kịch bản TS-FR-07-02: Chặn chuyển trạng thái sai của đơn hàng

**Mục tiêu:** Chứng minh Máy trạng thái đơn hàng (`OrderStateMachine`) ngăn chặn triệt để mọi hành vi chuyển trạng thái không hợp lệ, đặc biệt là việc hủy các đơn hàng đã thanh toán thành công (`PAID`) hoặc đã hoàn tất giao hàng (`COMPLETED`).

**Cơ chế kỹ thuật:** Hàm `assertTransition(fromStatus, toStatus)` trong `OrderStateMachine` lưu trữ ma trận hợp lệ gồm đúng 9 bước chuyển đổi. Bất kỳ nỗ lực chuyển tiếp nào nằm ngoài danh mục này (40 cặp trạng thái không hợp lệ) đều lập tức kích hoạt `ConflictException` với mã lỗi HTTP 409 Conflict. Đồng thời, API `POST /orders/:id/cancel` kiểm tra nghiêm ngặt trạng thái hiện tại: chỉ cho phép hủy đơn nếu đơn đang ở trạng thái `PENDING` hoặc `PAYMENT_FAILED`.

**Bằng chứng thực thi thực tế (Trích từ nhật ký E2E test):**

```
PASS test/orders-concurrency.e2e-spec.ts
```

```
  Orders & Payments Concurrency (e2e)√
     (a) Chuyen trang thai sai qua API: huy don da PAID tra 409; huy 2 lan
        tra 409
```

**Kết luận:** Kịch bản kiểm thử đạt tiêu chuẩn chất lượng. Đơn hàng đã ở trạng thái kết thúc (`CANCELLED`, `COMPLETED`) hoặc đang pha chế/đã trả tiền không thể bị hủy lặp lại, giữ vững tính toàn vẹn trạng thái.

### 8.3.2 Kịch bản TS-FR-08-01: Cùng Idempotency-Key chỉ tạo đúng một payment trong CSDL

**Mục tiêu:** Đảm bảo rằng khi client gửi nhiều yêu cầu thanh toán trùng lặp mang cùng một khóa chống trùng lặp (`Idempotency-Key`) do sự cố mạng chập chờn hoặc do người dùng nhấp đúp chuột liên tục, hệ thống chỉ tạo đúng duy nhất 1 bản ghi giao dịch thanh toán trong cơ sở dữ liệu và tuyệt đối không phát sinh mã lỗi 500.

**Cơ chế kỹ thuật:** Bảng `payments` thiết lập chỉ mục duy nhất (`UNIQUE`) trên cột `idempotency_key` và lưu trữ giá trị băm `request_hash` (SHA-256 của chuỗi `{orderId, method}`). Khi các request đồng thời cùng đến, request đầu tiên ghi nhận vào CSDL. Các request đến sau đụng phải lỗi vi phạm ràng buộc duy nhất (`PrismaClientKnownRequestError P2002`); khối `try/catch` của `PaymentsService` bắt lỗi này và gọi hàm `replayExistingPayment` để trả lại kết quả giao dịch ban đầu một cách an toàn. Trường hợp client gửi cùng key nhưng thay đổi thông tin thanh toán, hệ thống sẽ ném ngoại lệ `ConflictException` (HTTP 409 Conflict).

**Bằng chứng thực thi thực tế (Trích từ nhật ký Concurrency test):**

```
--- [TEST 2] KIEM TRA THANH TOAN DONG THOI CUNG IDEMPOTENCY-KEY ---
Da tao don hang test payment: ID = cmvp3t1w60003qo2re0orm3c3
Ban dong thoi 5 requests POST /payments cung key "concurrency-pay
    -1793009911361"...
- So request thanh cong (200/201): 5
- So request bi loi 500: 0
- So payment ID phan biet tra ve: 1
- So ban ghi Payment trong DB voi key nay: 1
PASS [TEST 2]: Idempotency xu ly tranh chap dong thoi an toan, dung 1 ban
    ghi thanh toan.
```

**Kết luận:** Hệ thống hoàn toàn miễn nhiễm với lỗi trừ tiền trùng lặp và không gặp hiện tượng treo máy hoặc mã lỗi 500 khi chịu tải tranh chấp thanh toán đồng thời.

### 8.3.3 Kịch bản TS-FR-09-01: Đặt đồng thời không làm tồn kho âm (Optimistic Locking)

**Mục tiêu:** Chứng minh cơ chế Khóa lạc quan (*Optimistic Locking*) sử dụng trường phiên bản (`version`) ngăn chặn triệt để hiện tượng bán vượt tồn kho (*overselling*) khi nhiều khách hàng cùng lúc tranh mua những sản phẩm cuối cùng.

**Cơ chế kỹ thuật:** Bảng `products` và `toppings` sở hữu trường `version Int @default(0)`. Khi thực hiện cập nhật trừ kho trong `OrdersService`, hệ thống thực thi truy vấn cập nhật có điều kiện:

```
UPDATE products
SET stock = stock - :quantity, version = version + 1
WHERE id = :id AND version = :currentVersion AND stock >= :quantity;
```

Nếu câu lệnh cập nhật trả về số dòng ảnh hưởng bằng 0 (do phiên bản đã bị thay đổi bởi giao dịch khác vừa commit trước đó hoặc số lượng tồn kho khả dụng không đủ), hệ thống chủ động ném ngoại lệ `BadRequestException` với thông điệp `"Insufficient stock or concurrent update for product <id>"` và rollback toàn bộ transaction tạo đơn.

**Bằng chứng thực thi thực tế (Trích từ nhật ký Concurrency test):**

```
--- [TEST 1] KIEM TRA DAT HANG DONG THOI (OPTIMISTIC LOCKING) ---
San pham test: "Bac xiu" (ID: cmvkxjx6p001bs81ykciddb4s)
Ton kho ban dau: 5, Version: 1
Da reset ton kho = 1, Version = 1
Ban dong thoi 10 requests POST /orders (moi don mua 1 ly size M)...
- Don tao thanh cong (201 Created): 1
- Don bi chan het hang/xung dot (400 Bad Request): 9
Ton kho cuoi cung trong DB: 0 (Version: 2)
PASS [TEST 1]: Optimistic locking chan tranh chap thanh cong, khong vuot
    ton kho.
```

**Kết luận:** Dưới áp lực 10 request đặt mua đồng thời vào món hàng chỉ còn duy nhất 1 sản phẩm trong kho, chỉ đúng 1 request mua thành công (HTTP 201), 9 request còn lại bị từ chối an toàn (HTTP 400). Tồn kho cuối cùng giảm về đúng bằng 0, không bao giờ bị âm.

## 8.4 Kiểm thử Giao diện Người dùng (User Interface Testing)

Trong phạm vi mã nguồn của dự án, hệ thống chưa tích hợp công cụ kiểm thử giao diện tự động hóa (như Playwright hoặc Cypress). Do đó, toàn bộ các ca kiểm thử giao diện người dùng được phân loại vào hình thức kiểm thử thủ công (Manual Testing) dựa trên các kịch bản thiết kế tại thư mục `docs/`.

Các màn hình chính và kịch bản tương tác người dùng bao gồm:

- **Xác thực tài khoản (Hình 5.4a, Hình 5.4b):** Kiểm thử form đăng ký tài khoản với các ràng buộc mật khẩu tối thiểu 6 ký tự, định dạng email chuẩn và thông báo lỗi trùng lặp; đăng nhập thành công lưu JWT token và hiển thị thông tin thành viên trên thanh điều hướng. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.
- **Thực đơn và Bộ lọc (Hình 5.8):** Kiểm thử việc tải danh sách 10 món đồ uống từ cơ sở dữ liệu, chuyển đổi giữa các tab danh mục (”Tất cả”, ”Cà phê”, ”Trà & Trái cây”), hiển thị chính xác tên món, hình ảnh đại diện và đơn giá gốc. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.
- **Chi tiết món và Tùy biến (Hình 5.12):** Kiểm thử chuyển đổi kích cỡ (S/M/L) với đơn giá cập nhật tự động theo phụ thu (+5.000 VNĐ cho Size M, +10.000 VNĐ cho Size L); chọn thêm topping với ràng buộc tối đa 3 loại; điều chỉnh số lượng món và thêm vào giỏ hàng. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.
- **Quản lý Giỏ hàng (Hình 5.16):** Kiểm thử hiển thị danh sách món đã chọn lưu trong Zustand Store; tăng giảm số lượng hoặc xóa món; tự động cập nhật tổng giá trị tạm tính. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.
- **Xác nhận Đặt hàng và Mã giảm giá (Hình 5.20):** Kiểm thử áp dụng mã khuyến mãi `WELCOME10`; tính toán mức giảm 10% và hiển thị số tiền thanh toán cuối cùng (`finalAmount`). Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.

- **Cổng Thanh toán Trực tuyến (Hình 5.24):** Kiểm thử lựa chọn giữa Thẻ ngân hàng (`BANK_CARD`) và Ví điện tử (`E_WALLET`); hiển thị tổng tiền và xử lý chuyển trạng thái đơn hàng. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.
- **Tra cứu Lịch sử Đơn hàng (Hình 5.28):** Kiểm thử danh sách đơn hàng đã đặt phân theo các tab trạng thái; xem chi tiết từng món và thực hiện hủy đơn hàng khi còn ở trạng thái `PENDING`. Kết quả thực tế: **[TODO: điền kết quả kiểm thử thủ công]**.

## 8.5 Lỗi Phát hiện và Giới hạn Kỹ thuật Đã biết

Thông qua việc chạy kiểm thử thực tế và đối chiếu nghiêm ngặt với mã nguồn, nhóm ghi nhận trung thực các điểm sai lệch và giới hạn kỹ thuật như sau:

1. **Tính năng Đổi điểm thưởng (Redeem Loyalty Points) thuộc Post-MVP:** Trong mã nguồn thực tế, DTO tạo đơn hàng (`CreateOrderDto`) chỉ chứa danh sách món (`items`) và mã ưu đãi (`promotionCode`), không có trường nhập điểm thưởng. Backend đã hoàn thiện logic tích lũy điểm khi thanh toán thành công ($1.000$ VNĐ $= 1$ điểm), nhưng tính năng cho phép trừ điểm vào đơn hàng được phân loại là tính năng mở rộng ngoài MVP (Post-MVP).

2. **Quy cách tham số Mã giảm giá `WELCOME10` và `GIAM15K`:** Trong tập tin khởi tạo dữ liệu `backend/prisma/seed.ts`, mã `WELCOME10` được cấu hình giảm 10%, mức giảm trần tối đa là 20.000 VNĐ và giá trị đơn hàng tối thiểu là 40.000 VNĐ. Mã `GIAM15K` giảm thẳng 15.000 VNĐ cho đơn hàng từ 60.000 VNĐ.

3. **Mã lỗi phản hồi khi Xung đột Tồn kho / Phiên bản Khóa lạc quan:** Backend hiện tại trả về mã lỗi `400 BadRequestException` với thông điệp `"Insufficient stock or concurrent update for product <id>"` chung cho cả trường hợp hết hàng lẫn xung đột phiên bản `version` (thay vì phân tách mã `409 Conflict`). Phía Frontend hiện chưa có cơ chế Auto-Retry ngầm mà yêu cầu người dùng thao tác lại khi gặp thông báo lỗi.

4. **Thống nhất mã trạng thái HTTP 201 Created cho Replay Idempotency Thanh toán:** Trước đây tài liệu thiết kế ghi nhận mã HTTP 200 cho trường hợp replay Idempotency. Nhóm đã chuẩn hóa tài liệu thiết kế (`payment-test-cases.md` v1.2, `api-contract-payment.md`) và bộ kịch bản (`test-scenarios.md`) thống nhất trả về mã `201 Created` cho cả lần gửi đầu tiên lẫn các lần gửi lại trùng lặp, hoàn toàn khớp với hành vi mặc định của phương thức POST trong NestJS và cơ chế bảo vệ giao dịch không bị trừ tiền trùng lặp.

5. **Cơ chế Phát hiện Tranh chấp Idempotency khác nội dung:** Backend lưu trữ giá trị băm SHA-256 từ chuỗi `{orderId, method}`. Khi gửi cùng key nhưng thay đổi phương thức hoặc mã đơn hàng, hệ thống ném ngoại lệ `ConflictException` (HTTP 409 Conflict) đúng theo quy tắc nghiệp vụ.

6. **Hoàn trả điểm thưởng khi hủy đơn:** Mặc dù cơ sở dữ liệu đã thiết kế sẵn kiểu giao dịch `LoyaltyTransactionType.REFUNDED`, nghiệp vụ tự động hoàn lại điểm thưởng khi đơn hàng bị hủy là tính năng dự kiến cho Post-MVP.

7. **Chuẩn hóa Phụ thu Kích cỡ Size M:** Trong dữ liệu khởi tạo `seed.ts`, phụ thu Size M là +5.000 VNĐ và Size L là +10.000 VNĐ. Các tài liệu mô tả trước đây ghi nhận +6.000 VNĐ cho Size M đã được đính chính đồng bộ theo mã nguồn thực tế.

## 8.6 Ma trận Truy vết Test Case về Yêu cầu và Use Case

Bảng 8.2 thể hiện tính liên kết và độ phủ kiểm thử của toàn bộ các ca kiểm thử đối với từng Yêu cầu Chức năng (FR) và Ca sử dụng (Use Case) của hệ thống.

Bảng 8.2: Ma trận truy vết ca kiểm thử về Yêu cầu Chức năng và Use Case

| Mã FR | Mã UC | Mô tả nghiệp vụ | Nhóm Test Case | Số ca Đạt | Chưa chạy |
|---|---|---|---|---|---|
| FR-01 | UC-AUTH-01..02 | Đăng ký & Đăng nhập | TC-AUTH-01..08<br>TC-AUTH-UI-01..06 | 8 | 6 (UI) |
| FR-02 | UC-CAT-01, UC-DET-01 | Xem thực đơn & Chi tiết | TC-CAT-01..05<br>TC-DET-01..03<br>TC-CAT-UI-01..07<br>TC-DET-UI-01..05 | 8 | 12 (UI) |
| FR-03 | UC-CART-01 | Quản lý Giỏ hàng | TC-CART-UI-01..08 | 0 | 8 (UI) |
| FR-04 | UC-ORD-01 | Tạo đơn, Máy trạng thái & Hủy đơn | TC-ST-01..24<br>TC-CO-01..19<br>TC-CX-01..05<br>TC-ORD-UI-01..04 | 48 | 4 (UI) |
| FR-05 | UC-PAY-01 | Thanh toán Idempotent | TC-PAY-01..19<br>TC-PAY-UI-01..03 | 19 | 3 (UI) |
| FR-06 | UC-HIS-01 | Lịch sử đơn hàng | TC-HIS-01..07<br>TC-HIS-UI-01..08 | 7 | 8 (UI) |
| FR-07..09 | Concurrency | Task 10 Scenarios | TS-FR-07-02<br>TS-FR-08-01, 09-01 | 3 | 0 |
| **TỔNG CỘNG** | | | | **93** | **41** |

# Chương 9

# ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

## 9.1 Đánh giá Kết quả Đạt được

Trải qua hai giai đoạn phân tích, thiết kế, hiện thực hóa và kiểm thử nghiêm ngặt (Giai đoạn 1 và Giai đoạn 2), nhóm đã hoàn thành toàn diện các mục tiêu đề ra cho bài tập lớn môn Công nghệ Phần mềm: đạt **21/21 task cam kết (100%)**, vượt qua 100% các ca kiểm thử tự động hóa (84 unit tests, 5 E2E tests, 44 ca kiểm thử API và các bài test tương tranh Concurrency / Idempotency):

- **Chuẩn hóa Thiết kế Hệ thống:** Xây dựng hoàn chỉnh bộ tài liệu kỹ thuật từ sơ đồ kiến trúc thành phần, sơ đồ triển khai Docker, sơ đồ lớp Backend, sơ đồ thực thể liên kết (ERD) 12 bảng cho đến chi tiết Use Case, sơ đồ Hoạt động (Activity) và sơ đồ Tuần tự (Sequence) cho toàn bộ 7 phân hệ chức năng.
- **Hiện thực hóa Đầy đủ 7 Phân hệ Nghiệp vụ:** Cài đặt đồng bộ từ giao diện người dùng Next.js 15 đến hệ thống API NestJS 11 và cơ sở dữ liệu PostgreSQL 16. Mọi tương tác giao diện và gọi dịch vụ đều bám sát thiết kế.
- **Làm chủ các Bài toán Kỹ thuật Nâng cao:** Giải quyết trọn vẹn bài toán máy trạng thái đơn hàng (9 chuyển tiếp), bài toán kiểm soát giao dịch thanh toán bất biến (*Idempotency*) và bài toán kiểm soát xung đột tồn kho bằng khóa lạc quan (*Optimistic Locking*).

## 9.2 Các Hạn chế Hiện tại

Bên cạnh những kết quả đã đạt được, hệ thống BrewLite vẫn tồn tại một số điểm giới hạn kỹ thuật cần được ghi nhận trung thực:

1. **Gộp chung mã lỗi xung đột tồn kho và phiên bản:** Tại tầng Backend (`OrdersService`), cả hai trường hợp hết tồn kho thực tế (`stock < quantity`) và xung đột phiên bản Khóa lạc quan (`version` không khớp) hiện đang được xử lý chung thông qua ngoại lệ `400 BadRequestException`. Việc chưa phân tách riêng mã `409 Conflict` cho trường hợp xung đột phiên bản khiến phía Client chưa thể kích hoạt cơ chế tự động thử lại (Auto-Retry) ngầm mà người dùng phải thao tác lại thủ công.
2. **Phân hệ Loyalty chỉ mới kích hoạt chiều tích điểm (Earn Points):** Phiên bản MVP hiện tại đã hoàn thiện trọn vẹn luồng tự động tích lũy điểm thưởng khi thanh toán thành công ($1.000$ VNĐ $= 1$ điểm). Tính năng dùng điểm để giảm trừ trực tiếp vào đơn hàng mới (*Redeem Points*) và xử lý hoàn điểm tự động khi hủy đơn được lên kế hoạch bàn giao ở bản phát hành tiếp theo.
3. **Giao diện Quầy Pha chế (Barista UI) ngoài phạm vi MVP:** Dù Backend đã hoàn thiện trọn vẹn logic chuyển tiếp trạng thái (`PREPARING`, `READY`, `COMPLETED`), nhưng giao diện web/tablet dành riêng cho nhân viên pha chế tại quầy chưa được hiện thực hóa trong phạm vi bản phát hành MVP này.

4. **Tích hợp Cổng thanh toán Thực tế:** Hệ thống hiện đang sử dụng dịch vụ Mock Payment Gateway để giả lập luồng thanh toán. Việc tích hợp các cổng thanh toán thương mại thực tế tại Việt Nam (như VNPay, MoMo, ZaloPay) đòi hỏi quy trình đăng ký doanh nghiệp và thẩm định pháp lý phức tạp nên được định hướng ở giai đoạn sau.

## 9.3 Hướng Phát triển trong Tương lai

Nhằm đưa BrewLite trở thành một nền tảng vận hành hoàn chỉnh cho các chuỗi cửa hàng cà phê thương mại, các định hướng phát triển tiếp theo bao gồm:

- **Hiện thực hóa tính năng Đổi điểm thưởng (Redeem Loyalty Points):** Cung cấp giao diện cho phép khách hàng chủ động chọn số điểm muốn sử dụng tại bước thanh toán để khấu trừ trực tiếp vào giá trị đơn hàng, tích hợp cơ chế hoàn trả điểm tự động khi đơn hàng bị hủy theo chuẩn cấu trúc cơ sở dữ liệu đã thiết kế sẵn (đây là tính năng dự kiến cho giai đoạn Post-MVP, hiện tại mã nguồn chưa kích hoạt luồng đổi điểm và hoàn điểm).
- **Xây dựng Màn hình Pha chế (Barista POS Display):** Phát triển giao diện chuyên dụng cho quầy bar, cập nhật danh sách đơn hàng theo thời gian thực sử dụng công nghệ Web-Sockets hoặc Server-Sent Events (SSE).
- **Tích hợp Cổng thanh toán VNPay / MoMo:** Kết nối trực tiếp qua Webhook và chuẩn thanh toán mã VietQR, hỗ trợ hoàn tiền tự động khi đơn hàng bị hủy sau khi đã thanh toán.
- **Phân tách mã lỗi 409 Conflict và Cơ chế Client-side Auto-Retry:** Nâng cấp `OrdersService` để phân tách rõ ràng giữa mã lỗi `400 Bad Request` (khi hết hàng thực tế) và mã lỗi `409 Conflict` (khi tồn kho còn nhưng xảy ra xung đột version giữa các request đồng thời), kết hợp xây dựng lớp client tự động thử lại (Retry với Exponential Backoff) giúp nâng cao trải nghiệm người dùng trong các đợt cao điểm.
- **Phân hệ Báo cáo và Quản trị Doanh thu (Admin Analytics):** Bổ sung biểu đồ trực quan hóa dữ liệu bán hàng, dự báo nhu cầu nguyên vật liệu và quản lý các chiến dịch khuyến mãi theo mùa.

# Chương A

# MA TRẬN TRUY VẾT YÊU CẦU (TRACEABILITY MATRIX)

Ma trận truy vết yêu cầu cung cấp khả năng kiểm soát chất lượng từ yêu cầu chức năng (FR), đặc tả Use Case (UC), API Endpoint, bảng cơ sở dữ liệu cho đến các ca kiểm thử (TC) tương ứng.

Bảng A.1: Ma trận Truy vết Toàn diện Yêu cầu - Thiết kế - Cài đặt - Kiểm thử

| Mã FR | Mã UC | API Endpoint | Bảng CSDL | Mã Test Case | Màn hình |
|---|---|---|---|---|---|
| **FR-01** | UC-AUTH-01 | `POST /auth/register` | `users` | `TC-AUTH-01..03` | `/auth/register` |
| | UC-AUTH-02 | `POST /auth/login` | `users` | `TC-AUTH-04..08` | `/auth/login` |
| **FR-02** | UC-CAT-01 | `GET /products` | `products, categories` | `TC-CAT-01..05` | `/menu` |
| | UC-DET-01 | `GET /products/:id` | `products, sizes, toppings` | `TC-DET-01..03` | `/products/:id` |
| **FR-03** | UC-CART-01 | *(Client-side)* | *(LocalStorage)* | `TC-CART-UI-01..08` | `/cart` |
| **FR-04** | UC-ORD-01 | `POST /orders` | `orders, items, toppings, history` | `TC-CO-01..19` | `/checkout` |
| | | `GET /orders/:id` | `orders, items, payments` | `TC-HIS-05` | `/orders/:id` |
| | | `POST /orders/:id/cancel` | `orders, history, products` | `TC-CX-01..05` | `/orders/:id` |
| | | *(State Machine)* | `orders, order_status_history` | `TC-ST-01..24` | *(Backend)* |
| **FR-05** | UC-PAY-01 | `POST /payments` | `payments, orders, loyalty, history` | `TC-PAY-01..19` | `/checkout/payment` |
| **FR-06** | UC-HIS-01 | `GET /orders/me` | `orders` | `TC-HIS-01..04` | `/orders` |
| | | `GET /orders/:id` | `orders, items, status_history` | `TC-HIS-05..07` | `/orders/:id` |

Tiếp tục ở trang sau...

**Bảng A.1 – Tiếp tục từ trang trước**

| Mã FR | Mã UC | API Endpoint | Bảng CSDL | Mã Test Case | Màn hình |
|---|---|---|---|---|---|
| **FR-07..09** | Concurrency | `POST /orders, /payments` | `orders, products, payments` | `TS-FR-07-02, 08-01, 09-01` | *(Automated E2E)* |

# Chương B

# DANH MỤC API ENDPOINTS

Bảng B.1 liệt kê toàn bộ các API Endpoints được cài đặt và cung cấp bởi hệ thống Backend NestJS của BrewLite. Tất cả các endpoint đều có tiền tố chuẩn `/api/v1`.

Bảng B.1: Danh mục chi tiết toàn bộ các API Endpoints hệ thống BrewLite

| Phương thức | Đường dẫn (Path) | Xác thực | Mô tả chức năng |
|---|---|---|---|
| `POST` | `/api/v1/auth/register` | Công khai | Đăng ký tài khoản người dùng mới |
| `POST` | `/api/v1/auth/login` | Công khai | Đăng nhập và nhận mã Bearer JWT |
| `GET` | `/api/v1/products` | Công khai | Lấy danh sách sản phẩm kèm phân loại danh mục (`categoryName`), hỗ trợ tìm kiếm (`search`) và lọc danh mục (`categoryId`) |
| `GET` | `/api/v1/products/:id` | Công khai | Xem chi tiết thông tin sản phẩm, bảng giá các kích cỡ (`sizePrices`) và danh sách topping khả dụng (`toppings`) |
| `POST` | `/api/v1/orders` | Bearer JWT | Tạo đơn hàng mới từ giỏ hàng, trừ kho với khóa lạc quan, áp voucher |
| `GET` | `/api/v1/orders/me` | Bearer JWT | Lấy danh sách lịch sử đơn hàng của người dùng đang đăng nhập |
| `GET` | `/api/v1/orders/:id` | Bearer JWT | Xem chi tiết thông tin đơn hàng, danh sách món, trạng thái xử lý và giao dịch thanh toán |
| `POST` | `/api/v1/orders/:id/cancel` | Bearer JWT | Hủy đơn hàng khi còn ở trạng thái hợp lệ (`PENDING`/ `PAYMENT_FAILED`) và hoàn trả tồn kho |
| `POST` | `/api/v1/payments` | Bearer JWT + Idempotency | Thực hiện thanh toán trực tuyến an toàn với khóa Idempotency, gọi cổng thanh toán giả lập và cộng điểm loyalty |

*Ghi chú thiết kế API:* Nhằm tối ưu hóa hiệu năng truyền tải và hạn chế hiện tượng roundtrip mạng dư thừa, hệ thống tích hợp phân loại danh mục trực tiếp vào `GET /products` (thay vì tách riêng `GET /categories`), và nhúng danh sách topping khả dụng vào phản hồi chi tiết món `GET /products/:id` (thay vì tách riêng `GET /toppings`). Trạng thái giao dịch thanh toán được

truy vấn trực tiếp qua chi tiết đơn hàng `GET /orders/:id` thay vì tạo endpoint độc lập `GET /payments/:id`.

Mọi phản hồi thành công từ hệ thống đều được chuẩn hóa qua `TransformInterceptor` theo cấu trúc:

```
{
  "success": true,
  "data": { ... },
  "message": "Thong bao thanh cong",
  "timestamp": "2026-10-25T10:00:00.000Z"
}
```

Mọi phản hồi lỗi đều được chuẩn hóa qua `AllExceptionsFilter` theo cấu trúc:

```
{
  "success": false,
  "statusCode": 400,
  "message": "Thong tin loi chi tiet",
  "error": "Bad Request",
  "timestamp": "2026-10-25T10:00:00.000Z",
  "path": "/api/v1/orders"
}
```

# Chương C

# TỪ ĐIỂN DỮ LIỆU (DATA DICTIONARY)

Phụ lục này trình bày cấu trúc chi tiết của 12 bảng trong cơ sở dữ liệu PostgreSQL của BrewLite, bám sát tập tin mô hình dữ liệu `backend/prisma/schema.prisma`.

## C.1 Bảng `users`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính định danh người dùng |
| `email` | VARCHAR(255) | No | - | Địa chỉ email (UNIQUE) |
| `password_hash` | VARCHAR(255) | No | - | Mật khẩu băm Bcrypt 10 rounds |
| `full_name` | VARCHAR(100) | Yes | NULL | Họ và tên hiển thị |
| `loyalty_points` | INTEGER | No | 0 | Điểm thưởng tích lũy thành viên |
| `created_at` | TIMESTAMP | No | NOW() | Thời điểm tạo tài khoản |
| `updated_at` | TIMESTAMP | No | NOW() | Thời điểm cập nhật cuối cùng |

## C.2 Bảng `categories`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `name` | VARCHAR(100) | No | - | Tên phân loại (UNIQUE) |
| `slug` | VARCHAR(100) | No | - | Đường dẫn thân thiện (UNIQUE) |
| `display_order` | INTEGER | No | 0 | Thứ tự sắp xếp hiển thị tab |
| `created_at` | TIMESTAMP | No | NOW() | Thời điểm tạo |
| `updated_at` | TIMESTAMP | No | NOW() | Thời điểm cập nhật |

## C.3 Bảng `products`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `category_id` | VARCHAR(30) | Yes | NULL | Khóa ngoại → `categories.id` |
| `name` | VARCHAR(150) | No | - | Tên đồ uống |
| `base_price` | DECIMAL(12,0) | No | - | Đơn giá cơ bản (tiền chẵn VNĐ) |
| `image_url` | VARCHAR(500) | Yes | NULL | Đường dẫn hình ảnh |
| `stock` | INTEGER | No | 0 | Số lượng tồn kho khả dụng |
| `version` | INTEGER | No | 0 | Phiên bản Optimistic Locking |
| `is_available` | BOOLEAN | No | TRUE | Trạng thái mở bán |
| `created_at` | TIMESTAMP | No | NOW() | Thời điểm tạo |
| `updated_at` | TIMESTAMP | No | NOW() | Thời điểm cập nhật |

## C.4 Bảng `product_size_prices`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `product_id` | VARCHAR(30) | No | - | Khóa ngoại → `products.id` (Cascade) |
| `size` | ENUM | No | - | Giá trị: 'S', 'M', 'L' |
| `price_adjustment` | DECIMAL(12,0) | No | 0 | Phụ thu thêm theo size (UNIQUE:[product_id, size]) |

## C.5 Bảng `toppings`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `name` | VARCHAR(100) | No | - | Tên topping (UNIQUE) |
| `price` | DECIMAL(12,0) | No | - | Đơn giá topping |
| `stock` | INTEGER | No | 100 | Tồn kho topping |
| `is_available` | BOOLEAN | No | TRUE | Trạng thái kinh doanh |

## C.6 Bảng `promotions`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `code` | VARCHAR(50) | No | - | Mã voucher (UNIQUE) |
| `discount_percent` | INTEGER | Yes | NULL | % chiết khấu (ví dụ 10) |
| `discount_amount` | DECIMAL(12,0) | Yes | NULL | Tiền chiết khấu cố định |
| `max_discount` | DECIMAL(12,0) | Yes | NULL | Mức giảm tối đa (ví dụ 20.000đ) |
| `min_order_amount` | DECIMAL(12,0) | No | 0 | Giá trị đơn hàng tối thiểu |
| `is_active` | BOOLEAN | No | TRUE | Trạng thái kích hoạt |

## C.7 Bảng `orders`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `order_number` | SERIAL | No | autoincrement | Mã hiển thị khách hàng (UNIQUE) |
| `user_id` | VARCHAR(30) | No | - | Khóa ngoại → `users.id` (INDEX) |
| `promotion_id` | VARCHAR(30) | Yes | NULL | Khóa ngoại → `promotions.id` |
| `status` | ENUM | No | 'PENDING' | Trạng thái đơn (INDEX) |
| `total_amount` | DECIMAL(12,0) | No | - | Tạm tính trước giảm |
| `discount_amount` | DECIMAL(12,0) | No | 0 | Số tiền giảm giá |
| `final_amount` | DECIMAL(12,0) | No | - | Tiền thực trả |
| `points_earned` | INTEGER | No | 0 | Điểm loyalty tích được |
| `version` | INTEGER | No | 0 | Khóa lạc quan máy trạng thái |

## C.8 Bảng `order_items`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `order_id` | VARCHAR(30) | No | - | Khóa ngoại → `orders.id` (Cascade, INDEX) |
| `product_id` | VARCHAR(30) | No | - | Khóa ngoại → `products.id` |
| `size` | ENUM | No | 'M' | Kích cỡ: 'S', 'M', 'L' |
| `quantity` | INTEGER | No | 1 | Số lượng đặt |
| `unit_price` | DECIMAL(12,0) | No | - | Đơn giá snapshot tại thời điểm đặt |
| `line_total` | DECIMAL(12,0) | No | - | Thành tiền của dòng món |

## C.9 Bảng `order_item_toppings`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `order_item_id` | VARCHAR(30) | No | - | Khóa ngoại → `order_items.id` (Cascade) |
| `topping_id` | VARCHAR(30) | No | - | Khóa ngoại → `toppings.id` |
| `price` | DECIMAL(12,0) | No | - | Giá snapshot topping |

## C.10 Bảng `payments`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `order_id` | VARCHAR(30) | No | - | Khóa ngoại → `orders.id` (INDEX) |
| `idempotency_key` | VARCHAR(100) | No | - | Khóa chống trùng lặp (UNIQUE) |
| `request_hash` | VARCHAR(64) | Yes | NULL | Mã băm SHA-256 nội dung request |
| `amount` | DECIMAL(12,0) | No | - | Số tiền thanh toán |
| `method` | ENUM | No | - | 'BANK_CARD' hoặc 'E_WALLET' |
| `status` | ENUM | No | 'PENDING' | 'PENDING', 'SUCCESS', 'FAILED' |
| `transaction_ref` | VARCHAR(100) | Yes | NULL | Mã đối soát từ Gateway |

## C.11 Bảng `loyalty_transactions`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `user_id` | VARCHAR(30) | No | - | Khóa ngoại → `users.id` (INDEX) |
| `order_id` | VARCHAR(30) | Yes | NULL | Khóa ngoại → `orders.id` |
| `points` | INTEGER | No | - | Số điểm cộng hoặc trừ |
| `type` | ENUM | No | - | 'EARNED', 'SPENT', 'REFUNDED' |
| `reason` | VARCHAR(255) | No | - | Lý do biến động điểm |

## C.12 Bảng `order_status_history`

| Tên cột | Kiểu dữ liệu | Null | Mặc định | Mô tả / Ràng buộc |
|---|---|---|---|---|
| `id` | VARCHAR(30) | No | cuid() | Khóa chính |
| `order_id` | VARCHAR(30) | No | - | Khóa ngoại → `orders.id` (Cascade, INDEX) |
| `from_status` | ENUM | Yes | NULL | Trạng thái bắt đầu |
| `to_status` | ENUM | No | - | Trạng thái chuyển đến |
| `changed_by` | VARCHAR(50) | Yes | NULL | Tác nhân thực hiện (user, barista, system) |
| `reason` | VARCHAR(255) | Yes | NULL | Lý do chuyển trạng thái |
| `created_at` | TIMESTAMP | No | NOW() | Thời điểm ghi nhận |

# Chương D

# DANH MỤC CHI TIẾT CÁC CA KIỂM THỬ (TEST CASES)

Phụ lục này trình bày danh mục chi tiết toàn bộ các ca kiểm thử của hệ thống BrewLite được ánh xạ trực tiếp từ bộ tài liệu thiết kế tại thư mục `docs/`. Toàn bộ 93 ca kiểm thử tầng Backend (API, Unit, E2E, Concurrency) đều đạt kết quả **Đạt (Pass) tỷ lệ 100%** thông qua các bộ kiểm thử tự động của dự án. Toàn bộ 41 ca kiểm thử giao diện người dùng (UI) mang trạng thái **Chưa chạy (sẽ kiểm thử thủ công)** và sẽ do nhóm tự kiểm thử trực tiếp trên trình duyệt.

Bảng D.1: Danh mục đầy đủ các ca kiểm thử hệ thống BrewLite

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-AUTH-01 | Auth | Email chưa có trong DB | Gửi `POST /auth/register` hợp lệ | HTTP 201, tạo user mới, băm Bcrypt 10 rounds | HTTP 201, trả về user mới | **Đạt** |
| TC-AUTH-02 | Auth | Không có | Gửi `POST /auth/register` sai định dạng email hoặc password ngắn | HTTP 400 Bad Request, thông báo validation | HTTP 400, thông báo lỗi validation | **Đạt** |
| TC-AUTH-03 | Auth | Email đã có trong DB | Gửi `POST /auth/register` với email đã tồn tại | HTTP 409 Conflict, thông báo ”Email này đã được sử dụng” | HTTP 409 Conflict, đúng thông báo lỗi | **Đạt** |
| TC-AUTH-04 | Auth | Đã có tài khoản | Gửi `POST /auth/login` với đúng email và mật khẩu | HTTP 200, cấp Bearer JWT `accessToken` | HTTP 200, cấp JWT hợp lệ kèm thông tin user | **Đạt** |
| TC-AUTH-05 | Auth | Không có | Gửi `POST /auth/login` với sai mật khẩu hoặc email không tồn tại | HTTP 401 Unauthorized, thông báo sai thông tin | HTTP 401, từ chối đăng nhập | **Đạt** |
| TC-AUTH-06 | Auth | Vừa đăng ký | Truy vấn bản ghi trong bảng `users` qua Prisma | Trường `password_hash` khác chuỗi thô, băm 10 rounds | Khớp chuẩn băm Bcrypt 10 rounds | **Đạt** |
| TC-AUTH-07 | Auth | Không có token | Gửi request đến route bảo vệ (ví dụ: `POST /orders`) | HTTP 401 Unauthorized, ”Vui lòng đăng nhập để tiếp tục” | HTTP 401 Unauthorized | **Đạt** |
| TC-AUTH-08 | Auth | Token không hợp lệ | Gửi request với token hết hạn hoặc sai chữ ký | HTTP 401 Unauthorized, từ chối truy cập | HTTP 401 Unauthorized | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-AUTH-UI-01 | Auth | Chưa đăng nhập | Truy cập trực tiếp trang /orders | Hook useRequireAuth tự động chuyển hướng sang /auth/login | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-AUTH-UI-02 | Auth | Mở form đăng nhập | Quan sát màn hình /auth/login | Hiển thị đủ ô input email, password và nút điền nhanh demo | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-AUTH-UI-03 | Auth | Mở form đăng nhập | Bỏ trống email/password và bấm nút Đăng nhập | Báo lỗi validate đỏ dưới các trường tương ứng | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-AUTH-UI-04 | Auth | Điền tài khoản đúng | Bấm ”Đăng nhập” | Lưu token vào state và chuyển hướng về trang trước đó | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-AUTH-UI-05 | Auth | Mở form đăng ký | Nhập mật khẩu xác nhận không khớp | Hiển thị thông báo ”Mật khẩu xác nhận không khớp” | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-AUTH-UI-06 | Auth | Điền form đăng ký hợp lệ | Bấm nút ”Đăng ký tài khoản” | Tạo tài khoản thành công, tự động đăng nhập và về /menu | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-01 | Catalog | DB nạp 10 sản phẩm | Gửi GET /products không tham số | HTTP 200, danh sách 10 món kèm categoryName | HTTP 200, đủ 10 món Cà phê và Trà | **Đạt** |
| TC-CAT-02 | Catalog | DB nạp 10 sản phẩm | Gửi GET /products và kiểm tra trường giá | HTTP 200, các món có basePrice số nguyên VNĐ | HTTP 200, cấu trúc đơn giá chuẩn xác | **Đạt** |
| TC-CAT-03 | Catalog | DB nạp 10 sản phẩm | Gửi GET /products?search=bạc | HTTP 200, trả về sản phẩm Bạc xỉu | HTTP 200, lọc đúng món theo từ khóa | **Đạt** |
| TC-CAT-04 | Catalog | DB nạp 10 sản phẩm | Gửi GET /products và kiểm tra các trường ẩn | Không để lộ trường stock nội bộ trong danh mục | HTTP 200, không chứa trường stock | **Đạt** |
| TC-CAT-05 | Catalog | DB nạp 10 sản phẩm | Gửi GET /products và kiểm tra thứ tự sắp xếp | Sắp xếp theo tên món hoặc thứ tự displayOrder | HTTP 200, thứ tự sắp xếp chuẩn xác | **Đạt** |
| TC-CAT-UI-01 | Catalog | Truy cập trang chủ | Mở trang /menu | Hiển thị spinner loading sau đó hiện lưới thực đơn | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-UI-02 | Catalog | Mở trang thực đơn | Quan sát thanh phân loại danh mục | Hiển thị các tab: ”Tất cả”, ”Cà phê”, ”Trà & Trái cây” | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-UI-03 | Catalog | Đang ở /menu | Nhập ”Cà phê sữa” vào ô tìm kiếm | Lưới sản phẩm chỉ hiển thị món Cà phê sữa đá | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-UI-04 | Catalog | Đang ở /menu | Bấm chọn tab ”Cà phê” | Lọc và hiển thị đúng 6 món thuộc nhóm Cà phê | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-CAT-UI-05 | Catalog | Đang ở /menu | Bấm chọn tab ”Trà & Trái cây” | Lọc và hiển thị đúng 4 món thuộc nhóm Trà | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-UI-06 | Catalog | Đang ở /menu | Nhập từ khóa không tồn tại ”xyz123” | Hiển thị thông báo không tìm thấy món phù hợp | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CAT-UI-07 | Catalog | Đang ở /menu | Rê chuột vào thẻ món ăn | Thẻ món có hiệu ứng hover và nút xem chi tiết | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-DET-01 | Detail | Món tồn tại trong DB | Gửi GET /products/:id với ID hợp lệ | HTTP 200, nạp đủ sizePrices (S,M,L) và toppings | HTTP 200, đủ 3 size và danh sách topping | **Đạt** |
| TC-DET-02 | Detail | Món tồn tại trong DB | Gửi GET /products/:id kiểm tra giá topping | Danh sách toppings kèm giá: Trân châu, Kem Cheese, Đào miếng | HTTP 200, giá topping nguyên chẵn | **Đạt** |
| TC-DET-03 | Detail | Không có ID này | Gửi GET /products/:id với ID không tồn tại | HTTP 404 Not Found, ”Không tìm thấy sản phẩm” | HTTP 404 Not Found | **Đạt** |
| TC-DET-UI-01 | Detail | Mở /products/[id] | Quan sát giao diện trang chi tiết món | Hiển thị tên món, ảnh lớn, mô tả, nút chọn size/topping | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-DET-UI-02 | Detail | Ở trang chi tiết | Mặc định chọn Size S | Giá hiển thị bằng basePrice gốc của món | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-DET-UI-03 | Detail | Ở trang chi tiết | Bấm chọn Size M (+5.000đ) | Đơn giá tự động cộng thêm 5.000đ | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-DET-UI-04 | Detail | Ở trang chi tiết | Bấm chọn thêm Kem Cheese (+10.000đ) | Đơn giá tự động cộng thêm 10.000đ | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-DET-UI-05 | Detail | Đã chọn 3 topping | Cố gắng bấm chọn loại topping thứ 4 | Không cho chọn thêm, hiển thị thông báo tối đa 3 topping | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-01 | Cart | Giỏ hàng trống | Truy cập trang /cart | Hiển thị thông báo giỏ trống và nút Khám phá thực đơn | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-02 | Cart | Thêm món vào giỏ | Bấm ”Thêm vào giỏ hàng” tại trang chi tiết món | Icon giỏ hàng trên header cập nhật số lượng món | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-03 | Cart | Mở trang giỏ hàng | Quan sát danh sách các món đã thêm | Hiển thị tên món, size, topping và đơn giá từng món | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-04 | Cart | Có món trong giỏ | Bấm nút dấu cộng (+) tăng số lượng món | Số lượng tăng lên 1, tổng tiền tự động tính lại | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-05 | Cart | Món có số lượng > 1 | Bấm nút dấu trừ (-) giảm số lượng | Số lượng giảm 1, tổng tiền tự động trừ đi tương ứng | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-CART-UI-06 | Cart | Có món trong giỏ | Bấm icon thùng rác xóa hẳn một món | Món biến mất khỏi giỏ, tổng tiền cập nhật lại | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-07 | Cart | F5 tải lại trang | Refresh trình duyệt khi đang có giỏ hàng | Dữ liệu giỏ hàng vẫn giữ nguyên nhờ Zustand LocalStorage | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-CART-UI-08 | Cart | Giỏ có ít nhất 1 món | Bấm nút ”Tiến hành Đặt hàng” | Chuyển hướng người dùng sang trang `/checkout` | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-ST-01 | Order State | Đơn PENDING | Thanh toán thành công | PENDING → PAID | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-02 | Order State | Đơn PENDING | Thanh toán lỗi | PENDING → PAYMENT_ FAILED | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-03 | Order State | Đơn PENDING | Khách hủy đơn | PENDING → CANCELLED | Chuyển trạng thái hợp lệ, hoàn tồn kho | **Đạt** |
| TC-ST-04 | Order State | Đơn PAYMENT_ FAILED | Thử lại thanh toán | PAYMENT_ FAILED → PENDING | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-05 | Order State | Đơn PAYMENT_ FAILED | Khách hủy đơn | PAYMENT_ FAILED → CANCELLED | Chuyển trạng thái hợp lệ, hoàn tồn kho | **Đạt** |
| TC-ST-06 | Order State | Đơn PAID | Barista nhận đơn | PAID → PREPARING | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-07 | Order State | Đơn PAID | Hủy ở mức state machine/service | PAID → CANCELLED | Cho phép theo State Machine chuẩn | **Đạt** |
| TC-ST-08 | Order State | Đơn PREPAR- ING | Pha chế hoàn tất | PREPARING → READY | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-09 | Order State | Đơn READY | Giao đồ uống cho khách | READY → COMPLETED | Chuyển trạng thái hợp lệ | **Đạt** |
| TC-ST-10 | Order State | Đơn PENDING | Chuyển sang PREPARING khi chưa thanh toán | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-11 | Order State | Đơn PENDING | Nhảy cóc sang READY | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-12 | Order State | Đơn PENDING | Nhảy cóc sang COMPLETED | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-13 | Order State | PAYMENT_ FAILED | Bỏ qua PENDING chuyển thẳng sang PAID | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-14 | Order State | Đơn PAID | Quay lui về PENDING | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-ST-15 | Order State | Đơn PAID | Bỏ qua PREPARING chuyển thẳng sang READY | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-16 | Order State | Đơn PREPARING | Cố hủy đơn khi barista đang pha | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-17 | Order State | Đơn PREPARING | Quay lui về PAID | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-18 | Order State | Đơn READY | Cố hủy đơn khi đã pha xong | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-19 | Order State | Đơn COMPLETED | Chuyển về PAID | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-20 | Order State | Đơn COMPLETED | Cố hủy đơn đã hoàn tất | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-21 | Order State | Đơn CANCELLED | Phục hồi về PENDING | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-22 | Order State | Đơn CANCELLED | Chuyển sang PAID | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-23 | Order State | Đơn PENDING | Chuyển về chính nó (PENDING → PENDING) | Chặn, ném HTTP 409 Conflict | Chặn với HTTP 409 Conflict | **Đạt** |
| TC-ST-24 | Order State | Hai request đồng thời | Cùng đổi trạng thái đơn từ PENDING | Đúng 1 request thành công, request kia nhận 409 | 1 request thành công, 1 nhận 409 | **Đạt** |
| TC-CO-01 | Order | Đủ tồn kho, có JWT | Gửi `POST /orders` hợp lệ | HTTP 201 Created, đơn PENDING, trừ tồn kho chính xác | HTTP 201 Created | **Đạt** |
| TC-CO-02 | Order | Client gửi kèm giá | Body cố tình truyền trường `price` sai lệch | Server bỏ qua giá client, tự tính toán từ CSDL | Server tính đúng đơn giá CSDL | **Đạt** |
| TC-CO-03 | Order | Không có JWT | Gửi `POST /orders` không có token | HTTP 401 Unauthorized | HTTP 401 Unauthorized | **Đạt** |
| TC-CO-04 | Order | Token hết hạn | Gửi `POST /orders` với JWT quá hạn | HTTP 401 Unauthorized | HTTP 401 Unauthorized | **Đạt** |
| TC-CO-05 | Order | Giỏ hàng rỗng | Gửi `POST /orders` với `items: []` | HTTP 400 Bad Request, từ chối tạo đơn | HTTP 400 Bad Request | **Đạt** |
| TC-CO-06 | Order | Số lượng bằng 0 | Gửi `POST /orders` với `quantity = 0` | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-CO-07 | Order | Số lượng quá lớn | Gửi `POST /orders` với `quantity = 100` | HTTP 400 Bad Request (`@Max(99)`) | HTTP 400 Bad Request | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-CO-08 | Order | Số lượng biên trên | Gửi POST /orders với quantity = 99 | HTTP 201 nếu đủ kho | Tạo đơn thành công nếu đủ kho | **Đạt** |
| TC-CO-09 | Order | Size không hợp lệ | Gửi POST /orders với size = "XL" | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-CO-10 | Order | Món không tồn tại | Gửi POST /orders với productId bịa | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-CO-11 | Order | Topping không có | Gửi POST /orders với toppingId không tồn tại | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-CO-11a | Order | Quá 3 topping | Gửi POST /orders chọn 4 loại topping cho 1 ly | HTTP 400 Bad Request (@ArrayMaxSize(3)) | HTTP 400 Bad Request | **Đạt** |
| TC-CO-11b | Order | Trùng topping | Gửi body chứa cùng 1 topping ID 2 lần trong 1 ly | HTTP 400 Bad Request (@ArrayUnique) | HTTP 400 Bad Request | **Đạt** |
| TC-CO-12 | Order | Body chứa userId | Client cố tình nhồi userId của người khác | ValidationPipe loại bỏ, lấy ID từ JWT token | Lấy sub an toàn từ JWT | **Đạt** |
| TC-CO-13 | Order | Món hết tồn kho | Đặt món có quantity lớn hơn stock khả dụng | HTTP 400 Bad Request, tồn kho không âm | HTTP 400 Bad Request, kho giữ nguyên | **Đạt** |
| TC-CO-14 | Order | Món thứ 2 hết kho | Đơn 2 món, món 1 còn hàng, món 2 hết hàng | HTTP 400 Bad Request, rollback toàn bộ transaction | Rollback an toàn, kho món 1 không mất | **Đạt** |
| TC-CO-15 | Order | Stock = 1, tranh mua | 2 request đồng thời cùng mua món cuối | Đúng 1 đơn 201, đơn kia nhận 400, stock về 0 | 1 đơn 201, 1 đơn 400, stock = 0 | **Đạt** |
| TC-CO-16 | Order | Stock = 5, tranh mua | 10 request đồng thời cùng mua | 5 đơn 201 thành công, 5 đơn nhận 400, stock về 0 | Đúng 5 đơn thành công, stock = 0 | **Đạt** |
| TC-CO-17 | Order | Lỗi DB bất ngờ | Giả lập lỗi DB khi đang chèn bản ghi | Rollback transaction toàn bộ, không rò rỉ dữ liệu | Rollback sạch sẽ | **Đạt** |
| TC-CO-18 | Order | Áp mã WELCOME10 | Áp mã WELCOME10 cho đơn từ 40.000đ | Giảm 10% (tối đa 20.000đ), tính đúng finalAmount | Giảm 10% chuẩn xác | **Đạt** |
| TC-CO-19 | Order | Áp mã sai điều kiện | Áp mã khi giá trị đơn chưa đạt minOrderAmount | HTTP 400 Bad Request, từ chối áp dụng mã | HTTP 400 Bad Request | **Đạt** |
| TC-CX-01 | Order Cancel | Đơn PENDING | Gửi POST /orders/:id/cancel | HTTP 200 OK, trạng thái CANCELLED, hoàn kho sản phẩm | HTTP 200 OK, hoàn kho trong transaction | **Đạt** |
| TC-CX-02 | Order Cancel | PAYMENT_FAILED | Gửi POST /orders/:id/cancel | HTTP 200 OK, trạng thái CANCELLED, hoàn kho sản phẩm | HTTP 200 OK, hoàn kho thành công | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-CX-03 | Order Cancel | Đơn đã PAID | Cố tình gửi POST /orders/:id/cancel khi đơn đã trả tiền | HTTP 409 Conflict, chặn không cho hủy | HTTP 409 Conflict | **Đạt** |
| TC-CX-04 | Order Cancel | Đơn của người khác | Gửi POST /orders/:id/cancel mã đơn của tài khoản khác | HTTP 404 Not Found, không làm lộ sự tồn tại của đơn | HTTP 404 Not Found | **Đạt** |
| TC-CX-05 | Order Cancel | Chưa đăng nhập | Gửi request hủy đơn không mang Bearer token | HTTP 401 Unauthorized | HTTP 401 Unauthorized | **Đạt** |
| TC-ORD-UI-01 | Order | Đang ở /checkout | Quan sát màn hình xác nhận đơn hàng | Hiển thị đủ danh sách món, phụ thu size, topping, tạm tính | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-ORD-UI-02 | Order | Đang ở /checkout | Nhập mã WELCOME10 vào ô voucher | Hiển thị dòng giảm giá màu xanh và cập nhật finalAmount | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-ORD-UI-03 | Order | Đang ở /checkout | Bấm nút ”Xác nhận đặt hàng” | Gọi API tạo đơn, chuyển hướng người dùng sang trang thanh toán | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-ORD-UI-04 | Order | Đang ở /checkout | Món trong giỏ vừa bị người khác mua hết | Hiển thị toast cảnh báo hết hàng, không chuyển trang | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-PAY-01 | Payment | Đơn PENDING, ví điện tử | Gửi POST /payments với method = "E_WALLET" | Payment SUCCESS, Order PAID, xóa rỗng giỏ | HTTP 201, Payment SUCCESS | **Đạt** |
| TC-PAY-02 | Payment | Đơn PENDING, thẻ ngân hàng | Gửi POST /payments với method = "BANK_CARD" | Payment SUCCESS, Order PAID, xóa rỗng giỏ | HTTP 201, Payment SUCCESS | **Đạt** |
| TC-PAY-03 | Payment | Gateway trả lỗi | Header X-Mock-Outcome: FAILED | Payment FAILED, Order PAYMENT_FAILED | HTTP 201, Payment FAILED | **Đạt** |
| TC-PAY-04 | Payment | Thiếu orderId | Body không có trường orderId | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-PAY-05 | Payment | Thiếu method | Body không có trường method | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-PAY-06 | Payment | method sai chuẩn | Body gửi method = "CASH" | HTTP 400 Bad Request (@IsEnum) | HTTP 400 Bad Request | **Đạt** |
| TC-PAY-07 | Payment | Order không tồn tại | Gửi POST /payments với ID đơn bịa | HTTP 404 Not Found, không gọi gateway | HTTP 404 Not Found | **Đạt** |
| TC-PAY-08 | Payment | Đơn của người khác | Gửi POST /payments cho đơn tài khoản khác | HTTP 404 Not Found, từ chối thanh toán | HTTP 404 Not Found | **Đạt** |
| TC-PAY-09 | Payment | Đơn đã thanh toán | Gửi POST /payments cho đơn đã ở trạng thái PAID | HTTP 409 Conflict, ”Trạng thái không cho phép” | HTTP 409 Conflict | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-PAY-10 | Payment | Cùng key, cùng nội dung | Gửi lại request trùng Idempotency-Key và body | Trả lại kết quả cũ đã lưu (HTTP 201 Created), không tạo giao dịch trùng | HTTP 201 Created, trả kết quả cũ đã lưu | **Đạt** |
| TC-PAY-11 | Payment | Cùng key, khác payload | Gửi trùng key nhưng thay đổi method | HTTP 409 Conflict (requestHash không khớp) | HTTP 409 Conflict | **Đạt** |
| TC-PAY-12 | Payment | 5 request cùng key đồng thời | Bắn song song 5 request thanh toán cùng key | Chỉ 1 giao dịch được tạo, request thua cuộc nhận kết quả đã lưu (HTTP 201 Created) | Đúng 1 payment trong DB, 5 request nhận 201 | **Đạt** |
| TC-PAY-13 | Payment | Thử lại sau thất bại | Đơn PAYMENT_FAILED, gửi payment mới với key mới | Chuyển Order sang PENDING → PAID thành công | Tạo payment mới thành công | **Đạt** |
| TC-PAY-14 | Payment | Gateway timeout | Giả lập gateway timeout | Ghi nhận Payment FAILED, Order PAYMENT_FAILED | Ghi nhận Payment FAILED chuẩn xác | **Đạt** |
| TC-PAY-15 | Payment | Thử lại sau timeout | Gửi request mới với key mới sau timeout | Chuyển Order PENDING → PAID thành công | Thanh toán thành công | **Đạt** |
| TC-PAY-16 | Payment | Header test giả lập | Header X-Mock-Outcome: FAILED ở dev | Mock gateway giả lập thất bại an toàn | Mock gateway trả FAILED | **Đạt** |
| TC-PAY-17 | Payment | Frontend gửi sai tiền | Client cố tình truyền số tiền khác | Backend tự lấy từ Order.finalAmount | Tiền luôn lấy từ CSDL | **Đạt** |
| TC-PAY-18 | Payment | Thử nhiều lần | Lần 1 thất bại, lần 2 thành công | Lưu đủ lịch sử các lần thử (1-N), Order PAID | Lưu đủ lịch sử các lần thanh toán | **Đạt** |
| TC-PAY-19 | Payment | Header giả lập ở Prod | NODE_ENV = 'production' | Backend bỏ qua header test, xử lý bình thường | Bỏ qua header ở môi trường production | **Đạt** |
| TC-PAY-UI-01 | Payment | Tại trang thanh toán | Quan sát màn hình /checkout/payment | Hiển thị tổng tiền và 2 tùy chọn Thẻ ngân hàng / Ví điện tử | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-PAY-UI-02 | Payment | Chọn phương thức | Bấm chọn Ví điện tử và bấm nút "Thanh toán ngay" | Hiển thị loading và chuyển sang trang thành công | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-PAY-UI-03 | Payment | Thanh toán thành công | Chuyển hướng về trang lịch sử /orders | Xuất hiện banner chúc mừng và đơn mang trạng thái PAID | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-01 | History | User đã có đơn | Gửi GET /orders/me có kèm Bearer token | HTTP 200, danh sách đơn của chính mình xếp mới nhất | HTTP 200, trả về đủ danh sách đơn | **Đạt** |
| TC-HIS-02 | History | User mới chưa có đơn | Gửi GET /orders/me từ tài khoản mới | HTTP 200, mảng đơn rỗng items: [] | HTTP 200, mảng rỗng | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TC-HIS-03 | History | Có Bearer JWT | Gửi GET /orders/me lọc status=PAID | HTTP 200, chỉ trả về các đơn có trạng thái PAID | HTTP 200, lọc đúng trạng thái | **Đạt** |
| TC-HIS-04 | History | Có Bearer JWT | Gửi GET /orders/me với trạng thái sai | HTTP 400 Bad Request | HTTP 400 Bad Request | **Đạt** |
| TC-HIS-05 | History | Đơn của chính mình | Gửi GET /orders/:id xem chi tiết đơn hàng | HTTP 200, kèm mảng items, toppings và payment | HTTP 200, chi tiết đơn đầy đủ | **Đạt** |
| TC-HIS-06 | History | Đơn của user khác | Gửi GET /orders/:id xem đơn của tài khoản khác | HTTP 404 Not Found, không làm lộ đơn người khác | HTTP 404 Not Found | **Đạt** |
| TC-HIS-07 | History | Đơn có chuyển trạng thái | Kiểm tra bảng order_status_history | Ghi nhận đủ lịch sử chuyển: NULL → PENDING → PAID | Ghi vết đầy đủ các mốc thời gian | **Đạt** |
| TC-HIS-UI-01 | History | Chưa đăng nhập | Truy cập trực tiếp /orders | Tự động chuyển hướng về /auth/login?next=/orders | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-02 | History | Mở /orders | User chưa có đơn hàng nào | Hiển thị "Chưa có đơn hàng nào" và nút Khám phá thực đơn | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-03 | History | Mở /orders | Bấm chọn tab "Chờ thanh toán" hoặc "Đã hủy" | Lọc và tải lại danh sách đơn tương ứng với tab | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-04 | History | Danh sách > 1 trang | Bấm nút "Trang sau" ở chân danh sách đơn | Tăng page, tải và hiển thị trang tiếp theo | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-05 | History | Tại /orders | Bấm nút "Xem chi tiết" của một đơn hàng | Mở trang chi tiết đơn với đầy đủ thông tin món và thanh toán | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-06 | History | Vừa thanh toán xong | URL chứa query ?paid=1 | Hiển thị banner màu xanh chúc mừng "Thanh toán thành công!" | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-07 | History | Quan sát trạng thái | Kiểm tra component badge trạng thái | Hiển thị đúng nhãn: PENDING, PAID, CANCELLED... | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TC-HIS-UI-08 | History | Đơn PENDING | Bấm nút "Hủy đơn" tại màn hình chi tiết | Hiển thị xác nhận, hủy thành công badge cập nhật Đã hủy | **[TODO: điền kết quả thủ công]** | *Chưa chạy* |
| TS-FR-07-02 | Task 10 | Đơn PAID hoặc CANCELLED | Chuyển trạng thái đơn hàng trái quy tắc Máy trạng thái | Chặn chuyển trạng thái, trả về HTTP 409 Conflict | Chặn thành công với HTTP 409 Conflict | **Đạt** |

Tiếp tục ở trang sau...

**Bảng D.1 – Tiếp tục từ trang trước**

| Mã TC | Phân hệ | Tiền điều kiện | Bước thực hiện | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|---|---|
| TS-FR-08-01 | Task 10 | 5 requests song song | Bắn 5 request POST /payments đồng thời cùng key | Đúng 1 bản ghi thanh toán trong CSDL, không có mã lỗi 500 | Đúng 1 bản ghi Payment, 0 lỗi 500, replay an toàn | **Đạt** |
| TS-FR-09-01 | Task 10 | 10 requests tranh chấp | Bắn 10 request POST /orders song song khi stock = 1 | Đúng 1 đơn thành công (201), 9 đơn bị chặn (400), stock = 0 | 1 đơn 201 Created, 9 đơn 400 Bad Request, stock = 0 | **Đạt** |